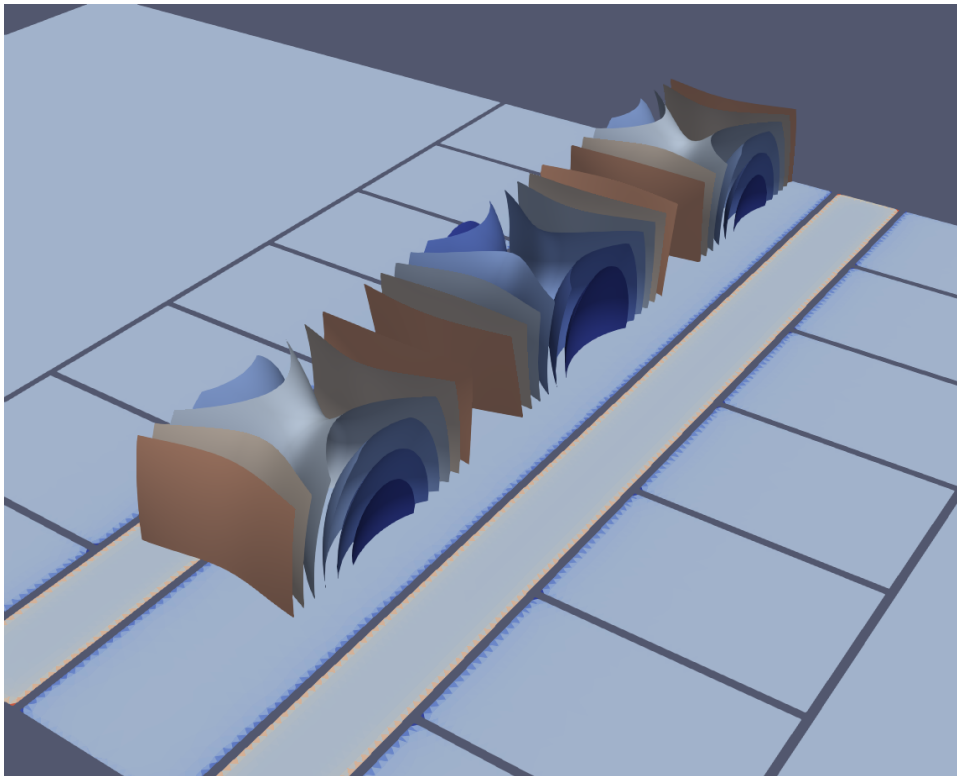




# **Nullspace ES 2024 R1**

## **User's Guide**



*Nullspace, Inc.*

---

July 23, 2024

# Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                        | <b>1</b> |
| 1.1      | Nullspace ES                               | 1        |
| 1.2      | System Requirements                        | 1        |
| 1.3      | Dependencies                               | 2        |
| 1.4      | Inputs                                     | 2        |
| 1.5      | Outputs                                    | 2        |
| 1.6      | General Workflow                           | 2        |
| <b>2</b> | <b>CAD Modeling and Meshing</b>            | <b>3</b> |
| <b>3</b> | <b>Creating and Editing Input Decks</b>    | <b>3</b> |
| 3.1      | MESH Tag Group                             | 4        |
| 3.1.1    | FILE Tag                                   | 4        |
| 3.1.2    | SCALE Tag                                  | 4        |
| 3.2      | MESH_QUALITY Tag Group                     | 5        |
| 3.2.1    | SHAPE and CONDITION NUMBER Tags            | 5        |
| 3.2.2    | REPORT Tag                                 | 6        |
| 3.3      | ELECTRODE_EXCITATION Tag Group             | 6        |
| 3.3.1    | FILE Tag                                   | 6        |
| 3.3.2    | LIST Tag                                   | 7        |
| 3.4      | SYMMETRY Tag Group                         | 7        |
| 3.4.1    | SYMMETRY_PLANE_XY Tag                      | 7        |
| 3.4.2    | SYMMETRY_PLANE_XY_OFFSET Tag               | 7        |
| 3.4.3    | SYMMETRY_PLANE_XY_WIDTH Tag                | 7        |
| 3.4.4    | SYMMETRY_PLANE_XY_N Tag                    | 7        |
| 3.4.5    | SYMMETRY_PLANE_XZ Tag                      | 7        |
| 3.4.6    | SYMMETRY_PLANE_XZ_OFFSET Tag               | 8        |
| 3.4.7    | SYMMETRY_PLANE_YZ Tag                      | 8        |
| 3.4.8    | SYMMETRY_PLANE_YZ_OFFSET Tag               | 8        |
| 3.4.9    | SYMMETRY_ROTATION_MODE Tag                 | 8        |
| 3.4.10   | SYMMETRY_ROTATION_Z Tag                    | 8        |
| 3.4.11   | Symmetry Plane Concepts                    | 8        |
| 3.4.12   | z-Rotational Symmetry Concepts             | 12       |
| 3.4.13   | z-Double Ground Plane Concepts             | 15       |
| 3.4.14   | Symmetry Combinations                      | 15       |
| 3.4.15   | Using Symmetry with Asymmetric Excitations | 16       |
| 3.5      | SOLVER Tag Group                           | 19       |
| 3.5.1    | PRECISION Tag                              | 20       |
| 3.5.2    | SYSTEM Tag                                 | 20       |
| 3.5.3    | FILL_COMPRESS_TOLERANCE Tag                | 20       |
| 3.5.4    | FILL_RECOMPRESS Tag                        | 20       |
| 3.5.5    | FILL_RECOMRPRESS_TOLERANCE Tags            | 21       |
| 3.5.6    | SOLVE_COMPRESS_TOLERANCE Tag               | 21       |
| 3.5.7    | ACA_FILL_RESTART Tags                      | 21       |
| 3.5.8    | SCALE_NEAR_BLOCKS                          | 21       |
| 3.5.9    | COMPRESS_ALL_OFF_DIAGONAL_BLOCKS           | 22       |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 3.6      | CLUSTERS Tag Group . . . . .                                 | 22        |
| 3.6.1    | CLUSTER_MAX_LEVEL Tag . . . . .                              | 22        |
| 3.6.2    | CLUSTER_MINIMAL_SIZE Tag . . . . .                           | 22        |
| 3.6.3    | CLUSTER_MINIMAL_COMPRESSION_SIZE Tag . . . . .               | 22        |
| 3.6.4    | CLUSTER_DISTANCE_METHOD Tag . . . . .                        | 22        |
| 3.6.5    | CLUSTER_COLLECTIONS Tag . . . . .                            | 23        |
| 3.7      | OUTPUT Tag Group . . . . .                                   | 23        |
| 3.7.1    | CHARGE_VTU Tag . . . . .                                     | 23        |
| 3.7.2    | GRID Tag . . . . .                                           | 23        |
| 3.7.3    | POTENTIAL Tag . . . . .                                      | 24        |
| 3.7.4    | ELETRIC_FIELD Tag . . . . .                                  | 24        |
| 3.7.5    | XMF Tag . . . . .                                            | 24        |
| 3.8      | Common Input Deck Usages . . . . .                           | 24        |
| 3.8.1    | Dense Linear System . . . . .                                | 24        |
| 3.8.2    | Multi-Run Workflows . . . . .                                | 26        |
| <b>4</b> | <b>Grid File Preparation</b>                                 | <b>27</b> |
| 4.1      | 3DRECTMESH . . . . .                                         | 28        |
| 4.2      | POLYVERTEX . . . . .                                         | 28        |
| <b>5</b> | <b>Nullspace ES Execution</b>                                | <b>29</b> |
| 5.1      | Standard Out . . . . .                                       | 30        |
| 5.2      | Diagnostic Output . . . . .                                  | 30        |
| <b>6</b> | <b>Post-processing Analysis</b>                              | <b>32</b> |
| 6.1      | Run-time Performance Information . . . . .                   | 32        |
| 6.2      | Engineering Data Files and Post Processing Scripts . . . . . | 33        |
| <b>7</b> | <b>Examples</b>                                              | <b>36</b> |
| 7.1      | Disk . . . . .                                               | 37        |
| 7.1.1    | Step 1. Nullspace Prep mesh model . . . . .                  | 37        |
| 7.1.2    | Step 2. Grids . . . . .                                      | 38        |
| 7.1.3    | Step 3. Input Deck . . . . .                                 | 39        |
| 7.1.4    | Step 4. Nullspace ES Execution . . . . .                     | 40        |
| 7.1.5    | Step 5. Post-processing Analysis . . . . .                   | 41        |
| 7.1.6    | Step 1. Nullspace Prep mesh model . . . . .                  | 48        |
| 7.1.7    | Step 2. Grids . . . . .                                      | 49        |
| 7.1.8    | Step 3. Input Deck . . . . .                                 | 49        |
| 7.1.9    | Step 4. Nullspace ES Execution . . . . .                     | 49        |
| 7.1.10   | Step 5. Post-processing Analysis . . . . .                   | 51        |
| 7.2      | Sphere . . . . .                                             | 52        |
| 7.2.1    | Step 1. Nullspace Prep mesh model . . . . .                  | 52        |
| 7.2.2    | Step 2. Grids . . . . .                                      | 54        |
| 7.2.3    | Step 3. Input Deck . . . . .                                 | 54        |
| 7.2.4    | Step 4. Nullspace ES Execution . . . . .                     | 55        |
| 7.2.5    | Step 5. Post-processing Analysis . . . . .                   | 57        |
| 7.2.6    | Step 1. Nullspace Prep mesh model . . . . .                  | 57        |
| 7.2.7    | Step 2. Grids . . . . .                                      | 59        |



|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 7.2.8    | Step 3. Input Deck . . . . .                         | 60        |
| 7.2.9    | Step 4. Nullspace ES Execution . . . . .             | 60        |
| 7.2.10   | Step 5. Post-processing Analysis . . . . .           | 62        |
| 7.3      | Planar Ion Trap . . . . .                            | 64        |
| 7.3.1    | Step 1. Nullspace Prep Mesh Model . . . . .          | 64        |
| 7.3.2    | Step 2. Grid File . . . . .                          | 65        |
| 7.3.3    | Step 3. Input Deck . . . . .                         | 66        |
| 7.3.4    | Step 4. Nullspace ES Execution . . . . .             | 67        |
| 7.3.5    | Step 5. Post-processing Analysis . . . . .           | 69        |
| 7.3.6    | Step 6. Second Grid File . . . . .                   | 72        |
| 7.3.7    | Step 7. Second Input Deck . . . . .                  | 72        |
| 7.3.8    | Step 8. Nullspace ES Execution . . . . .             | 73        |
| 7.3.9    | Step 9. Secondary Post-processing Analysis . . . . . | 75        |
| 7.4      | Ring Ion Trap . . . . .                              | 79        |
| 7.4.1    | Step 1. Nullspace Prep Mesh Model . . . . .          | 79        |
| 7.4.2    | Step 2. Grid File . . . . .                          | 80        |
| 7.4.3    | Step 3. Input Deck . . . . .                         | 81        |
| 7.4.4    | Step 4. Nullspace ES Execution . . . . .             | 82        |
| 7.4.5    | Step 5. Post-processing Analysis . . . . .           | 84        |
| <b>A</b> | <b>Nullspace Prep Meshing</b>                        | <b>87</b> |
| <b>B</b> | <b>Adaptive Cross Approximation (ACA)</b>            | <b>88</b> |
| B.1      | Troubleshooting . . . . .                            | 88        |
| <b>C</b> | <b>Reduced Singular Value Decomposition (rSVD)</b>   | <b>89</b> |
| <b>D</b> | <b>Nullspace ES Memory Requirements</b>              | <b>89</b> |



# 1 Introduction

Nullspace ES is a computational electromagnetics (CEM) simulation software suite based on cutting edge techniques and rigorous mathematical formulation. It is a single simulation tool capable of many types of analysis. Nullspace ES is based on the Method of Moments (MoM) and implements a Surface Integral Equation (SIE) formulation on a triangular mesh.

## 1.1 Nullspace ES

Nullspace ES is intended to solve static scalar electric potential problems. Consider a perfectly conducting surface ( $S$ ) which is charged to a potential  $V_0$ . The unknown surface charge density distribution  $\rho(\mathbf{r})$  may then be determined on  $S$  by solving the integral equation

$$V_0 = \frac{1}{4\pi\epsilon_0} \int_S \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} dS' \quad (1)$$

In addition to solving (1) for the surface charge density, a user may also solve for the the potential at points in space given the surface charge density has already been determined. To that end, we have

$$V(\mathbf{r}) = \frac{1}{4\pi\epsilon_0} \int_S \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} dS'. \quad (2)$$

Further, a user may also desire to solve for the electric fields at points in space as given by

$$\mathbf{E}(\mathbf{r}) = \frac{1}{4\pi\epsilon_0} \int_S \frac{\rho(\mathbf{r}')(\mathbf{r} - \mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|^3} dS'. \quad (3)$$

The solution to (1-3) requires an underlying discretization of the geometry (i.e., mesh model). Mesh models are composed of *blocks* (or *surfaces* or *groups*) of planar triangles. In essence all blocks can be thought of as potential surfaces. By default, all potential surfaces have a 0 constant potential. The model is *excited* by setting one or more blocks to a nonzero potential (typically 1.0 V). Such blocks are referred to as *electrodes*, and we say the model is *excited* by an electrode block, or perhaps an electrode block is *turned on*. The electrostatics problem is to then solve for the surface charge densities given a certain electrode is excited while all other blocks are set to a 0 constant potential. Specifying which electrodes to excite, and additional output quantities, are all configurable.

## 1.2 System Requirements

Nullspace ES supports the following operating systems :

- Red Hat Enterprise Linux 8.x, 9.x
- Ubuntu 20.04, 22.04

Nullspace ES generally supports Intel and AMD multicore/many-core 64-bit architectures. Nullspace ES is not intended to be used on 32-bit platforms. In general, performance can be increased using faster processors with increasing amounts of RAM. Nullspace ES is not supported on any type of virtual machine.



### 1.3 Dependencies

Nullspace ES is distributed with all resources required to execute the software. It uses ABAQUS mesh files to describe the CEM-friendly mesh. Nullspace recommends the use of Nullspace Prep for mesh creation and export. Nullspace ES provides generically formatted output data that can be used with any variety of post-processing tools. Your Nullspace ES license comes with a number of Python post-processing functions that will be discussed later.

Nullspace ES makes use of Python 3 ( $\geq 3.9$ ) for post-processing. It is recommended to create a Python environment specific to Nullspace ES. One can use the virtualenv package, conda, or equivalent to create and manage environments. Your installation comes with a “whl” file that can be installed into your environment. The whl file is located in:

```
/opt/nse/processing
```

assuming the default installation location. By installing the whl file, all other dependencies will be resolved. Using conda, this can be set up as follows:

```
conda create --name nse
conda activate nse
pip install /opt/nse/processing/nse-2024.1.0-py3-none-any.whl
```

### 1.4 Inputs

Nullspace ES uses a simple text file specification for all inputs. In general, NSE requires a mesh file and an input file (also referred to as an input deck) that specifies a variety of simulation setup information.

### 1.5 Outputs

Nullspace ES supports three standard types of output data:

- Surface charge density  $\rho(\mathbf{r})$
- Potential  $\phi(\mathbf{r})$  on a grid of points
- E-fields  $\mathbf{E}(\mathbf{r})$  on a grid of points

The NSE input deck is used to configure desired output as well as electrode excitations. These outputs are considered fundamental outputs. The Nullspace ES installation includes several post-processing functions that can be used to access NSE output data. All of the examples incorporate the post-processing functions into the workflows to give a complete understanding of Nullspace ES capabilities.

### 1.6 General Workflow

Regardless of the problem type, there is a general Nullspace ES simulation workflow that involves the following generic steps.

- CAD modeling and meshing
- Nullspace ES input deck configuration



- Optional grid file preparation
- Nullspace ES execution
- Post-processing analysis

Though these are general steps required for each Nullspace ES simulation, the actual implementation of the steps can vary significantly from model to model. The examples at the end of this guide will thoroughly demonstrate this workflow. In the following sections, a brief overview of each element of this workflow is given, including a complete schematic of the Nullspace ES input deck.

## 2 CAD Modeling and Meshing

All Nullspace ES simulations are predicated on an underlying physical model. The physical model is typically designed in a separate CAD software, then meshed and exported for use in NSES. The NSES input deck provides a mesh tag that directs Nullspace ES where to find the associated mesh.

Mesh models are crucial to good CEM simulations. It is imperative to create a CEM-friendly mesh for each simulation. In the discussions to follow, and primarily in the detailed examples, care is taken to explain mesh pitfalls and point out useful meshing tips. The Nullspace ES installation comes with Nullspace Prep, a sophisticated CAD and meshing software. All examples are presented in the context of Nullspace Prep. If you purchased a license of Nullspace Prep, it is highly recommended to make use of this software in your workflow. After reading this guide it may be useful to reference Appendix A : Nullspace Prep Mesh as a guide for generating a CEM-friendly mesh with Nullspace Prep. Most other meshing software will have similar commands and capabilities.

## 3 Creating and Editing Input Decks

Nullspace ES simulations are configured with an input deck, which uses a tagged formatting scheme syntactically similar to XML. The input deck is comprised of the following group tag configuration sections:

- Mesh Configuration
- Mesh Quality Configuration
- Electrode Excitation Configuration
- Solver Configuration
- Cluster Configuration
- Output Configuration

Each group has child configuration tags related to the group. A group tag has the following format.

```
<GroupTag_BEG>
  left-value = right-value
<GroupTag_END>
```



**GroupTag** refers to a particular set of functionality within Nullspace ES. The left-value is always a well-defined tag name specific to the group. The right-value takes on the context of the left-value and can be any of the following types.

- scalar
- list of scalars
- string
- list of strings
- well-defined tag name

Comments may be placed in the input deck with the (#) character. The tagged sections need not follow a particular ordering.

In the sections to follow we will give a thorough description of each configuration group and all child tags in each group. There are several example simulation walk-throughs at the end of the document that contain complete input deck formulations. You may obtain an example list of all possible Nullspace ES tags. At the command line, use the `--inputDeck` input argument. This will cause Nullspace ES to generate on screen all currently usable tag sets without running a simulation.

```
nres --inputDeck
```

## 3.1 MESH Tag Group

Nullspace ES requires an Abaqus mesh file or ACAD facet file that satisfies the criteria given in the CAD Modeling and Meshing section. The mesh tag section is given below with comments explaining usage.

```
<MESH_BEG>          # The mesh begin tag is required.  
  FILE = sphere.inp  # The Abaqus/Facet mesh file is required.  
  SCALE = 1          # The mesh scale is optional (default is 1.0).  
<MESH_END>          # The mesh end tag is required.
```

### 3.1.1 FILE Tag

The FILE tag supports an absolute path or a relative path to the mesh file. A relative path is with respect to the current working directory.

### 3.1.2 SCALE Tag

The SCALE tag is a multiplicative factor used to scale the mesh to a unit meter coordinate system. For example, if the mesh file is in millimeters, then  $SCALE = 0.001$  to convert the mesh to meters.





## 3.2 MESH\_QUALITY Tag Group

Nullspace ES integrates with the Verdict mesh quality library, the same mesh quality engine used in VTK and Nullspace Prep. For Nullspace ES, the two primary mesh quality metrics are shape and condition number. Nullspace ES supports a tag set dedicated to querying these two quality metrics as shown below.

```
<MESH_QUALITY_BEG>
  REPORT = OFF                                # (Default) and optional.
                                              # ON will generate quality report and terminate
                                              # Nullspace ES.
  SHAPE_MIN = 0.25                            # (Default) and optional.
  SHAPE_MAX = 1.0                             # (Default) and optional.
  SHAPE_SEVERITY = 0                          # (Default) and optional :
                                              # 0 implies warning :
                                              # 1 implies error and exit.
  CONDITION_NUMBER_MIN = 1.0                  # (Default) and optional.
  CONDITION_NUMBER_MAX = 3.0                  # (Default) and optional.
  CONDITION_NUMBER_SEVERITY = 0                # (Default) and optional :
                                              # 0 implies warning :
                                              # 1 implies error and exit.
<MESH_QUALITY_END>
```

### 3.2.1 SHAPE and CONDITION NUMBER Tags

The MESH\_QUALITY group tag and all tags within it are optional. However, Nullspace ES always checks mesh quality after loading the mesh. If the MESH\_QUALITY tags are not in the input deck then Nullspace ES will use the indicated default settings to query each planar triangle in the mesh against the shape metric and the condition number metric. The default metric bounds for both shape and condition number should indicate a good quality mesh. The user can adjust the metric bounds for each using the SHAPE\_MIN, SHAPE\_MAX tags or the CONDITION\_NUMBER\_MIN, CONDITION\_NUMBER\_MAX tags. In most cases there is no need to change the default settings. They are provided to aid in more detailed diagnostics of the mesh.

As Nullspace ES processes the input mesh it will collect all planar triangle id's that violate the shape and condition number metrics. If there are no violations, Nullspace ES will issue the following message.

```
Shape metric minimum (0.25) and maximum (1) passed for all triangles
Condition number metric minimum (1) and maximum (3) passed for all triangles
```

The {0.25, 1.0} are the configured shape metric bounds (default) and {1.0, 3.0} are the configured condition number bounds (default). If there are more than one element violations then each list (shape and condition number) will be sorted in ascending order.

How should Nullspace ES treat mesh quality violations? Let's suppose we change the default metric bounds with SHAPE\_MIN = 0.7 and CONDITION\_NUMBER\_MAX = 1.1. These are rather unrealistic bounds for mesh quality, but they are used to illustrate a point. Under such conditions, Nullspace ES might produce messages such as

```
Element (463941) violates shape metric minimum (0.7) with shape (0.4996)
Element (228610) violates condition number metric maximum (1.1) with condition
number (1.101)
```

After Nullspace ES issues the metric violation messages for all planar triangle violations in the mesh, it can either exit or continue. The default operation is to continue but you can force Nullspace ES to exit



whenever it finds one or more violations. This control is given via the SHAPE\_SEVERITY and CONDITION\_NUMBER\_SEVERITY tags. By default both are 0, and as shown in the comments for the tags, Nullspace ES will only warn and not exit. If one or both of the severity tags are set to 1 and Nullspace ES finds a violation for the metric, Nullspace ES will terminate after reporting on all violating elements.

### 3.2.2 REPORT Tag

The REPORT tag provides the user an opportunity for deeper offline mesh analysis. The REPORT tag is OFF by default, in which case no mesh quality report is generated. If REPORT = ON then Nullspace ES will load the input mesh and generate a metric report on every element in the mesh. The output report might look like the following.

| Native ID | Shape | Condition |
|-----------|-------|-----------|
| 43618     | 1.00  | 1.00      |
| 43619     | 1.00  | 1.00      |
| 43620     | 1.00  | 1.00      |
| 43621     | 1.00  | 1.00      |
| 43622     | 1.00  | 1.00      |
| 43623     | 1.00  | 1.00      |
| 43624     | 1.00  | 1.00      |
| 43625     | 1.00  | 1.00      |
| 43626     | 1.00  | 1.00      |
| 43627     | 1.00  | 1.00      |

After Nullspace ES generates the report it will issue the following message :

```
Printing mesh quality report ...
... exiting by design due to mesh quality report tag
```

As indicated by the message, Nullspace ES will then terminate.

## 3.3 ELECTRODE\_EXCITATION Tag Group

The electrode excitation group is a means to specify a list of electrodes to be turned on, in sequence, with each one yielding a *solution* of surface charge densities to the electrostatics system. The list of electrodes can be given as scalar list or specified in a separate flat ASCII file, also as a scalar list. In both cases the list can be space or comma delimited. Structurally, the electrode excitation group tag and all child tags are illustrated below. The electrode excitation group is optional. If it isn't specified then all blocks in the mesh file are designated electrodes.

```
<ELECTRODE_EXCITATION_BEG>
  FILE = myElectrodes.txt # Specify electrodes by file name or list
  LIST = 55, 66, 77      # Specify electrodes by file name or list
<ELECTRODE_EXCITATION_END>
```

### 3.3.1 FILE Tag

Specify the electrode block list in a separate ASCII comma or space delimited text file.



### 3.3.2 LIST Tag

Specify the electrode block list in an in-line comma or space delimited list.

## 3.4 SYMMETRY Tag Group

The symmetry tag group is a means to associate coordinate plane or coordinate axes symmetry to models. Structurally, the symmetry group tag and all child tags are illustrated below. See the following Concepts sections for understanding and usage.

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = NONE or EVEN or ODD or DOUBLE_ODD #(Default = NONE)
  SYMMETRY_PLANE_XY_OFFSET = 0.0 #(Default = 0.0)
  SYMMETRY_PLANE_XY_WIDTH = 0.0 #(Default = 0.0)
  SYMMETRY_PLANE_XY_N = 5 #(Default = 5)
  SYMMETRY_PLANE_XZ = NONE or EVEN or ODD #(Default = NONE)
  SYMMETRY_PLANE_XZ_OFFSET = 0.0 #(Default = 0.0)
  SYMMETRY_PLANE_YZ = NONE or EVEN or ODD #(Default = NONE)
  SYMMETRY_PLANE_YZ_OFFSET = 0.0 #(Default = 0.0)
  SYMMETRY_ROTATION_MODE = NONE or EVEN or ODD #(Default = NONE)
  SYMMETRY_ROTATION_Z = 90.0 #(Default = -1.0, which indicates not used)
<SYMMETRY_END>
```

### 3.4.1 SYMMETRY\_PLANE\_XY Tag

Specify *xy* plane symmetry for the model as EVEN or ODD or DOUBLE\_ODD (default is NONE). The DOUBLE\_ODD usage is specific to *z*-double ground plane.

### 3.4.2 SYMMETRY\_PLANE\_XY\_OFFSET Tag

Specify *xy* plane symmetry offset, or *z*-double ground plane offset, for the model (default is 0.0). Units are meters.

### 3.4.3 SYMMETRY\_PLANE\_XY\_WIDTH Tag

Specify *z*-double ground plane width for the model (default is 0.0). Units are meters.

### 3.4.4 SYMMETRY\_PLANE\_XY\_N Tag

Specify *z*-double ground plane number of reflections (default is 5).

### 3.4.5 SYMMETRY\_PLANE\_XZ Tag

Specify *xz* plane symmetry for the model as EVEN or ODD (default is NONE).



### 3.4.6 SYMMETRY\_PLANE\_XZ\_OFFSET Tag

Specify  $xz$  plane symmetry offset for the model (default is 0.0). Units are meters.

### 3.4.7 SYMMETRY\_PLANE\_YZ Tag

Specify  $yz$  plane symmetry for the model as EVEN or ODD (default is NONE).

### 3.4.8 SYMMETRY\_PLANE\_YZ\_OFFSET Tag

Specify  $yz$  plane symmetry offset for the model (default is 0.0). Units are meters.

### 3.4.9 SYMMETRY\_ROTATION\_MODE Tag

Specify  $z$  axis rotation symmetry for the model as EVEN or ODD (default is NONE).

### 3.4.10 SYMMETRY\_ROTATION\_Z Tag

Specify  $z$  axis rotation symmetry for the model (default is -1.0). Units are degrees. It must also be an integral factor of 360.0° (for example, {180.0°, 90.0°, 45.0°, ...}). In the case SYMMETRY\_ROTATION\_MODE is ODD (sometimes referred to as *anti*), the number of divisions must be even.

### 3.4.11 Symmetry Plane Concepts

Symmetry planes have two primary purposes: a) Model infinite ground planes with equivalent image models. b) Exploit geometric symmetry in the model to reduce memory and run-time, also using equivalent image models. We will not delve into the depths of image theory. We will instead demonstrate how the concepts are used with the Nullspace ES symmetry tag group.

Suppose we wish to model the ground plane model shown to left in Figure 1. Image theory allows us to replace the physical problem with an equivalent image problem shown to the right in Figure 1 in which the physical ground plane is removed and replaced with an image plane. A few definitions are in order.

- The ground plane boundary condition  $V = 0$  is referred to as a *Dirichlet* boundary condition.
- In the equivalent image plane problem we have labeled the original model as *real*, which indicates it is the actual surface model geometry from the original physical problem.
- In the equivalent image plane problem we have labeled the mirror surface image of the *real* surface as *odd*, which distinguishes the image as related to a ground plane symmetry.

From a workflow perspective we would model the equivalent image problem. This would require meshing only the *real* surface and using the symmetry tags. For this example the image plane is the  $xy$  coordinate plane (hence no offset). Thus, the symmetry tags would be



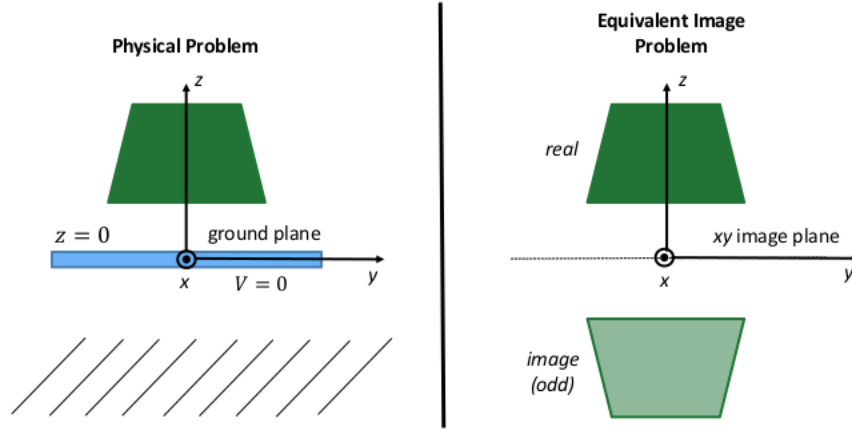


Figure 1: Ground plane physical model and equivalent image model.

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = ODD
  SYMMETRY_PLANE_XY_OFFSET = 0.0
<SYMMETRY_END>
```

We could incorporate an offset to this model as well as shown in Figure 2. As demonstrated in the figure, *offset* refers to the offset of the image plane w/r to the coordinate axes. In this case the image plane is at  $z = h$ , which is an offset of  $h$  from the  $xy$  coordinate plane. The only difference in this model versus the previous is in specifying the offset. For example, if  $h = 2.0$  our symmetry tag set would be.

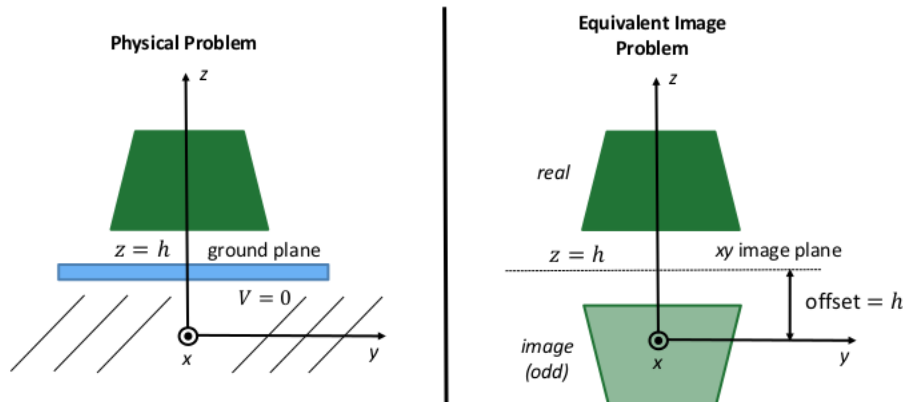


Figure 2: Ground plane physical model and equivalent image model, with an offset.

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = ODD
  SYMMETRY_PLANE_XY_OFFSET = 2.0
<SYMMETRY_END>
```

In some cases we may wish to simply exploit symmetry in the geometry (no ground plane) as shown in Figure 3. Notice in this case the geometry symmetry results in the boundary condition  $\frac{\partial V}{\partial n} = 0$  on the  $z = 0$  plane. As a result, the image is designated with the *even* classification. This type of boundary condition is called a *Neumann* boundary condition. Furthermore we observe this condition implies the



normal component of the electric field vanishes since

$$E_n = \mathbf{E} \cdot \hat{\mathbf{n}} = -\nabla V \cdot \hat{\mathbf{n}} = -\frac{\partial V}{\partial n} = 0 \quad (4)$$

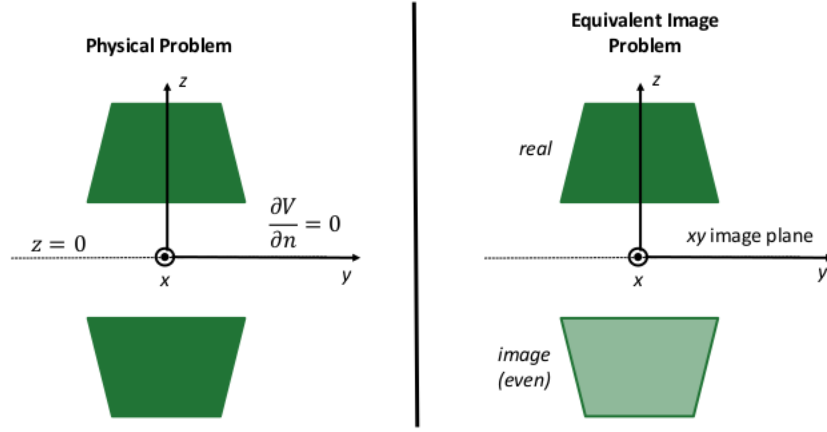


Figure 3: Symmetry physical model and equivalent image model.

As before, from a workflow perspective we would model the equivalent image problem. This would require meshing only the *real* surface and using the symmetry tags. For this example the image plane is the *xy* coordinate plane (hence no offset). Thus, the symmetry tags would be

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = EVEN
  SYMMETRY_PLANE_XY_OFFSET = 0.0
<SYMMETRY_END>
```

Of course, the offset discussion applies here as well though in the figure it is 0.

Nullspace ES supports multiple symmetry planes as well. For example, consider the ground plane model shown to left in Figure 4. In this model we have inherent geometry symmetry about the *xz* coordinate plane (hence a *Neumann* boundary condition) in addition to the ground plane (hence a *Dirichlet* boundary condition). Thus, the equivalent image problem exploits symmetry for both boundary conditions. To be clear, the *real* surface (the surface we actually need to model/mesh) in this example is not identical to the actual physical model. It is only *half* the original model since we can absorb the other half as geometry symmetry due to the Neumann boundary condition. The symmetry tag set for this model would then be

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = ODD
  SYMMETRY_PLANE_XY_OFFSET = 0.0
  SYMMETRY_PLANE_XZ = EVEN
  SYMMETRY_PLANE_XZ_OFFSET = 0.0
<SYMMETRY_END>
```

Notice the *xy* symmetry plane is designated ODD whereas the *xz* symmetry plane is designated EVEN due to the corresponding boundary conditions.



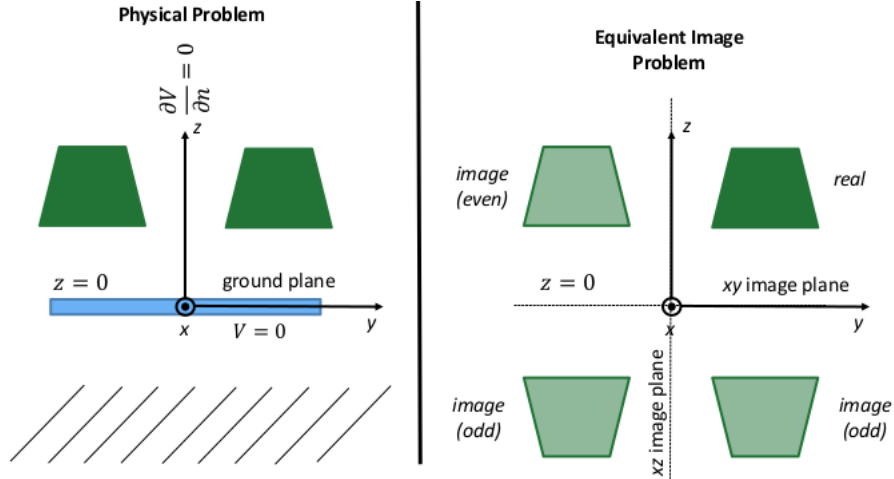


Figure 4: Ground plane physical model and equivalent image model with multiple symmetry planes.

Of course there is a pure geometry symmetry two plane context as well as shown in Figure 5. In this case both boundary conditions are *Neumann* and solely due to geometry symmetry (hence EVEN designations in both cases). The symmetry tag group would then be

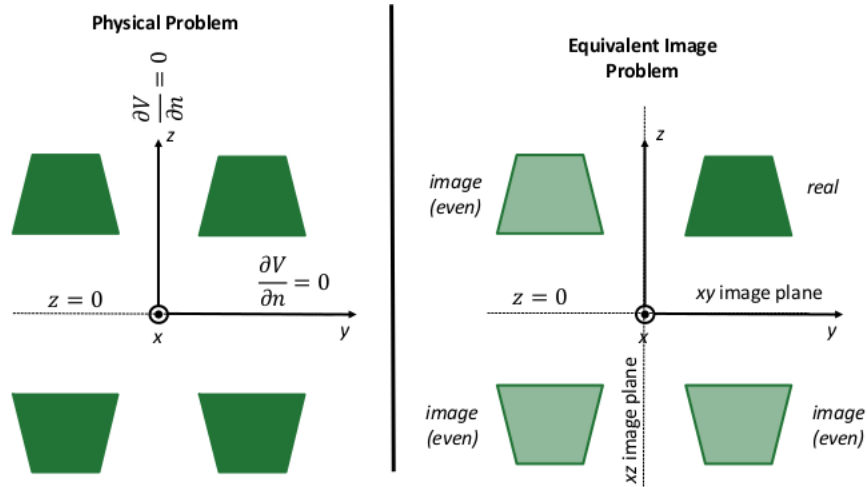


Figure 5: Symmetry physical model and equivalent image model with multiple symmetry planes.

```
<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = EVEN
  SYMMETRY_PLANE_XY_OFFSET = 0.0
  SYMMETRY_PLANE_XZ = EVEN
  SYMMETRY_PLANE_XZ_OFFSET = 0.0
<SYMMETRY_END>
```

We present one last example in Figure 6 in which we have a model consisting of four symmetric pieces in four quadrants in the upper  $z > 0$  half-space. The model is full of symmetry with respect to each of the coordinate planes in addition to having the  $xy$  ground plane. The equivalent image plane model is shown in Figure 7. As a result we would use the following symmetry tag set.

```
<SYMMETRY_BEG>
```



```

SYMMETRY_PLANE_XY = ODD
SYMMETRY_PLANE_XY_OFFSET = 0.0
SYMMETRY_PLANE_XZ = EVEN
SYMMETRY_PLANE_XZ_OFFSET = 0.0
SYMMETRY_PLANE_YZ = EVEN
SYMMETRY_PLANE_YZ_OFFSET = 0.0
<SYMMETRY_END>

```

To be clear, we need only mesh the upper most quadrant model part shown as green in Figure 7. The other blue parts are (symmetry) image pieces and need not be modeled.

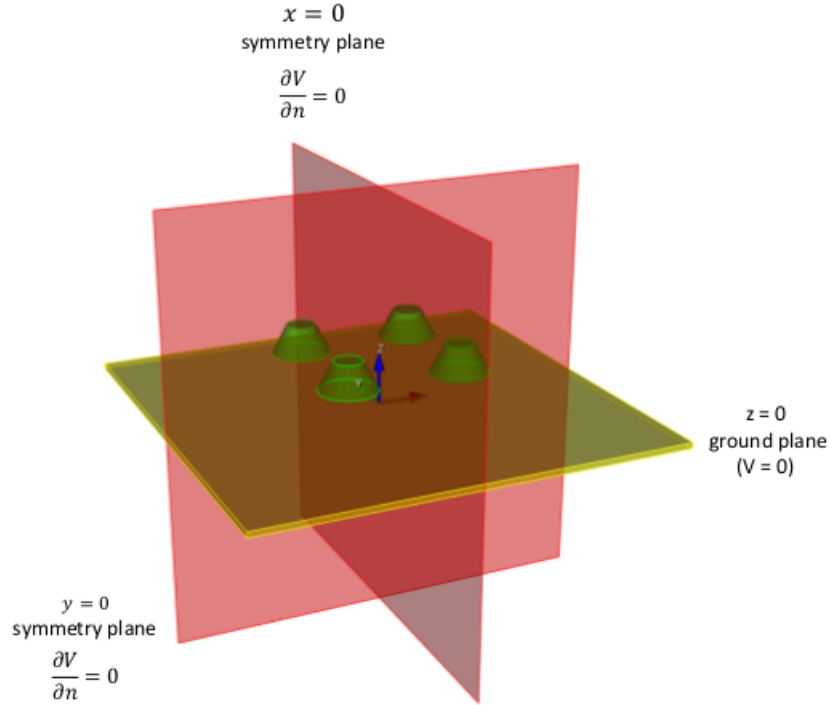


Figure 6: Ground plane physical model with two additional symmetry planes.

There are other combinations we haven't shown. In general, applying symmetry is as simple as mapping the boundary condition specified on the given coordinate plane with the convention (*Dirichlet* → *ODD*), (*Neumann* → *EVEN*), in addition to specifying any coordinate plane offsets.

### 3.4.12 *z*-Rotational Symmetry Concepts

In addition to symmetry planes, Nullspace ES supports *z*-rotational symmetry. For now let's focus on *EVEN* *z*-rotational symmetry. We will illustrate usage with an example. Consider Figure 8. The *physical* boundary value problem is periodic with periodic boundary conditions. Specifically, it is required

$$V(\rho, 0, z) = V(\rho, \pi/2, z). \quad (5)$$

The physical problem (on the right) is desired with periodic boundary conditions. The additional light green *images* are all *z*-rotations of the *real* image where the rotation is 90° in this example. As noted in





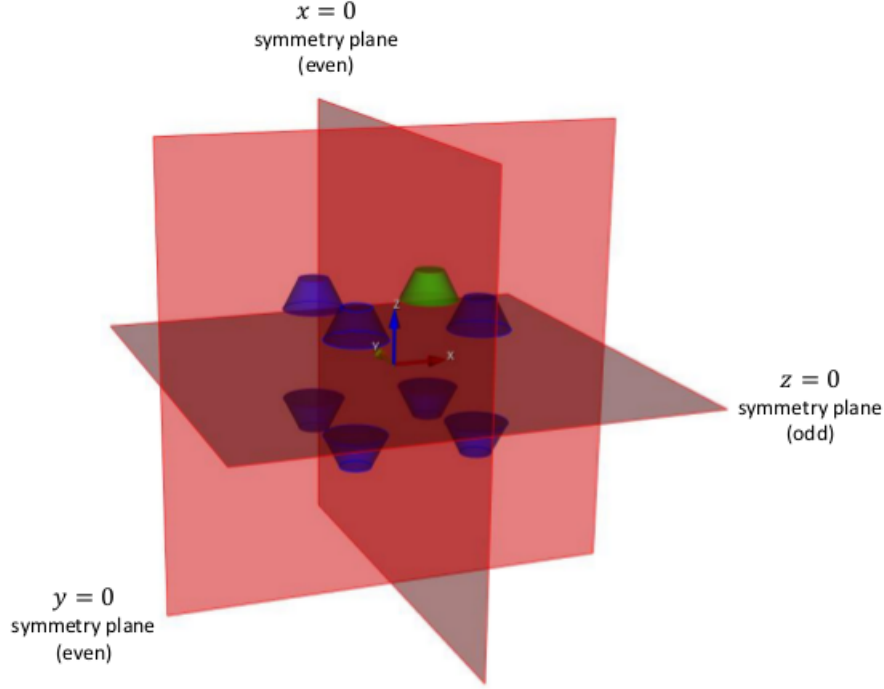


Figure 7: Equivalent image plane model with three symmetry planes.

the SYMMETRY\_ROTATION\_Z tag description, the symmetry period must be an integral factor of  $360.0^\circ$ , which in this  $N_\theta = 4 = 360.0^\circ/90.0^\circ$ .

To be clear, all that is required is to mesh the original physical model and specify the  $z$ -rotational symmetry period desired ( $\theta = \text{SYMMETRY\_ROTATION\_Z}$  in degrees). NSES will internally model the equivalent problem so that the periodic boundary conditions are satisfied across all periods. That is

$$V(\rho, 0, z) = V(\rho, n\theta, z), \quad (6)$$

for  $1 \leq n \leq N_\theta - 1$ , where  $N_\theta = 360.0^\circ/\theta$ .

Nullspace ES also supports composite  $z$ -rotational symmetry with  $xy$ -image plane symmetry (even or odd and with offsets). For example, in Figure 9 the darker red plate in the upper  $x^+y^+z^+$  quadrant is the *physical* (or *real*) image, the blue plates in the upper  $z^+$  hemisphere are due to  $z$ -rotation, the maroon plates in the lower  $z^-$  hemisphere are  $xy$ -images of the  $z$ -rotated blue images, and the golden plane could be a ground plane or image plane as used with the  $xy$ -image plane tag. In this case all that is required is to mesh the darker red plate and specify both the  $xy$ -image plane tags and the  $z$ -rotation tag. NSES will ensure the periodic boundary conditions are satisfied in addition to the image plane boundary conditions.

Similarly, for ODD  $z$ -rotational symmetry we must have  $N_\theta$  even, and the boundary conditions become

$$V(\rho, 0, z) = (-1)^n V(\rho, n\theta, z), \quad (7)$$

Example usage of  $z$ -rotation will be given in the examples section.



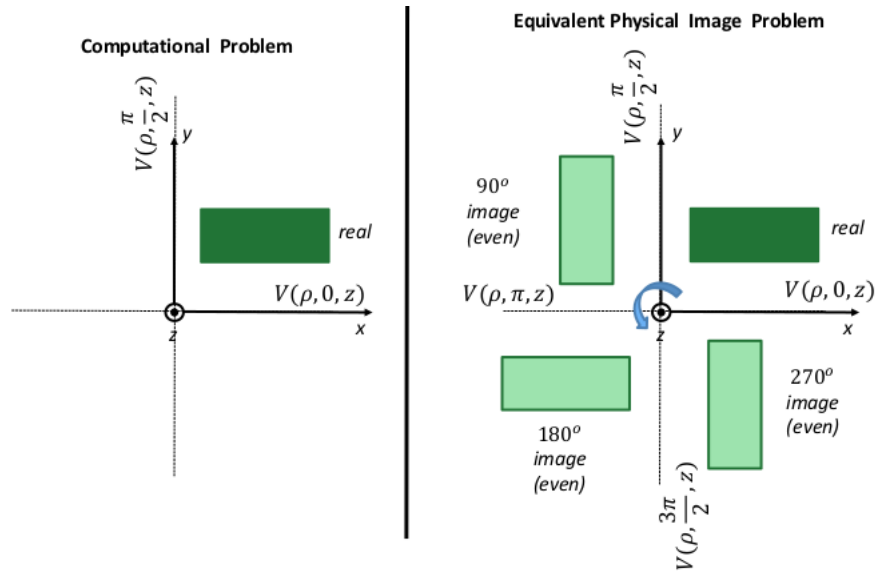


Figure 8:  $z$ -rotational (EVEN) symmetry physical model and equivalent image model.

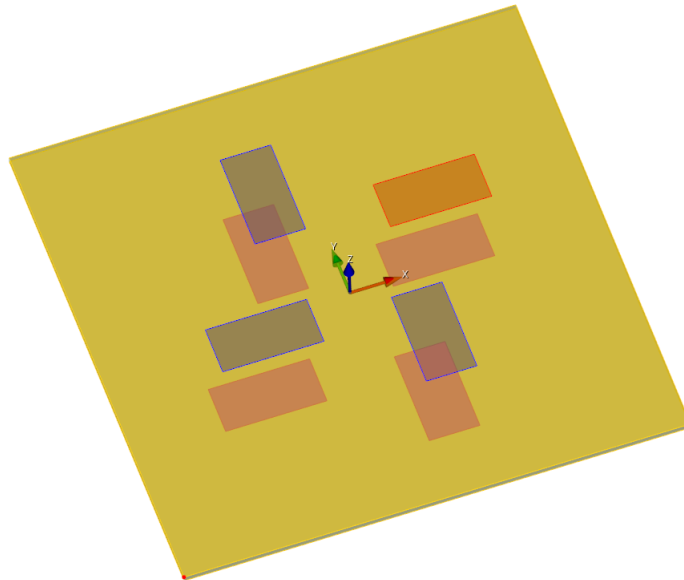


Figure 9: Composite  $z$ -rotational symmetry with  $xy$ -image plane.

### 3.4.13 $z$ -Double Ground Plane Concepts

In addition to symmetry planes, Nullspace ES supports  $z$ -double ground planes. Here the model is assumed to be bound in the  $z$  direction by two infinite ground planes centered at  $z = z_0$  with width  $w$ . Thus, the infinite ground planes are located at  $z = z_0 \pm w/2$ , in which the associated *Dirichlet* (ODD) boundary condition is  $V(x, y, z_0 \pm w/2) = 0$ . This problem has a well known image theory solution.

Of course, this model can be approximated by adding the two physical ground planes to the CAD model. However, using the Nullspace ES  $z$ -double ground plane tag set alleviates the user from this modeling burden. Nullspace ES will account for the (infinite) ground planes internally so that the user need not add the physical ground planes to the model.

The  $z$ -double ground plane configuration is managed with the  $xy$  symmetry tag set. To include an assumed (infinite)  $z$ -double ground plane into a Nullspace ES simulation add the following tags.

```
SYMMETRY_PLANE_XY = DOUBLE_ODD
SYMMETRY_PLANE_XY_OFFSET = z0
SYMMETRY_PLANE_XY_WIDTH = w
SYMMETRY_PLANE_XY_N = 5
```

The DOUBLE\_ODD tag triggers  $z$ -double ground plane usage with the associated  $w$ ,  $z_0$ . The SYMMETRY\_PLANE\_XY\_N controls the number of successive plane images applied. In general, as the double ground planes approach closer to the underlying model, more image terms (N) are required. Note, (SYMMETRY\_PLANE\_XY\_N = -1) is equivalent to no double ground plane.

Nullspace ES also supports composite  $z$ -double ground planes with  $z$ -rotational symmetry (even or odd) or  $xz$  or  $yz$  or  $xz + yz$ -image plane symmetry (even or odd and with offsets).

### 3.4.14 Symmetry Combinations

Nullspace ES allows for combination of several of the above described symmetry options under different conditions. The motivation for using such symmetries is primarily computational. For example, when using the direct solver, a single planar symmetry option will result in a 4X reduction in memory and a 8X reduction in solve time. No significant time savings is realized for the matrix fill since essentially all of the same work is required as the full problem. As multiple symmetry types are invoked, this memory and solve time savings can be substantial. For example, invoking  $xy$ ,  $xz$ , and  $yz$  planar symmetries results in a 64X reduction in memory and a 512X reduction in solve time. Thus, judicious use of symmetry can have a striking impact on the runtime performance of Nullspace ES.

From a purely geometrical perspective, there are four basic symmetries to consider:  $xy$ ,  $xz$ , and  $yz$  planar and  $z$ -rotational. No matter the physics symmetry (e.g., even or odd), the geometric symmetries can be reduced to these four types. Table 1 below shows the allowed combinations of geometric symmetry. The keyword ‘None’ refers to no symmetry applied for that case. The keyword ‘On’ refers to symmetry applied in that case. The double odd symmetry case is captured in the  $xy$ -odd entry.

Note in the table above that combinations of  $xz$  and  $yz$  planar symmetries are incompatible with  $z$ -rotational symmetry and are therefore not allowed.

With these geometric symmetries in mind, one can extend to the set of allowable physics symmetries. Currently, there are 44 possible symmetry configurations allowed in Nullspace ES. In the table below, all currently supported symmetry types and combinations are listed. The keyword ‘None’ retains the same meaning



Table 1: Geometry Symmetries (10 Total Possibilities)

| XY   | XZ   | YZ   | Z Rotation |
|------|------|------|------------|
| None | None | None | None       |
| None | None | On   | None       |
| None | On   | None | None       |
| None | On   | On   | None       |
| On   | None | None | None       |
| On   | None | On   | None       |
| On   | On   | None | None       |
| On   | On   | On   | None       |
| None | None | None | On         |
| On   | None | None | On         |

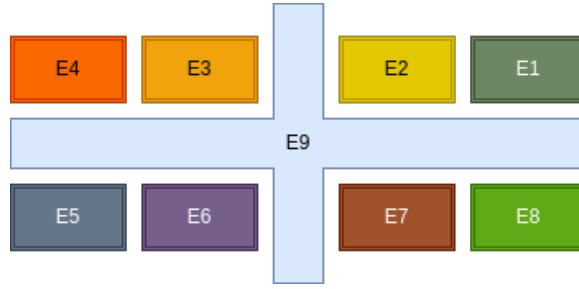


Figure 10: A notional layout of electrodes with a central large electrode and eight surrounding electrodes.

as above. The keywords ‘Even’, ‘Odd’, and ‘Double Odd’ refer to the type of boundary condition imposed with that symmetry option.

### 3.4.15 Using Symmetry with Asymmetric Excitations

In many cases, the combination of geometry, physics, and excitations allow for a single simulation using symmetry. However, there are many cases in which the geometry and boundary conditions promote use of symmetry, yet there is a desire to synthesize the effect of asymmetrically applied excitations.

As example, consider the notional layout of electrodes in Figure 10. The geometry has four-fold planar symmetry with eight unique electrodes (E1-E8) isolated within quadrants of symmetry. There is a central electrode (E9) that has four-fold symmetry but also crosses the apparent lines of symmetry. If this geometry is executed as is, the memory cost is  $\mathcal{O}(N_{\text{tri}}^2)$  when using the direct solver (less so when using the compression solver), and there are nine electrodes to solve. Thus, the solution time is  $\mathcal{O}(N_{\text{tri}}^3 + N_{\text{electrodes}} N_{\text{tri}}^2)$  (again, for the direct solver), where in this case  $N_{\text{electrodes}} = 9$ . For large problems with many electrodes, the second term can begin to dominate the solution time. Even though we may want to apply unique voltages to the nine electrodes in post-processing (which would make the problem asymmetric from the perspective of excitations), we can still exploit symmetry to great effect.

Figure 11 shows the lines of geometric symmetry demarcated with the upper right quadrant noted as the “unit cell” or the “minimum unique geometry.” Running only this problem requires memory on the order of  $\mathcal{O}(N_{\text{tri}}^2/16)$ , where  $N_{\text{tri}}$  is the number of triangles in the original problem of Figure 10. This is a substantial memory savings of 16X. The solve time is similarly reduced to  $\mathcal{O}(N_{\text{tri}}^3/64)$ . Note also that there are at most



Table 2: Physics Symmetries (44 Total Possibilities)

| XY         | XZ   | YZ   | Z Rotation |
|------------|------|------|------------|
| None       | None | None | None       |
| None       | None | Even | None       |
| None       | None | Odd  | None       |
| None       | Even | None | None       |
| None       | Even | Even | None       |
| None       | Even | Odd  | None       |
| None       | Odd  | None | None       |
| None       | Odd  | Even | None       |
| None       | Odd  | Odd  | None       |
| Even       | None | None | None       |
| Even       | None | Even | None       |
| Even       | None | Odd  | None       |
| Even       | Even | None | None       |
| Even       | Even | Even | None       |
| Even       | Even | Odd  | None       |
| Even       | Odd  | None | None       |
| Even       | Odd  | Even | None       |
| Even       | Odd  | Odd  | None       |
| Odd        | None | None | None       |
| Odd        | None | Even | None       |
| Odd        | None | Odd  | None       |
| Odd        | Even | None | None       |
| Odd        | Even | Even | None       |
| Odd        | Even | Odd  | None       |
| Odd        | Odd  | None | None       |
| Odd        | Odd  | Even | None       |
| Odd        | Odd  | Odd  | None       |
| Double Odd | None | None | None       |
| Double Odd | None | Even | None       |
| Double Odd | None | Odd  | None       |
| Double Odd | Even | None | None       |
| Double Odd | Even | Even | None       |
| Double Odd | Even | Odd  | None       |
| Double Odd | Odd  | None | None       |
| Double Odd | Odd  | Even | None       |
| Double Odd | Odd  | Odd  | None       |
| None       | None | None | Even       |
| None       | None | None | Odd        |
| Even       | None | None | Even       |
| Even       | None | None | Odd        |
| Odd        | None | None | Even       |
| Odd        | None | None | Odd        |
| Double Odd | None | None | Even       |
| Double Odd | None | None | Odd        |



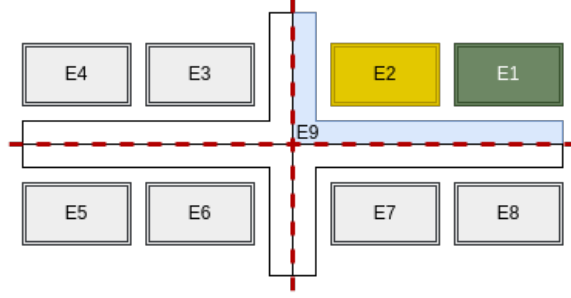


Figure 11: The unit cell capturing minimum unique geometry.

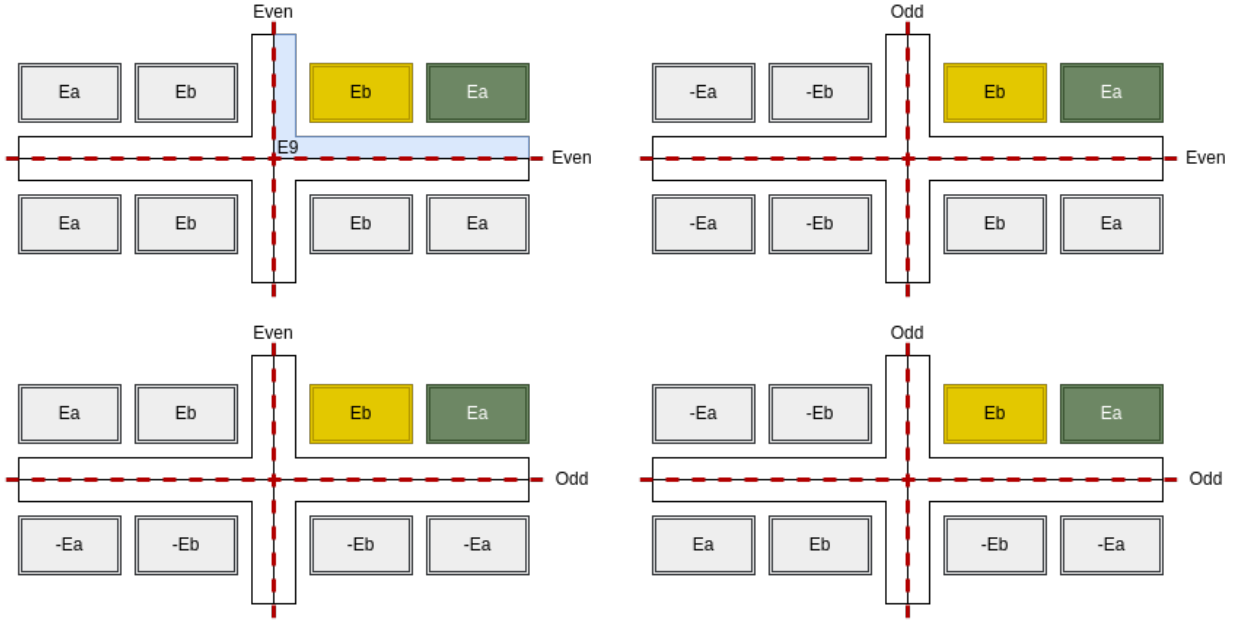


Figure 12: Four variations of symmetry plane options.

3 unique electrodes to solve, which further reduces the solution time.

Now consider Figure 12 that shows four possible symmetry plane variations. The figure shows the effect of the four types of symmetry on the imaged electrodes. Note that only in the Even-Even case is electrode 9 considered, since solving for it in an excited state does not make physical sense in the other three symmetry scenarios. In this figure, the notation on the electrodes is changed to “Ea” and “Eb” to differentiate from the individual electrodes in the full case. Using these four symmetry cases, are there linear superpositions such that we can synthesize the effect of the full geometry in Figure 10? Table 3 shows how solutions of the four symmetric cases can be combined to yield independent solutions for each of the eight electrodes in Figure 10 (electrode 9 can simply be taken alone from the even-even case). In the table, the notation “EO” refers to even symmetry along the vertical and odd symmetry along the horizontal. Since Nullspace ES uses 1V to excite each electrode, the superpositions from Table 3 result in the net effect of a 4V excitation applied to each of the individual electrodes from Figure 10.

Recall that the memory requirement for the full case was proportional to  $N_{tri}^2$ , and that application of four-fold symmetry resulted in a 16X memory reduction. This is still true when running the four independent symmetry cases. Solve time is reduced proportionally from  $N_{tri}^3 + 9N_{tri}^2$  to  $N_{tri}^3/16 + 3N_{tri}^2/4 + N_{tri}^2/16$ ,



Table 3: Superposition Formulas To Obtain Unique Solutions

| Quadrant    | Superposition     | Excitation (1V) | E1 | E2 | E3 | E4 | E5 | E6 | E7 | E8 |
|-------------|-------------------|-----------------|----|----|----|----|----|----|----|----|
| Upper Right | EE + OE + EO + OO | Ea              | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             |                   | Eb              | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  |
| Upper Left  | EE - OE + EO - OO | Ea              | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0  |
|             |                   | Eb              | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  |
| Lower Left  | EE - OE - EO + OO | Ea              | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  |
|             |                   | Eb              | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  |
| Lower Right | EE + OE - EO - OO | Ea              | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  |
|             |                   | Eb              | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  |

which is approximately a 16X reduction in runtime. Thus, while four simulations are required, the runtime and memory are both reduced by approximately 16X. Of course, similar operations can be performed to synthesize full geometries when using other types and combinations of symmetry.

### 3.5 SOLVER Tag Group

The SOLVER tag group is used to expose control parameters for the internal computational algorithms. The entire tag group is shown below. We will discuss each child tag in detail.

```
<SOLVER_BEG>
# (Default = SINGLE if SYSTEM = DENSE)
# (Default = HYBRID if SYSTEM = COMPRESS)
PRECISION = SINGLE or HYBRID or DOUBLE

# (Default = DENSE)
SYSTEM = DENSE or COMPRESS

# (Default = 1E-04)
FILL_COMPRESS_TOLERANCE = 1E-04
# (Default = 1E-04)
SOLVE_COMPRESS_TOLERANCE = 1E-04

# (Default = NONE)
FILL_RECOMPRESS = NONE or RSVD
# (Default = FILL_COMPRESS_TOLERANCE)
FILL_RECOMPRESS_TOLERANCE = 1E-04

# (Default = OFF)
COMPRESS_ALL_OFF_DIAGONAL_BLOCKS = ON or OFF

# (Default = ON)
ACA_FILL_RESTART = ON or OFF
# (Default = -1 - must be positive to use)
ACA_FILL_RESTART_SIZE = -1
# (Default is 0.0025 and it is the default method)
ACA_FILL_RESTART_PERCENT = 0.0025
<SOLVER_END>
```



### 3.5.1 PRECISION Tag

Overall Nullspace ES numerical precision can be controlled with the PRECISION tag. If (SYSTEM = DENSE) the default precision is SINGLE, which corresponds to all computations being performed in 32-bit floating point arithmetic. If (SYSTEM = COMPRESS) the default precision is HYBRID. A hybrid precision indicates the underlying computations are performed in double precision (64-bit) but the storage is in single precision. In most cases the default precision is sufficient.

### 3.5.2 SYSTEM Tag

The electrostatic linear system is dense by nature of the mathematical formulation. However, it can be exploited with compression techniques. The SYSTEM tag designates how Nullspace ES treats the linear system. If (SYSTEM = DENSE) Nullspace ES will brute force fill the linear system and brute force solve it with a point-wise LU decomposition. If (SYSTEM = COMPRESS) Nullspace ES will fill the linear system using the Adaptive Cross Approximation (ACA) method, which results in a substantial reduction in memory and runtime for electrically large geometries. Nullspace ES will then solve the system with ACA and the Reduced Singular Value Decomposition (rSVD) method.

Compression techniques are often referred to as “fast solvers” and are intended to solve electrically large problems that may otherwise exhaust system resources (in particular, RAM). Roughly speaking, fill compression is achieved by first splitting the geometry into clusters. Then the original system matrix is split into blocks in which each block represents cluster-to-cluster interactions. See the CLUSTER tag group for cluster control. The remaining tags below are dedicated to controlling compression.

### 3.5.3 FILL\_COMPRESS\_TOLERANCE Tag

Matrix fill compression is error-controllable, and the FILL\_COMPRESS\_TOLERANCE tag is used to control the tolerance. A higher fill tolerance will allow for higher block fill compression, which in general yields faster run time and lower RAM overhead. Likewise, a lower fill tolerance will allow for lower block fill compression, which in general yields slower run time and higher RAM overhead but an increase in accuracy. As is the case with any approximation technique, balancing performance with accuracy is the goal. For most problems, FILL\_COMPRESS\_TOLERANCE = 1.0e-4 is a sufficient balance to fill run-time and accuracy, but it is recommended to explore a lower tolerance (typically not lower than 1.0e-5 for single precision) where possible to verify fill accuracy is sufficient.

### 3.5.4 FILL\_RECOMPRESS Tag

On electrically large problems fill re-compression can significantly reduce memory and run-time. Nullspace ES does not currently activate fill re-compression by default (FILL\_RECOMPRESS = NONE). Note the FILL\_RECOMPRESS tag is only applicable with the SYSTEM = COMPRESS. The current re-compression technique is (rSVD - see Appendices for more information). To activate fill re-compression with rSVD use FILL\_RECOMPRESS = RSVD.

Fill rSVD re-compression is often used with solver rSVD re-compression (to be discussed later). Both fill and solve re-compressions typically yield a very small run-time penalty. Moreover, these small additional costs are often negated since the smaller memory block sizes lead to significant savings in total run-time and memory.





### 3.5.5 FILL\_RECOMPRESS\_TOLERANCE Tags

If the FILL\_RECOMPRESS tag is activated it uses an error-controllable tolerance to perform the fill re-compression. By default, Nullspace ES will use the FILL\_COMPRESS\_TOLERANCE for the fill re-compression tolerance. The fill re-compression tolerance can be overridden with the optional FILL\_RECOMPRESSION\_TOL

### 3.5.6 SOLVE\_COMPRESS\_TOLERANCE Tag

The SOLVE\_COMPRESS\_TOLERANCE tag controls accuracy of the compression based block LU decomposition solvers. The accuracy vs run-time performance arguments above for fill tolerance apply to the LU tolerance as well with the difference being in relation to solving vs filling the matrix. For most problems, SOLVE\_COMPRESS\_TOLERANCE = 1.0e-4 is a sufficient balance to solve run-time and accuracy, but it is recommended to explore a lower tolerance (typically not below 1.0e-5 for single precision) where possible to verify solution accuracy is sufficient.

### 3.5.7 ACA\_FILL\_RESTART Tags

ACA is known to prematurely converge on certain problems. To overcome this limitation Nullspace ES implements an ACA re-start logic during fill compression. The ACA fill re-start is activated with ACA\_FILL\_RESTART = ON/OFF, and it is ON by default (highly recommended) when ACA based fill and solve compression tags are invoked.

The default re-start criteria is  $s \cdot \min(m, n)$ , where  $m, n$  are the block outer dimensions, and  $s$  is a percentage scale (defaulted to 0.0025). During fill compression of a block, Nullspace ES will monitor the ACA stopping criteria to determine convergence. Once convergence is reached Nullspace ES will re-start ACA up to  $s \cdot \min(m, n)$  number of re-starts. ACA will then stop after convergence has been reached for the last re-start. For example, if a block is of size  $m = 2500, n = 2000$ , then the default criteria would be  $0.0025 \cdot 2000 = 5$ , implying ACA would re-start 5 times on the block and must converge on the last re-start.

The percentage scale  $s$  is configurable with ACA\_FILL\_RESTART\_PERCENT tag (default 0.0025). Another way to override the re-start criteria is to set the ACA\_FILL\_RESTART\_SIZE tag in which every compressed block will use the same number of re-start attempts regardless of block size.

### 3.5.8 SCALE\_NEAR\_BLOCKS

Nullspace ES declares linear system sub-blocks (cluster-to-cluster interactions) as *compressible* based on the following criteria

$$\min(\text{diam } C_t, \text{diam } C_s) \leq \eta \text{dist}(C_t, C_s) \quad (8)$$

where  $C_t$  and  $C_s$  are clusters that form the sub-block  $Z_{ts}$ , and ( $\eta = \text{SCALE\_NEAR\_BLOCKS}$ ) is a configurable scale with default value 2.0. In this way  $\eta$  controls the number of full vs compress filled blocks in the system. Typically the default value is sufficient.

There is one caveat to the criteria. All  $Z_{ts}$  must satisfy CLUSTER\_MINIMAL\_COMPRESSION\_SIZE as noted in the CLUSTERS tag group.



### 3.5.9 COMPRESS\_ALL\_OFF\_DIAGONAL\_BLOCKS

In some cases it is known in advance compressing all blocks will yield sufficient accuracy. In such cases it is possible to override admissibility checks and declare all blocks to be compressed with COMPRESS\_ALL\_OFF\_DIAGONAL = ON (default is OFF).

## 3.6 CLUSTERS Tag Group

Fill compression is based on a mesh clustering and grouping (block) scheme. The entire tag group is shown below.

```
<CLUSTERS_BEG>
  CLUSTER_MAX_LEVEL = 12                # (Default = 12)
  CLUSTER_MINIMAL_SIZE = 2000           # (Default = 2000)
  CLUSTER_MINIMAL_COMPRESSION_SIZE = 10 # (Default = 10)
  CLUSTER_DISTANCE_METHOD = BOUNDING_CORNERS # (Default = BOUNDING_CORNERS)
                                         or CENTROID
<CLUSTERS_END>
```

The entire CLUSTER tag group is optional. Moreover, in most cases all the CLUSTER tag default values are sufficient, and thus this tag group is usually not specified.

### 3.6.1 CLUSTER\_MAX\_LEVEL Tag

The cluster (binary) tree has a maximum depth level controlled by the CLUSTER\_MAX\_LEVEL tag (defaulted to 12 with a maximum value of 15).

### 3.6.2 CLUSTER\_MINIMAL\_SIZE Tag

As the cluster tree *splits* clusters into smaller clusters it must have a threshold stopping criteria. The CLUSTER\_MINIMAL\_SIZE tag pertains to a *targeted* minimum number of basis indices contained in a cluster (defaulted to 2000.) Though most clusters will tend to hover around the default threshold size, certain geometries can produce relatively small clusters below the threshold.

### 3.6.3 CLUSTER\_MINIMAL\_COMPRESSION\_SIZE Tag

As discussed in the SOLVER tag group section, sub-blocks  $Z_{ts}$  in a compressed linear system are formed by two clusters  $C_t$  and  $C_s$ . The CLUSTER\_MINIMAL\_COMPRESSION\_SIZE tag pertains to the minimum number of basis indices contained in both  $C_t$  and  $C_s$  to be a candidate for a compressed block. It is defaulted to 10.

### 3.6.4 CLUSTER\_DISTANCE\_METHOD Tag

The CLUSTER\_DISTANCE\_METHOD tag is used to specify the manner in which distance between clusters is calculated. The BOUNDING\_CORNERS method implies the minimal distance between corners defining



clusters. The CENTROID method implies distance between cluster centroids. The default is BOUNDING\_CORNERS and has shown to be a good overall method.

### 3.6.5 CLUSTER\_COLLECTIONS Tag

The default behavior of clustering is to assign a single cluster tree to cluster all potential surfaces in the entire mesh. In some circumstances it may be advantageous to apply a cluster tree to each potential surface. This can be achieved by setting CLUSTER\_COLLECTIONS = ON (default OFF). In most cases the default is sufficient.

## 3.7 OUTPUT Tag Group

Nullspace ES supports a variety of output data which it stores in an HDF5 file. The HDF5 name is derived from the input deck name. For example

```
myInputDeck.in
myInputDeck.in.h5
```

The electrostatics linear system is solved for the surface charge densities associated with each electrode excitation noted in the Electrode Excitation tag group. By default the surface charge density solution for each electrode is written to the HDF5 file. There are no tags required for this action.

The Output tag group is dedicated to directing additional output to the HDF5 file and generating optional VTU and XMF files. The entire tag group is shown below.

```
<OUTPUT_BEG>
  CHARGE_VTU = ON or OFF      # (Default = OFF)
  GRID = myGridFile.h5        # secondary HDF5 grid file
  POTENTIAL = ON or OFF      # (Default = ON)
  ELECTRIC_FIELD = ON or OFF  # (Default = ON)
  XMF = ON or OFF             # (Default = OFF)
<OUTPUT_END>
```

### 3.7.1 CHARGE\_VTU Tag

As noted before the surface charge density solution for each electrode is written to the HDF5 file by default. To support Parview visual workflows, the surface charge density may be written to an ASCII VTU file as well with CHARGE\_VTU = ON (default = OFF). With the VTU, users may visualize the surface charge density solutions for each electrode.

### 3.7.2 GRID Tag

The potential and/or E-fields are computed at one or more *grids* of points. The GRID tag specifies a secondary HDF5 file containing the grid specifications. The Nullspace ES pre- and post-processing API is used to build grid files (discussed later).



### 3.7.3 POTENTIAL Tag

The potential may be obtained at a specified collection of grids with `POTENTIAL = ON`, which is the default. This will result in the potential written to the HDF5 file. Note a grid file is required (see `GRID` tag.)

### 3.7.4 ELETRIC\_FIELD Tag

The E-fields may be obtained at a specified collection of grids with `ELETRIC_FIELD = ON`, which is the default. This will result in the E-fields written to the HDF5 file. Note a grid file is required (see `GRID` tag.)

### 3.7.5 XMF Tag

As noted before, the potential and E-fields are written to the simulation HDF5 file. The grid points are in the secondary HDF5 file. To support Paraview visual workflows, the potential and/or E-fields and grid points can be referenced into an XMF (XDMF) file with `XMF = ON` (default is OFF). By loading the XMF file in Paraview, users may visualize the potentials and E-fields using Paraview's built-in tools.

## 3.8 Common Input Deck Usages

There are three primary input deck usages: (1) Dense system workflows; (2) Compressed system workflows; (3) Multi-run workflows. We will discuss each in the following sections.

### 3.8.1 Dense Linear System

**Example 1:** The following is a minimal tag set formulation to solve a dense linear system accepting all tag defaults not specified and specifying the electrodes via the `LIST` tag. It will solve for the surface charge densities and store into the HDF5 file. The Mesh Quality, Cluster, and Output configuration tag groups have applied defaults and are not specified.

```
<MESH_BEG>
  FILE = myMesh.inp
<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 33, 44, 55, 66
<ELECTRODE_EXCITATION_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>
```

**Example 2:** The following is a minimal tag set formulation to solve a dense linear system accepting all tag defaults not specified, and specifying the electrodes via the `LIST` tag. It will solve for the surface charge densities and store into the HDF5 file. It will also write the surface charge densities to an ASCII



based VTU file. The Mesh Quality and Cluster configuration tag groups have applied defaults and are not specified.

```
<MESH_BEG>
  FILE = myMesh.inp
<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 33, 44, 55, 66
<ELECTRODE_EXCITATION_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  CHARGE_VTU = ON
<OUTPUT_END>
```

**Example 3:** The following is a minimal tag set formulation to solve a dense linear system accepting all tag defaults not specified, and specifying the electrodes via the LIST tag. It will solve for the surface charge densities and store into the HDF5 file. It will also compute the potential and E-fields and write to the HDF5 file. It will also write the secondary XMF file. The Mesh Quality and, Cluster configuration tag groups have applied defaults and are not specified.

```
<MESH_BEG>
  FILE = myMesh.inp
<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 33, 44, 55, 66
<ELECTRODE_EXCITATION_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  GRID = myGridFile.h5
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  XMF = ON
<OUTPUT_END>
```

Simply change (SYSTEM = DENSE) to (SYSTEM = COMPRESS) in the above examples for equivalent examples with compression.



### 3.8.2 Multi-Run Workflows

Often a user may wish to take advantage of previously performed simulations and calculate the potential or electric field at additional points in space. Similarly, a user may wish only to use high performance computing hardware for the main solution stage of the problem and use lesser hardware to compute potentials and fields. Nullspace ES enables these options through a multi-run workflow. After an initial run, the pertinent simulation information is stored on the output HDF5 file. This file may be used to initialize a subsequent run assuming the mesh file and, if applicable, electrodes excitation file are also present. For the subsequent run, only the OUTPUT tag group is allowed. Presence of any other tag group will trigger Nullspace ES to validate the input deck as if it was a full, initial run. Several examples follow that provide a helpful illustration of the multi-run workflow.

**Example 1:** The user initially creates a minimal input file without any output data requested.

| myInitialRun.in                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <MESH_BEG><br>FILE = myMesh.inp<br><MESH_END><br><br><ELECTRODE_EXCITATION_BEG><br>LIST = 33, 44, 55, 66<br><ELECTRODE_EXCITATION_END><br><br><SOLVER_BEG><br>SYSTEM = DENSE<br><SOLVER_END> |

This will produce the output file myInitialRun.in.h5 which, along with the mesh file, contains all of the information required for further processing. In this case, the user desires to obtain the charge density VTU file for visualizing the results. This is accomplished by creating an additional input deck like the following:

| myVTU.in                                        |
|-------------------------------------------------|
| <OUTPUT_BEG><br>CHARGE_VTU = ON<br><OUTPUT_END> |

To obtain the VTU file, execute Nullspace ES from the command line as follows:

```
nres myVTU.in myInitialRun.in.h5
```

which will generate myVTU.in.vtu.

**Example 2:** The user initially creates an input file with a full set of output tags as follows:



#### myInitialRun.in

```
<MESH_BEG>
  FILE = myMesh.inp
<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 33, 44, 55, 66
<ELECTRODE_EXCITATION_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  GRID = myGridFile.h5
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  XMF = ON
  CHARGE_VTU = ON
<OUTPUT_END>
```

which produces myInitialRun.in.h5, myInitialRun.in.vtu, and myInitialRun.in.xmf. After performing some initial analysis with these data files, the user desires to obtain electric fields at additional grid points. To do so, the following input deck is created:

#### my2ndRun.in

```
<OUTPUT_BEG>
  GRID = my2ndGridFile.h5
  POTENTIAL = OFF
  ELECTRIC_FIELD = ON
<OUTPUT_END>
```

To obtain the additional data, execute Nullspace ES from the command line as follows:

```
nres my2ndRun.in myInitialRun.in.h5
```

which will generate my2ndRun.in.h5. This file contains the newly requested electric fields.

## 4 Grid File Preparation

Nullspace ES provides the user the ability to compute potentials and electric fields at points in space through the specified of a grid file. The term “grid” is simply borrowed terminology from the XDMF (i.e., XMF file) and Paraview file formats since Paraview is one of the primary intended visualization tools for NSES. A grid is a specification for a set of points in 3D space. Of the XDMF grid types, two are currently supported: 3DRECTMESH and POLYVERTEX. Each grid in the grid file is assigned a unique numerical ID, and the file can contain an arbitrary number of grids. The grid file points are specified in meters regardless of the mesh file’s units.



## 4.1 3DRECTMESH

A 3DRECTMESH is a rectilinear grid of points specified by monotonically increasing lists of x, y, and z points. Along a particular axis, the points need not be evenly spaced but must not be repeating. Nullspace ES provides a Python function to write the grid.

As an example, consider the grid of points shown in Figure 13. Using the provided writeGrid function, this grid is obtained using the following Python code:

```
import numpy as np
import nses as nses

x = np.array([0, 1, 2, 4, 8])
y = np.linspace(0, 10, 6)
z = np.array([0, 5])

nses.writeGrid('exampleGrid.h5', 1, '3drectmesh', x, y, z)
```

In this example, the three arrays defining the 3DRECTMESH grid are written to the HDF5 file exampleGrid.h5 with a grid ID of 1.

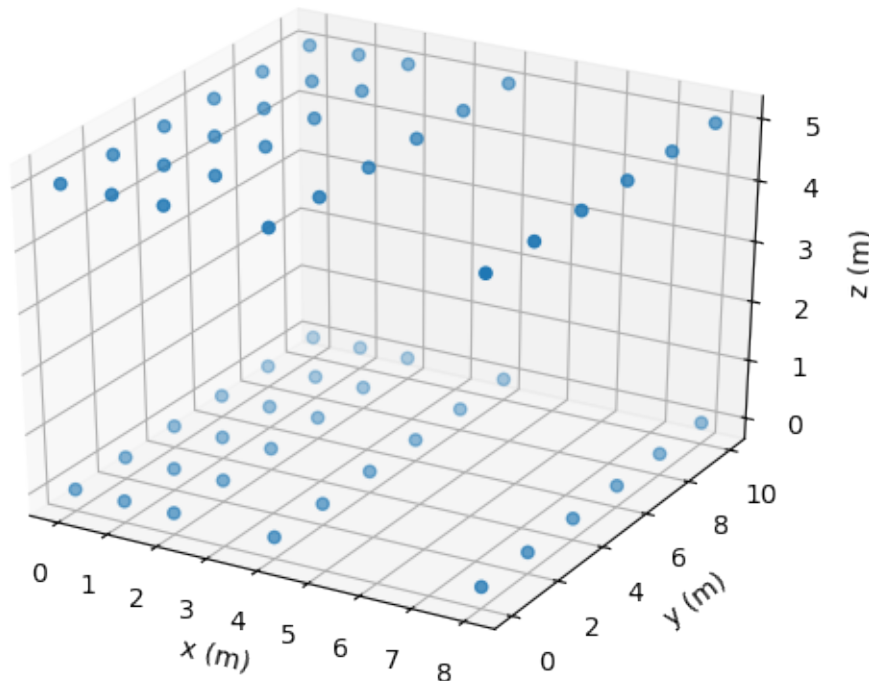


Figure 13: Example 3DRECTMESH grid

## 4.2 POLYVERTEX

A POLYVERTEX grid is simply an arbitrary list of 3D points in space. The same Python function is also used to write these points to a grid file. As an example, consider the spiral of points shown in Figure 14.





Continuing with the previous example, this grid can be added to the same file so long as a unique grid ID is given.

```
numPoints = 101
t = np.linspace(0, 4*math.pi, numPoints)
x = t*np.cos(t)
y = t*np.sin(t)
z = t/math.pi

nses.writeGrid('exampleGrid.h5', 2, 'polyvertex', x, y, z)
```

Assuming the above code follows that of the 3DRECTMESH example, the three arrays defining the POLYVERTEX grid are written to the HDF5 file exampleGrid.h5 with a grid ID of 2.

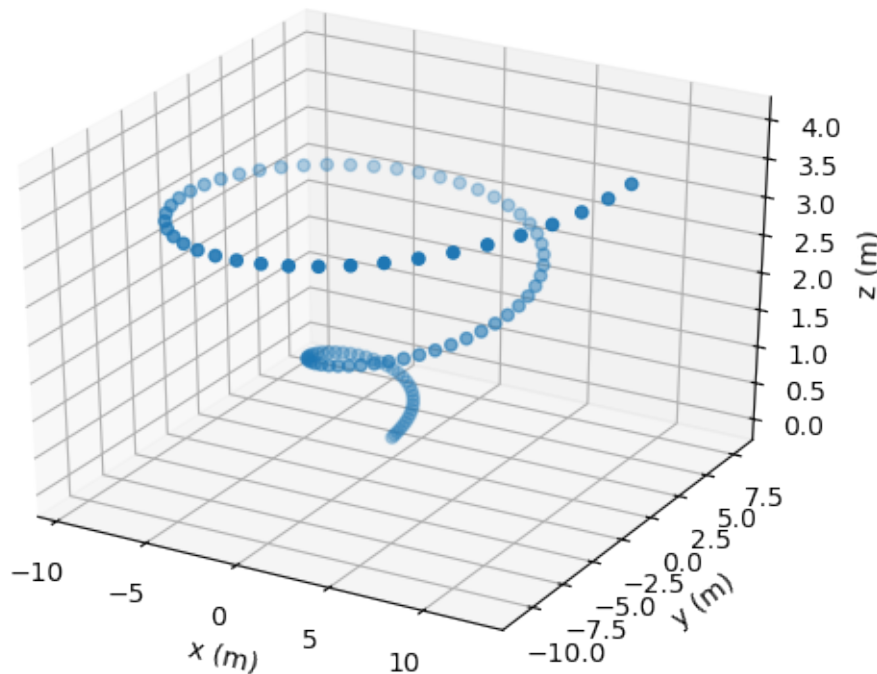


Figure 14: Example POLYVERTEX grid

Retrieving grid information and post-processing data is covered in a later section.

## 5 Nullspace ES Execution

Nullspace ES can be run on any machine meeting the requirements listed at the beginning of this document. If the typical installation procedure is followed, the NSES executable will be in the user's path. Nullspace ES can be invoked from a command line terminal with the current directory containing the input deck. To invoke Nullspace ES, simply execute the following command assuming the input deck is named sphere.in:

```
>> nses sphere.in
```



## 5.1 Standard Out

Nullspace ES reports a variety of statistics and processing information from the various internal modules. The overall flow of the program is conveyed in the standard out stream and appears on the console screen. The information is presented in a user friendly and self explanatory manner.

## 5.2 Diagnostic Output

If the input deck is configured to solve a dense system (SYSTEM = DENSE) then the standard out stream is the only processing information issued by Nullspace ES. If the input deck is configured to solve a compressed system (SYSTEM = COMPRESS) then a variety of additional diagnostic files are written as discussed below.

### ACAProcessing.txt

As noted before, during run-time it is often desirable to track performance of compression. This is done primarily with the ACAProcessing.txt file. As the product is running the typical workflow is to perform a running query such as

```
>> tailf ACAProcessing.txt
```

At the top of ACAProcessing.txt is information related to the block grid. An example is shown below (taken from the disk compression example run).

```
Build blocks beg
  Making new collection 0
  Cluster size BT (92) from collection (0)
  Cluster size BT trimmed (92) from collection (0)
  Appending (92) clusters from collection (0)
  Cluster size BT (92) from collection (0)
  Cluster size BT trimmed (92) from collection (0)
  Clusters size = 92

  Z block grid
    number of unknowns : 105253
    number of blocks : 8464 : (92,92)
    number of full fill blocks : 664 : 92 (diagonal) : 572 (off diagonal)
    number of compressed blocks : 7800
Build blocks end

Permutations beg
Permutations end
```

After displaying the block grid Nullspace ES begins filling the compressed block system. The ACAProcessing.txt file will begin rapidly displaying information about each block as it is compressed. An example is shown below (taken from the disk compression example run).

```
----- Compression Begin -----

1 : 6 : (0, 6) : (0,5,4) (0,6,23) : 0 599 4055 4868 12 : 96.5 4.4e-02
1 : 7 : (0, 7) : (0,5,4) (0,6,29) : 0 599 4869 5687 10 : 97.1 4.9e-02
1 : 8 : (0, 8) : (0,5,4) (0,7, 1) : 0 599 5688 7356 9 : 98.0 5.5e-02
1 : 12 : (0,12) : (0,5,4) (0,7,22) : 0 599 9219 10090 10 : 97.2 1.1e-02
1 : 15 : (0,15) : (0,5,4) (0,7,31) : 0 599 12365 12755 9 : 96.2 6.8e-03
```



There are 17 fields for each block entry. The explanations for each field are given below.

Field (1) : If the block is dense 0. If the block is compressed 1.

Field (2) : The block linear index  $iBlock$  where  $0 \leq iBlock < NBlocks$ .

Field (3,4) : The block grid index  $(iRowBlock, jColBlock)$  where  $0 \leq iRowBlock, jColBlock < mBlocks$ , and  $mBlocks = \sqrt{NBlocks}$ .

Field (5,6,7) : The block test cluster  $(iCollection, iLevel, iRun)$  where the test cluster is in collection  $(iCollection)$  at depth level  $(iLevel)$  with run into the depth level  $(iRun)$ .

Field (8,9,10) : The block source cluster  $(iCollection, iLevel, iRun)$  where the source cluster is in collection  $(iCollection)$  at depth level  $(iLevel)$  with run into the depth level  $(iRun)$ .

Field (11,12,13,14,15) : The block being filled has row extents  $(mBeg, mEnd)$ , which correspond to fields 11,12. The block being filled has column extents  $(nBeg, nEnd)$ , which correspond to fields 13,14. The block estimated rank is field 15. Note if the block is a full filled the rank field will be 0.

Field (16) : The block compression percentage.

Field (17) : The estimated block wall-clock fill time (sec).

Example : Based on the above definitions, let's examine the second block entry listed. We would read this as a compressed block, linear index 7, block grid index (0,7), test cluster in collection 0 at level depth 5 with run 4 into the level depth, source cluster 0 at level depth 6 with run 29 into the level depth, embedded into global matrix at block slice (0,599)x(4869,5687), rank is 10, compression is 97.1 %, with a rough wall-clock estimate compute time of 0.049 (sec).

After block filling is complete Nullspace ES will enter block decompose with a block LU algorithm based on rSVD. The progress of block LU continues into the ACAProcessing.txt file. At each step in the block LU process the *keystone* diagonal block is computed and related  $L$  and  $U$  block. A sample from this example is shown below.

The *block set compression time* is a rough wall-clock time as to how many seconds it took to do the prior set of blocks.

The *memory* row is an estimate of system memory. The first field is an algorithmic rough estimate on total block memory used thus far in the process. The second memory field is virtual memory. The third memory field is page memory.

$D_{11}$  implies the  $D_{11}$  diagonal block is computed at roughly 0.039 (sec) wall-clock time. The entries under the diagonal block are  $L$  and  $U$  blocks tied to the given *keystone* diagonal block with first field corresponding to block index, second field LU block compression percentage, third field fill block compression percentage (for comparison), and forth field estimated block LU compression wall-clock time in seconds.

Initially, there are many blocks in each block set and they are reduced quickly. As the product sweeps across the block grid the blocks in each block set reduce, but the blocks get more expensive to compute. Each block set reduces the number of blocks in the next set by 2. The final block set has 2 blocks in it, but it typically requires many prior computed blocks to reduce it.

```
Block set compression time : 6.2e-02
Memory (GB) : 3.38e+00 : 4.59e+00 : 3.69e+00
D_1_1 : 3.9e-02
    L_6_1 :    98.3 :    97.4 :    2.0e-03
    L_13_1 :   98.7 :    97.8 :    1.9e-03
```



```
L_8_1 : 99.3 : 98.4 : 1.9e-03
L_7_1 : 98.3 : 96.8 : 2.0e-03
```

After block LU completes Nullspace ES will *back solve* the system for each electrode excitation. Back solve progress continues in the ACAProcessing.txt file. The first few entries for this example are shown below. In essence the product back solves for all electrode excitations at once as indicated below.

```
Back Solve (Begin)

Applying permutation to all RHS ... end (1.1e-03 sec)

Applying L column block set 1 of 91 to all RHS
Applying L column block set 2 of 91 to all RHS
Applying L column block set 3 of 91 to all RHS
Applying L column block set 4 of 91 to all RHS
```

There are a few additional diagnostic files related to compression. However, ACAProcessing.txt is usually all that is needed.

### **ACABlocks.txt**

Contains only the compressed fill diagnostic data.

### **ACAlu.txt**

Contains only the block LU diagnostic data.

### **ACACollections.txt**

Contains information about the clustering binary trees.

### **ACAPermutation.txt**

Contains the permutation vector for the internal linear system.

## **6 Post-processing Analysis**

Nullspace ES provides a variety of post-processing output files. The output files fall into one of two categories: run-time performance files and engineering data files.

### **6.1 Run-time Performance Information**

Nullspace ES presents much information to the user at run-time. A user can readily monitor the progress of a run and verify the validity of many inputs by analyzing the information presented on screen. When utilizing the compression, several log files are also available for monitoring and analysis. The primary log file of interest is ACAProcessing.txt. It contains a variety of information related to matrix fill and solution progress as well as the compression level and compression time for individual blocks in the matrix. A typical monitoring would include the following.

```
>> tail -f ACAProcessing.txt
```



## 6.2 Engineering Data Files and Post Processing Scripts

Nullspace ES engineering data encompasses surface charge density, potential, and E-fields. All engineering data are stored in a single HDF5 file. The HDF5 file name has the format below:

```
<inputDeckFileName>.in.h5
```

The HDF5 engineering data is accessed via the Nullspace ES post-processing module, which is part of the NSES install and can be found in the `processing/` directory. Table 4 gives a list of processing functions provided by Nullspace ES for use in Python and contained in the `nses.py` module.

Table 4: Pre- and post-processing functions descriptions

| Name                           | Description                                                                 |
|--------------------------------|-----------------------------------------------------------------------------|
| <code>writeGrid</code>         | Write a rectilinear grid or list of points to an HDF5 grid file.            |
| <code>readGrid</code>          | Read a rectilinear grid or list of points from an HDF5 grid or output file. |
| <code>getSimulationInfo</code> | Retrieve simulation parameters from the HDF5 file.                          |
| <code>getChargeDensity</code>  | Retrieve charge density data from the HDF5 file.                            |
| <code>getPotentials</code>     | Retrieve potentials data from the HDF5 file.                                |
| <code>getElectricFields</code> | Retrieve electric fields data from the HDF5 file.                           |

The following provides a description of the input and output parameters for each function and expected values:

| <code>writeGrid</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>writeGrid(h5Filename, gridId, type, x, y, z, overwrite)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <ul style="list-style-type: none"><li>• <code>h5Filename</code> - A string with the HDF5 filename to which grid data shall be written.</li><li>• <code>gridId</code> - Unique integer ID of the grid to be used in the HDF5 file</li><li>• <code>type</code> - String for either rectilinear grid or list of points - POLYVERTEX or 3DRECTMESH</li><li>• <code>x</code> - An array of x positions. For 3DRECTMESH, <code>x</code> is a monotonic list of <math>N_x</math> points defining the points along the x dimension. The points in the array need not be evenly spaced. For POLYVERTEX, <code>x</code> is an arbitrary list of <math>N_{\text{points}}</math> values corresponding to the x coordinates of the list.</li><li>• <code>y</code> - An array of y positions. For 3DRECTMESH, <code>y</code> is a monotonic list of <math>N_y</math> points defining the points along the y dimension. The points in the array need not be evenly spaced. For POLYVERTEX, <code>y</code> is an arbitrary list of <math>N_{\text{points}}</math> values corresponding to the y coordinates of the list.</li><li>• <code>z</code> - An array of z positions. For 3DRECTMESH, <code>z</code> is a monotonic list of <math>N_z</math> points defining the points along the z dimension. The points in the array need not be evenly spaced. For POLYVERTEX, <code>z</code> is an arbitrary list of <math>N_{\text{points}}</math> values corresponding to the z coordinates of the list.</li><li>• <code>overwrite</code> - Flag to overwrite an existing HDF5 file of the given name. Default is FALSE.</li></ul> |



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>readGrid</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>[x, y, z] = readGrid(h5Filename, gridId)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <ul style="list-style-type: none"> <li>• <b>x</b> - Output array with the x points defining the grid. For 3DRECTMESH, the array will be an <math>[N_x, N_y, N_z]</math> array corresponding to the x coordinates in the entire grid.</li> <li>• <b>y</b> - Output array with the y points defining the grid.</li> <li>• <b>z</b> - Output array with the z points defining the grid.</li> <li>• <b>h5Filename</b> - A string with the HDF5 filename from which grid data shall be read. The file can be a grid file or an output file.</li> <li>• <b>gridId</b> - Grid integer ID to retrieve</li> </ul>                                                                                                                                                                                                                                                                                                                                                                      |
| <b>getSimulationInfo</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>data = nses.getSimulationInfo(h5FileName, parameterName, gridId)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <ul style="list-style-type: none"> <li>• <b>h5Filename</b> - A string with the HDF5 filename from which to retrieve information</li> <li>• <b>parameterName</b> - A string of which parameter to retrieve. Options include “electrodes” and “grid.”</li> <li>• <b>gridId</b> - Grid integer ID for the grid data that should be retrieved if the <b>parameterName</b> = “grid.”</li> <li>• <b>data</b> - On output, the data variable may take on the following forms depending on the selection for <b>parameterName</b>: <ol style="list-style-type: none"> <li>1. <b>electrodes</b> - An <math>[N_{\text{electrodes}}, 1]</math> array of the block IDs used as electrode excitations.</li> <li>2. <b>grid</b> - In the case of reading a grid, <b>data</b> will actually be three separate outputs and follow identically the format described for <b>readGrid</b>. Thus, the syntax will be <code>[x, y, z] = nses.getSimulationInfo(...)</code>.</li> </ol> </li> </ul> |
| <b>getChargeDensity</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>rho = nses.getChargeDensity(h5FileName, electrodeId)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <ul style="list-style-type: none"> <li>• <b>h5Filename</b> - A string with the HDF5 filename from which to retrieve information</li> <li>• <b>electrodeId</b> - Electrode integer ID for the charge density data that should be retrieved. This ID corresponds to the mesh block number provided in the input deck ELECTRODE tag.</li> <li>• <b>rho</b> - An <math>[N_e, 1]</math> array of the potential at each of the elements in the mesh.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>getPotentials</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>phi = nses.getPotentials(h5FileName, gridId, electrodeId)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <ul style="list-style-type: none"> <li>• <b>h5Filename</b> - A string with the HDF5 filename from which to retrieve information</li> <li>• <b>gridId</b> - Grid integer ID for the potential data that should be retrieved</li> <li>• <b>electrodeId</b> - Electrode integer ID for the potential data that should be retrieved. This ID corresponds to the mesh block number provided in the input deck ELECTRODE tag.</li> <li>• <b>phi</b> - An array of the potential at each of the points in the particular grid for a particular electrode. For 3DRECTMESH grids, <b>phi</b> is an <math>[N_x, N_y, N_z]</math> array. For POLYVERTEX grids, <b>phi</b> is an <math>[N_{\text{points}}]</math> array.</li> </ul>                                                                                                                                                                                                                                                       |



### getElectricFields

```
[Ex, Ey, Ez] = nses.getElectricFields(h5FileName, gridId, electrodeId)
```

- h5Filename - A string with the HDF5 filename from which to retrieve information
- gridId - Grid integer ID for the electric field data that should be retrieved
- electrodeId - Electrode integer ID for the potential data that should be retrieved. This ID corresponds to the mesh block number provided in the input deck ELECTRODE tag.
- Ex - An array of the  $E_x$  electric field component at each of the points in the particular grid for a particular electrode. For 3DRECTMESH grids, Ex is an  $[N_x, N_y, N_z]$  array. For POLYVERTEX grids, Ex is an  $[N_{\text{points}}]$ .
- Ey - An array of the  $E_y$  electric field component at each of the points in the particular grid. The conventions for Ex apply as well.
- Ez - An array of the  $E_z$  electric field component at each of the points in the particular grid. The conventions for Ex apply as well.

As a general example of processing function usage, consider that the user would like to create a 3D uniform grid and a line of points for calculating potentials and fields.

### Pre-processing - Grid file creation

```
import nses
import numpy as np

# Create a 3DRECTMESH grid
x = np.linspace(-1.0, 1.0, 21)
y = np.linspace(0.0, 2.0, 21)
z = np.linspace(-2.0, 2.0, 41)

writeGrid('grid.h5', 0, '3DRECTMESH', x, y, z)

# Create a list of points
x = np.array([0.0, 1.0, 2.0, 3.0])
y = np.array([0.0, 2.0, 4.0, 6.0])
z = np.array([2.0, 2.0, 2.0, 2.0])

writeGrid('grid.h5', 1, 'POLYVERTEX', x, y, z)
```

When preparing the grid file, the user also creates the geometry, mesh, and input file. Once all inputs are completed, the user runs the application, and then moves to post-processing. The user needs to perform a weighted summation of the potentials and electric fields along the line of points specified above.



## Post-processing

```
import nses
outputFilename = 'example.in.h5'
electrodes = nses.getSimulationInfo(outputFilename, 'electrodes')
e5 = np.where(electrodes == 5)
e6 = np.where(electrodes == 6)
gridId = 1
x, y, z = nses.getSimulationInfo(outputFilename, 'grid', gridId)

phi5 = nses.getPotentials(outputFilename, gridId, e5)
phi6 = nses.getPotentials(outputFilename, gridId, e6)
phiNew = 3.0*phi5 - 2.0*phi6

Ex5, Ey5, Ez5 = nses.getElectricFields(outputFilename, gridId, e5)
Ex6, Ey6, Ez6 = nses.getElectricFields(outputFilename, gridId, e6)
ExNew = 3.0*Ex5 - 2.0*Ex6
EyNew = 3.0*Ey5 - 2.0*Ey6
EzNew = 3.0*Ez5 - 2.0*Ez6

# Additional processing ...
```

The following Examples section has many specific use cases.

## 7 Examples

The following is a collection of detailed examples that demonstrate the typical Nullspace ES simulation workflow for a variety of problem types. Recall the typical Nullspace ES simulation workflow can be summarized as follows:

- CAD modeling and meshing
- Nullspace ES input deck configuration
- Grid file preparation
- Nullspace ES execution
- Post-processing analysis

All of the examples are included in the `examples/` directory contained in the NSES installation path.





## 7.1 Disk

Consider a disk of radius  $R = 1.0$  (m), oriented in the  $z = 0$   $x$ - $y$  plane, and held to a constant potential of  $V_0 = 1.0$  (V). The disk is one of a few bounded geometries in which analytic formulations are known for charge density, potential, and E-fields.

The surface charge density for ( $r < R$ ) on the disk is given by

$$\rho(r) = \frac{4V_0\epsilon_0}{\pi\sqrt{R^2 - r^2}}. \quad (9)$$

The potential is given by

$$V(r, z) = \frac{2V_0}{\pi} \arcsin \left( \frac{2R}{\sqrt{(r+R)^2 + z^2} + \sqrt{(r-R)^2 + z^2}} \right). \quad (10)$$

The E-field radial component in the  $x$ - $y$  plane ( $r > R$ ) is given by

$$E_r = \frac{2V_0}{\pi} \frac{R}{r} \frac{1}{\sqrt{r^2 - R^2}} \quad (11)$$

The E-field  $z$ -component along the  $z$  axis is given by

$$E_z = \frac{2V_0}{\pi} \frac{R}{\sqrt{z^2 + R^2}} \quad (12)$$

The following describes the setup, execution, and post-processing for Nullspace ES for this problem along with a comparison to the analytic result. We will solve the problem as a DENSE system and as a COMPRESS system.

See examples/disk in the installation directory.

### DENSE

First, we solve the problem as a DENSE linear system.

#### 7.1.1 Step 1. Nullspace Prep mesh model

The mesh model can be created in Nullspace Prep using the following journal file code snippet. Notice we use an adaptive mesh since charge will rapidly change near the disk boundary.

```
reset

#{R = 1}
create surface circle radius {R} zplane

curve all scheme equal
curve all size {0.01}
trimesher surface gradation {1.1}
surface all scheme trimesh
mesh surface all
```



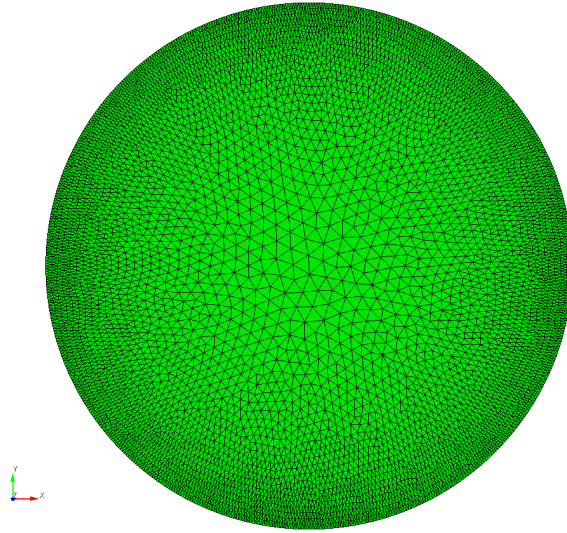


Figure 15: Disk mesh ( $R = 1.0$  m).

```
delete block all
block 1 surface 1

surface 1 smooth scheme centroid area pull
smooth surface 1

set abaqus precision 15
export abaqus "disk.inp" dimension 3 overwrite everything

quality surface 1 shape global list
```

### 7.1.2 Step 2. Grids

To compare Nullspace ES with the analytic formulations in (10-12), a grid file is created for a ring  $r = 2.0$  in the  $x$ - $y$  plane with  $N_\theta = 1001$  points and along the  $z$ -axis  $(0.1, 4.0)$  with  $N_{\text{points}} = 70$  points.

The grids described above are easily generated with the NSES API as shown below. Note all grid types are POLYVERTEX.

```
import numpy as np
import nses
import math

fileName = 'grid.h5'

def sind(x):
    return math.sin(math.radians(x))

def cosd(x):
    return math.cos(math.radians(x))

def makeRingPoints(z0, r, nTheta):
    thetad = np.linspace(0, 360.0, nTheta)
    x = np.zeros(nTheta)
```



```

y = np.zeros(nTheta)
z = np.zeros(nTheta)
for iTheta in range(nTheta):
    x[iTheta] = r * cosd(thetad[iTheta])
    y[iTheta] = r * sind(thetad[iTheta])
    z[iTheta] = z0

return x, y, z

nP = 1001
x = np.linspace(-0.9, 0.9, nP)
y = np.zeros(nP)
z = np.zeros(nP)
nses.writeGrid(fileName, 1, 'POLYVERTEX', x, y, z, True)

nTheta = 1001
z0 = 0.0
r = 2.0
x, y, z = makeRingPoints(z0, r, nTheta)
nses.writeGrid(fileName, 2, 'POLYVERTEX', x, y, z, False)

nP = 70
x = np.zeros(nP)
y = np.zeros(nP)
z = np.linspace(0.1, 4.0, nP)
nses.writeGrid(fileName, 3, 'POLYVERTEX', x, y, z, False)

```

### 7.1.3 Step 3. Input Deck

The disk mesh has 11,730 triangle elements, a relatively small mesh. A typical low-end work station will easily have RAM to solve this problem as a dense linear system. Moreover, in this problem there is only one potential surface and thus only one electrode. There is no need to specify an electrode FILE or LIST since NSES will default to setting the electrode list to the one-and-only-one potential surface in the mesh file.

Putting all the pieces together we arrive at the following input deck. This input deck will compute the surface charge densities ( $\rho$ ) and store the results in the HDF5 file. It will also compute (and store in the HDF5 file) the potential ( $\phi$ ) and E-fields ( $\mathbf{E}$ ) for the single electrode and all grids in the grid file. We will not perform any Paraview-related visual workflows so there is no VTU or XMF desired output.

```

<MESH_BEG>
  FILE = disk.inp
<MESH_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  GRID = grid.h5
<OUTPUT_END>

```



### 7.1.4 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses disk.in
```

The Nullspace ES output should look similar to the following.

```
*****
***                      Nullspace Electrostatics                      ***
***                      Nullspace, Inc.                                ***
***                      Copyright 2024                                ***
***                      Version - 2024 R1                             ***
*****

parsing precision from disk.in ... done ... (single)
parsing full deck disk.in ... done
parsing grid file grid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    symmetry rotation not applied
  processing symmetry rotation end
processing simulation base end
parsing mesh disk.inp ... done
processing mesh disk.inp ... done
  6180 nodes
  11730 elements
  17909 edges
  11730 unknowns
  1 surfaces

  minimum edge length is 0.00902992 owned by element 172
  maximum edge length is 0.0742907 owned by element 11670
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

1 of 1 electrodes selected for excitation
1

xy symmetry : NONE : 0.000000
xz symmetry : NONE : 0.000000
yz symmetry : NONE : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 524.9 (MB) : 0.5 (GB) : 0.0 (TB)
memory required dense B (SINGLE) : 0.0 (MB) : 0.0 (GB) : 0.0 (TB)

hdf create simulation skeleton beg
  create pre process beg
  create pre process end
  create post process beg
  create post process end
hdf create simulation skeleton end

simulation beg

  B fill beg
  B fill end
```



```

B fill time : 0 (sec)

Z fill beg
  dense fill beg
  dense fill end
Z fill end
Z fill time : 1 (sec)

J solve beg
  dense solve beg
  dense solve end
J solve end
J solve time : 1 (sec)

output beg

  hdf output density beg
    rho :      1
  hdf output density end

  vtu output density beg
    vtu output not specified
  vtu output density end

  hdf V and E potential beg
    V and E :      1 :      1 : POLYVERTEX : (0 sec) (0 sec)
    V and E :      1 :      2 : POLYVERTEX : (0 sec) (0 sec)
    V and E :      1 :      3 : POLYVERTEX : (0 sec) (0 sec)
  hdf V and E potential end

  xmf output beg
    xmf output not specified
  xmf output end

output end

simulation end
simulation time : 3 (sec)

Simulation completed

```

### 7.1.5 Step 5. Post-processing Analysis

In Figures 16 - 19 we see excellent agreement between Nullspace ES and the analytic solution. The plots are generated using the Python post-processing functions as follows:

```

import nses
import numpy as np
import math
from matplotlib import pyplot

fileName = 'disk.in.h5'

def V_analytic(V0, R, z, r):
    Q = 2.0 * R * V0 / math.pi
    Va = (Q / R) * math.asin(2.0 * R / (np.sqrt(np.power(r+R,2) + np.power(z,2)) + np.
        sqrt(np.power(r-R,2) + np.power(z,2))))

```



```

    return Va

# Valid for r in the xy-plane and r > R
def Er_analytic(V0, R, r):
    Q = 2.0 * R * V0 / math.pi
    Er = Q / (np.power(r,2) * np.sqrt(1.0 - np.power(R/r,2)))
    return Er

# Valid along the z-axis
def Ez_analytic(V0, R, z):
    Q = 2.0 * V0 / math.pi
    Ez = Q * R / (R**2 + np.power(z, 2))
    return Ez

V0 = 1.0
R = 1.0
electrodeId = 1

gridId = 2

x, y, z = nses.getSimulationInfo(fileName, 'grid', gridId)
V = nses.getPotentials(fileName, gridId, electrodeId)
Ex, Ey, Ez = nses.getElectricFields(fileName, gridId, electrodeId)

z0 = 0.0
r0 = 2.0
nTheta = 1001
theta = np.linspace(0, 360, nTheta)

Va = V_analytic(V0, R, z0, r0)
Va = Va * np.ones(nTheta)

fig2a = pyplot.figure()
ax = fig2a.add_subplot(1, 1, 1)
ax.plot(theta, Va, 'k', linewidth=3.0)
ax.plot(theta, V, 'r', linewidth=2.0)
ax.set_xlabel('$\\theta$ (deg)')
ax.set_ylabel('Potential (V)')
ax.legend(['Analytic', 'Nullspace ES'], loc=1)
ax.set_title('Potential along ring (r = 2.0 m)')
ax.set_ylim(0.25, 0.5)
ax.grid()
fig2a.savefig('V_compare_r_2p0.png', transparent=True, bbox_inches='tight')

Er = np.sqrt(np.power(Ex, 2) + np.power(Ey, 2) + np.power(Ez, 2))
Er_mean = np.mean(Er);
Er_mean = Er_mean * np.ones(nTheta)
Era = Er_analytic(V0, R, r0)
Era = Era * np.ones(nTheta)

fig2b = pyplot.figure()
ax = fig2b.add_subplot(1, 1, 1)
ax.plot(theta, Era, 'k', linewidth=3.0)
ax.plot(theta, Er, 'r', linewidth=2.0)
ax.set_xlabel('$\\theta$ (deg)')
ax.set_ylabel('$E_{r}$ (V/m)')
ax.legend(['Analytic', 'Nullspace ES'], loc=1)
ax.set_title('$E_{r}$ along ring (r = 2.0 m)')

```



```

ax.set_ylim(0.0, 0.25)
ax.grid()
fig2b.savefig('Er_compare_r_2p0.png', transparent=True, bbox_inches='tight')

gridId = 3

x, y, z = nses.getSimulationInfo(fileName, 'grid', gridId)
V = nses.getPotentials(fileName, gridId, electrodeId)
Ex, Ey, Ez = nses.getElectricFields(fileName, gridId, electrodeId)

r = 0.0
Va = np.zeros(len(z))
Eza = np.zeros(len(z))
for i in range(0, len(z)):
    Va[i] = V_analytic(V0, R, z[i], r)
    Eza[i] = Ez_analytic(V0, R, z[i])

fig3a = pyplot.figure()
pyplot.plot(z, Va, 'k', linewidth=3.0)
pyplot.plot(z, V, 'r', linewidth=2.0)
pyplot.xlabel('z (m)')
pyplot.ylabel('Potential (V)')
pyplot.legend(['Analytic', 'Nullspace ES'], loc=1)
pyplot.title('Potential along $z$-axis')
pyplot.grid()
fig3a.savefig('V_compare_zaxis.png', transparent=True, bbox_inches='tight')

fig3b = pyplot.figure()
pyplot.plot(z, Eza, 'k', linewidth=3.0)
pyplot.plot(z, Ez, 'r', linewidth=2.0)
pyplot.xlabel('z (m)')
pyplot.ylabel('$E_z$ (V/m)')
pyplot.legend(['Analytic', 'Nullspace ES'], loc=1)
pyplot.title('$E_z$ along $z$-axis')
pyplot.grid()
fig3b.savefig('Ez_compare_zaxis.png', transparent=True, bbox_inches='tight')

pyplot.show(block=True)

```



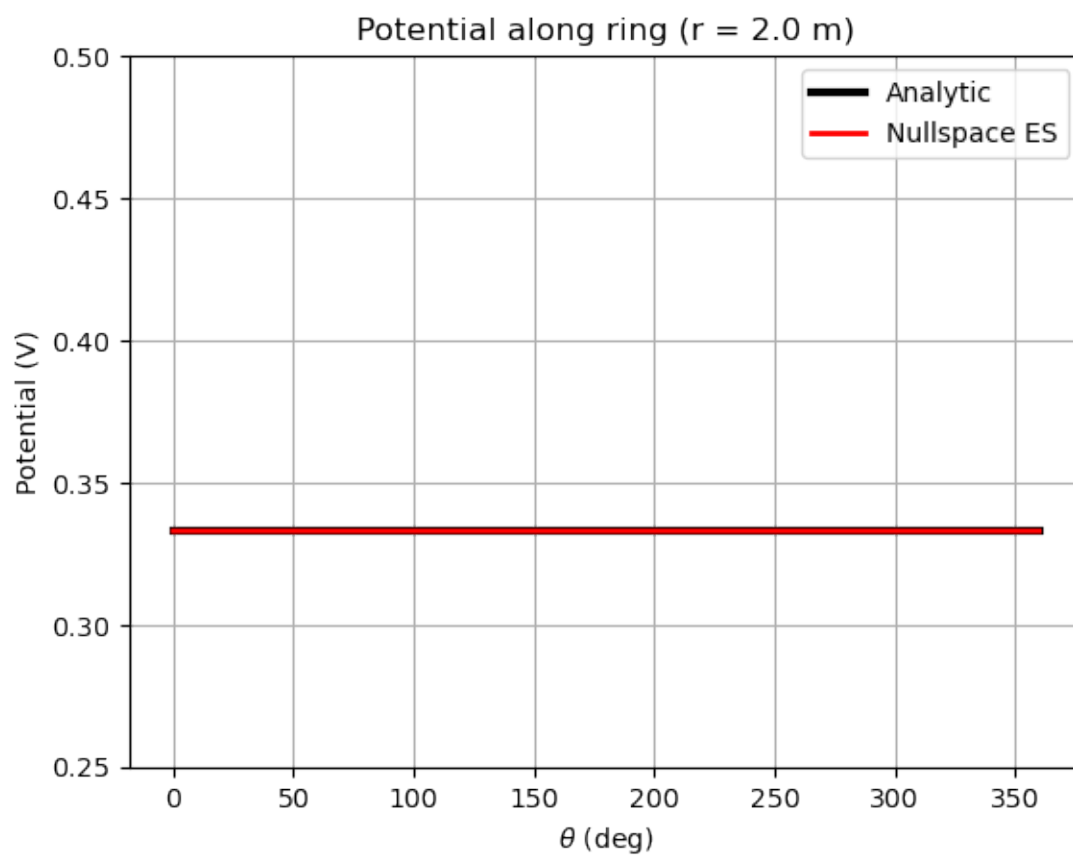


Figure 16: NSES potential  $V$  vs analytic along ring ( $r = 2.0$  m).





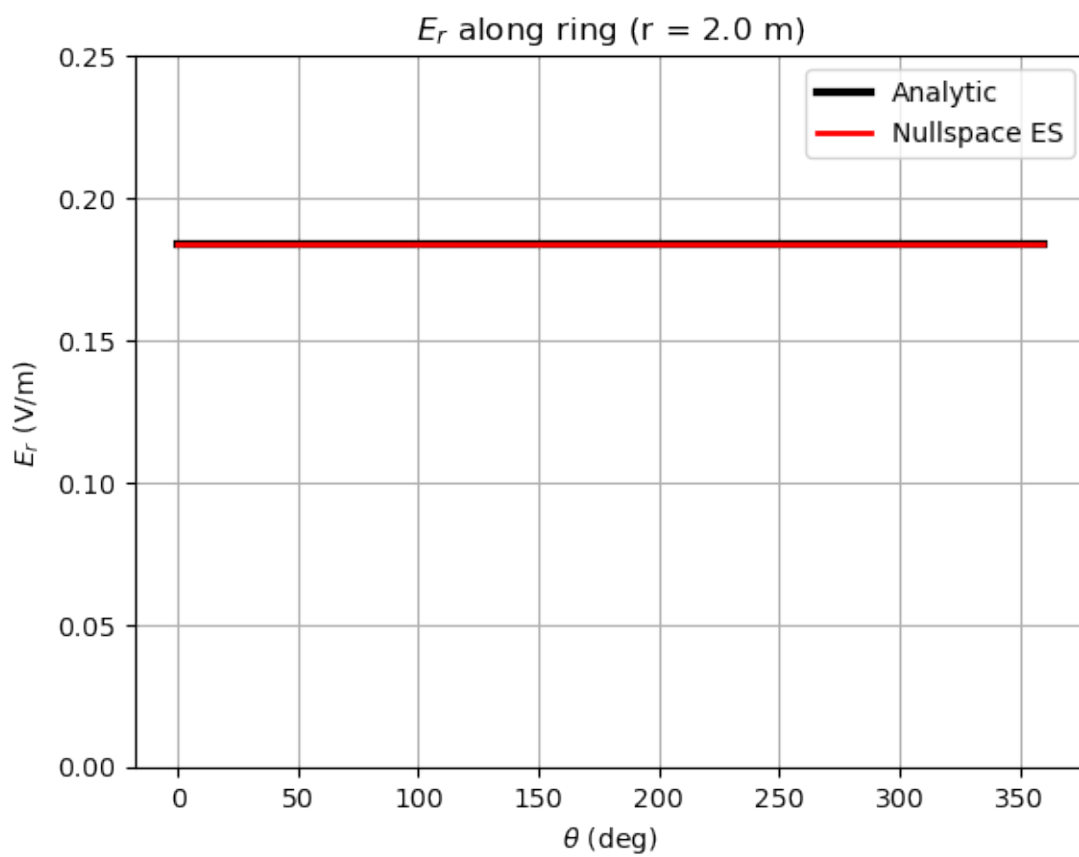


Figure 17: NSES  $E_r$  vs analytic along ring ( $r = 2.0$  m).



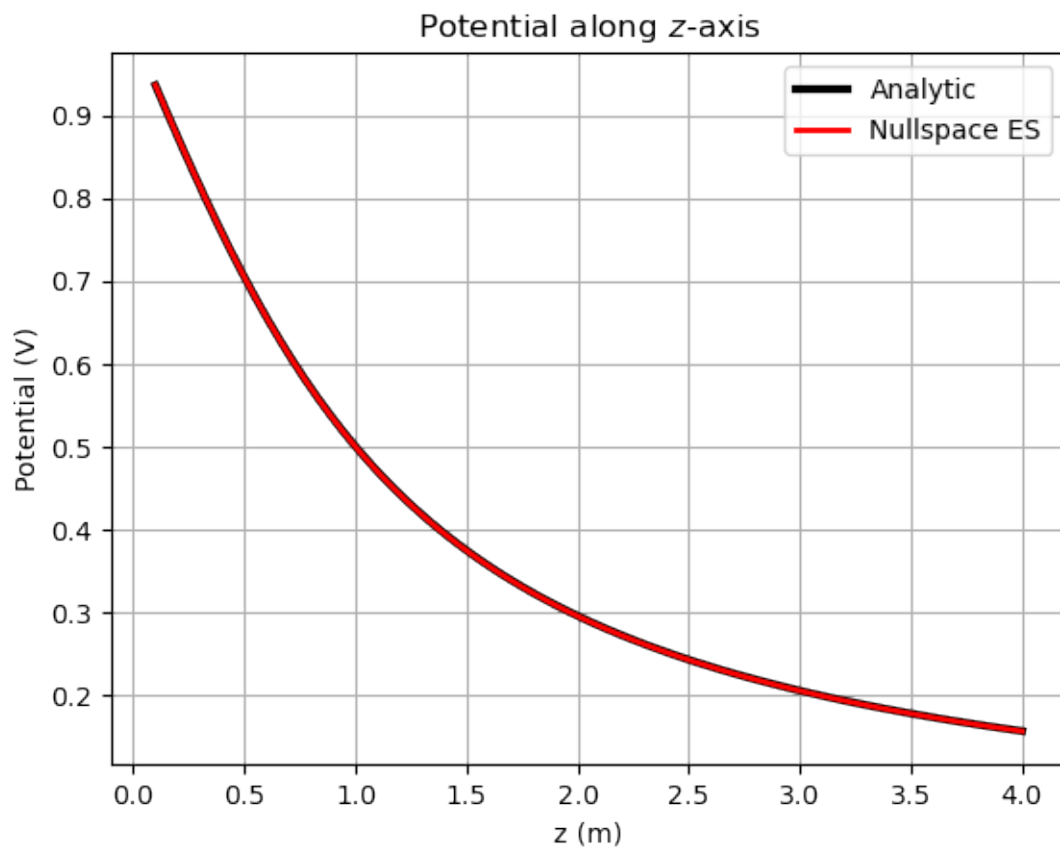


Figure 18: NSES potential  $V$  vs analytic along z-axis.

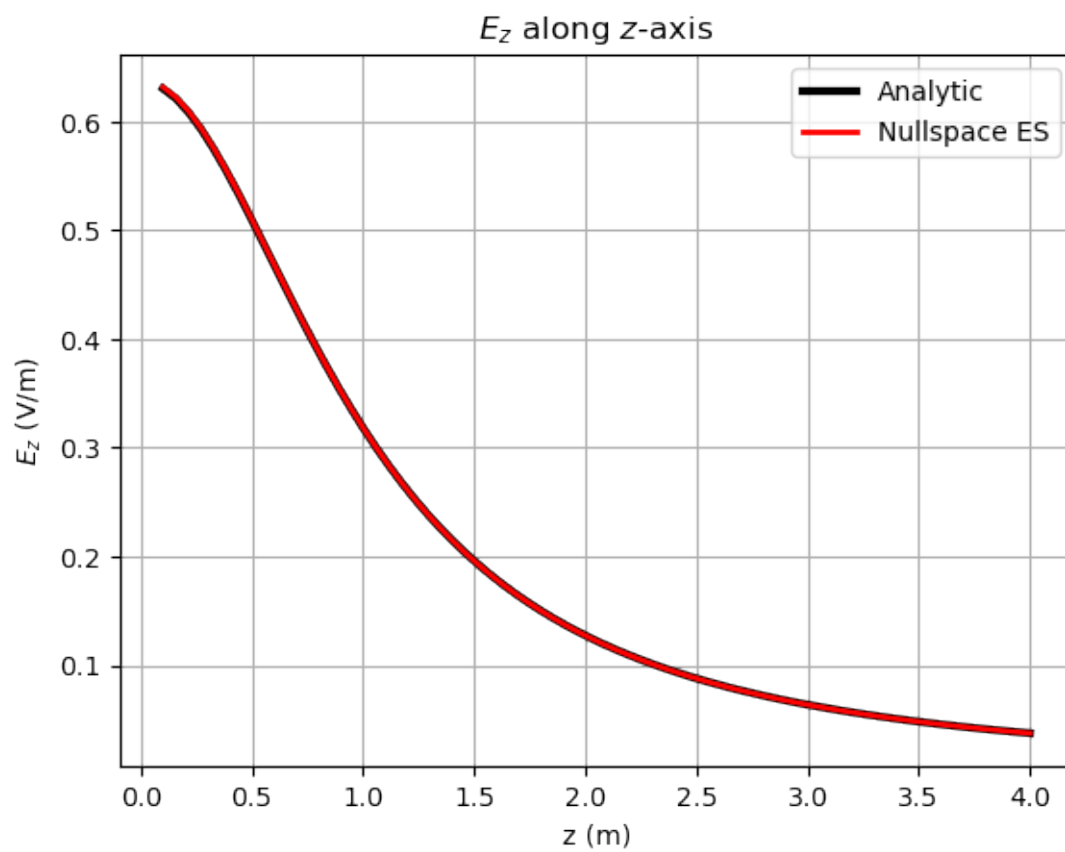


Figure 19: NSES  $E_z$  vs analytic along z-axis.



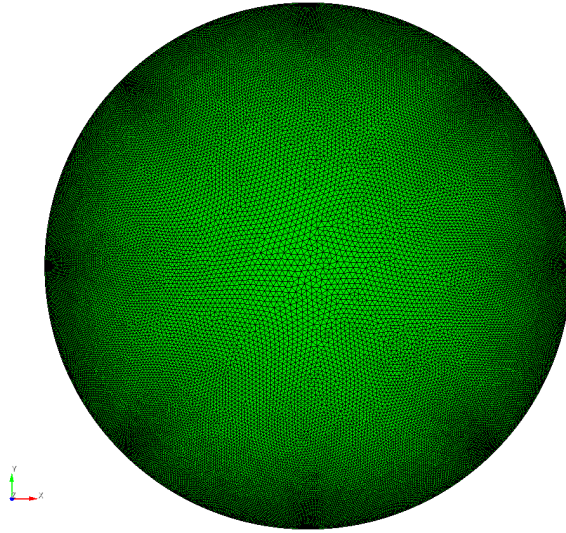


Figure 20: Disk ( $R = 1.0$  m) with a fine mesh for demonstrating compression.

## COMPRESS

Next, we solve the problem as a compressed linear system. The workflow is nearly identical. However, we will make a denser mesh to illustrate compression.

### 7.1.6 Step 1. Nullspace Prep mesh model

The mesh model can be created in Nullspace Prep using the following journal file code snippet. As before, we use an adaptive mesh since charge will rapidly change near the disk boundary. Notice differences in the adaptive mesh parameters compared to the DENSE case.

```
reset

#{R = 1}
create surface circle radius {R} zplane

curve all scheme equal
curve all size {0.005}
trimesher surface gradation {1.02}
surface all scheme trimesh
mesh surface all

delete block all
block 1 surface 1

surface 1 smooth scheme centroid area pull
smooth surface 1

set abaqus precision 15
export abaqus "diskCompress.inp" dimension 3 overwrite everything

quality surface 1 shape global list
```



### 7.1.7 Step 2. Grids

This step is identical to Step 2 for DENSE case.

### 7.1.8 Step 3. Input Deck

The disk mesh has 69,287 triangle elements. This size mesh could pose a problem for some workstations. We will solve it with compression to alleviate the RAM load. Moreover, in this problem there is only one potential surface and thus only one electrode. There is no need to specify an electrode FILE or LIST since Nullspace ES will default to setting the electrode list to the one-and-only-one potential surface in the mesh file.

Putting all the pieces together we arrive at the following input deck. (Note the input deck is identical to the (SYSTEM = DENSE) case except now (SYSTEM = COMPRESS)). This input deck will compute the surface charge densities ( $\rho$ ) and store the results in the HDF5 file. It will also compute (and store in the HDF5 file) the potential ( $\phi$ ) and E-fields ( $\mathbf{E}$ ) for the single electrode and all grids in the grid file. We will not perform any Paraview-related visual workflows so there is no VTU or XMF desired output.

```
<MESH_BEG>
  FILE = diskCompress.inp
<MESH_END>

<SOLVER_BEG>
  SYSTEM = COMPRESS
<SOLVER_END>

<OUTPUT_BEG>
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  GRID = grid.h5
<OUTPUT_END>
```

### 7.1.9 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses diskCompress.in
```

The Nullspace ES output should look similar to the following. Compare to the DENSE case. Here we see verbiage related to a compress run.

```
*****
***                               Nullspace Electrostatics          ***
***                               Nullspace, Inc.                    ***
***                               Copyright 2024                      ***
***                               Version - 2024 R1                   ***
*****

parsing precision from diskCompress.in ... done ... (hybrid)
parsing full deck diskCompress.in ... done
parsing grid file grid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
```



```

    symmetry rotation not applied
  processing symmetry rotation end
processing simulation base end
parsing mesh diskCompress.inp ... done
processing mesh diskCompress.inp ... done
  35273 nodes
  69287 elements
  104559 edges
  69287 unknowns
  1 surfaces

  minimum edge length is 0.00397785 owned by element 806
  maximum edge length is 0.0310636 owned by element 69109
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

1 of 1 electrodes selected for excitation
1

xy symmetry : NONE : 0.000000
xz symmetry : NONE : 0.000000
yz symmetry : NONE : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory estimated 60% compress Z (HYBRID) : 7325.3 (MB) : 7.2 (GB) : 0.0 (TB)
memory estimated 80% compress Z (HYBRID) : 3662.6 (MB) : 3.6 (GB) : 0.0 (TB)
memory estimated 95% compress Z (HYBRID) : 915.7 (MB) : 0.9 (GB) : 0.0 (TB)
memory required dense B (HYBRID) : 0.3 (MB) : 0.0 (GB) : 0.0 (TB)

hdf create simulation skeleton beg
  create pre process beg
  create pre process end
  create post process beg
  create post process end
hdf create simulation skeleton end

simulation beg

  B fill beg
  B fill end
  B fill time : 0 (sec)

  Z fill beg
  compress fill beg
    ZBlocks initialize ... done
    ZBlocks build ... done
    ZBlocks fill ... done (19 sec)
  compress fill end
  Z fill end
  Z fill time : 19 (sec)

  J solve beg
  compress solve beg
    ZBlocks LU decompose ... done (54 sec)
    ZBlocks backsolve ...done (0 sec)
  compress solve end
  J solve end
  J solve time : 54 (sec)

```



```

output beg

  hdf output density beg
    rho :      1
  hdf output density end

  vtu output density beg
    vtu output not specified
  vtu output density end

  hdf V and E potential beg
    V and E :      1 :      1 : POLYVERTEX : (0 sec) (0 sec)
    V and E :      1 :      2 : POLYVERTEX : (0 sec) (0 sec)
    V and E :      1 :      3 : POLYVERTEX : (0 sec) (0 sec)
  hdf V and E potential end

  xmf output beg
    xmf output not specified
  xmf output end

output end

simulation end
simulation time : 74 (sec)

Simulation completed

```

See the Diagnostic Output discussion, and in particular the ACAProcessing.txt file discussion, for a workflow to monitor compression as the product is running.

### 7.1.10 Step 5. Post-processing Analysis

The comparison COMPRESS accuracy plots are essentially identical to the DENSE accuracy plots so those will not be repeated. To see the plots for the compressed run simply change the disk.in.h5 entry to diskCompress.in.h5 in the makePlots.py file.



## 7.2 Sphere

In this example we will demonstrate symmetry usage in two contexts. In the first context we will demonstrate pure geometric symmetry (Neumann boundary condition) for a sphere. In the second context, we will demonstrate usage of symmetry to solve a sphere above a ground plane (Dirichlet boundary condition).

### Geometry symmetry → Neumann boundary condition

Consider a sphere of radius  $R = 1.0$  (m) held to a constant potential of  $V_0 = 1.0$  (V). The sphere is another geometry in which analytic formulations are known for charge density, potential, and E-fields.

The surface charge density on the sphere is constant and given by

$$\rho(r) = \frac{V_0 \epsilon_0}{R}. \quad (13)$$

The potential is given by

$$V(r) = \begin{cases} 1.0, & \text{for } r \leq R \\ \frac{R}{r}, & \text{for } r > R. \end{cases} \quad (14)$$

The E-field radial component is given by

$$E_r = \begin{cases} 0.0, & \text{for } r < R \\ \frac{R}{r^2}, & \text{for } r > R. \end{cases} \quad (15)$$

The following describes the setup, execution, and post-processing for Nullspace ES for this problem along with a comparison to the analytic result.

### 7.2.1 Step 1. Nullspace Prep mesh model

Suppose the sphere is offset to a center location (0.0, 0.0, 1.5). We will exploit symmetry and mesh only one octant of the sphere though we are solving for the entire sphere. The mesh can be created in Nullspace Prep using the following journal file code snippet. Notice in this case the sphere has no edges, thus there is no need to use an adaptive mesh.

```
reset

create sphere radius 1 zpositive ypositive xpositive

move volume 1 z 1.5 include_merged

block 1 surface 4

surface 4 size 0.05
surface 4 scheme trimesh
mesh surface 4

export abaqus "sphere_octant.inp" overwrite everything
```





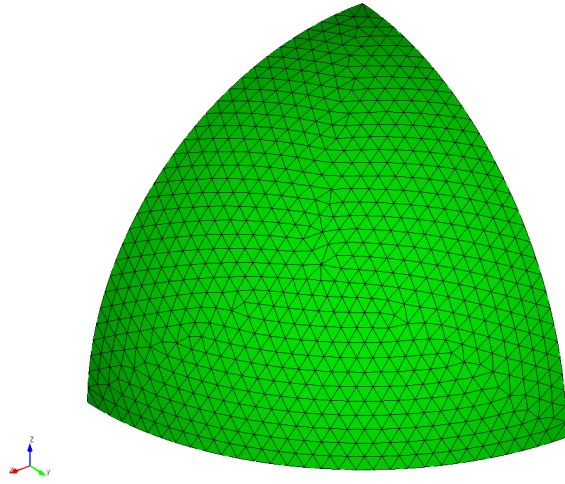


Figure 21: Sphere octant mesh ( $R = 1.0$  m, offset to  $z = 1.5$  m).

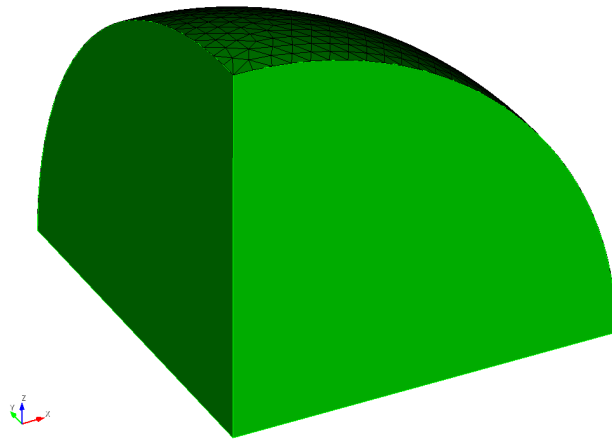


Figure 22: Back side of the octant illustrating unmeshed surfaces.

### 7.2.2 Step 2. Grids

We wish to validate the density, potential, and electric field against the analytic formulas above. For example, a sampling of 1001 points along the  $z$ -axis covering the sphere centered at  $(0.0, 0.0, 1.5)$  can be used to interrogate the interior and exterior values. We will also examine the electric field around the sphere. The grid is easily generated with the NSES API as shown below. Note the grid type is POLYVERTEX for the  $z$ -axis sampling and 3DRECTMESH for the electric field grid.

```
import numpy as np
import nses
import math

fileName = 'grid.h5'

nP = 1001
x = np.zeros(nP)
y = np.zeros(nP)
z = np.linspace(-0.5, 3.5, nP)
nses.writeGrid(fileName, 1, 'POLYVERTEX', x, y, z, True)

x = np.linspace(-2, 2, 31)
y = np.array([0])
z = np.linspace(-0.5, 3.5, 31)
nses.writeGrid(fileName, 2, '3DRECTMESH', x, y, z, False)
```

### 7.2.3 Step 3. Input Deck

As noted, we are using symmetry here about the  $xy$ -symmetry plane with an offset of  $z = 1.5$  as well as  $yz$ - and  $xz$ -symmetry. Since the symmetry imposed is geometric only, EVEN symmetry tags must be specified.

The octant-sphere mesh has 1,453 triangle elements, a relatively small mesh. A typical low-end work station will easily have RAM to solve this problem as a dense linear system. Moreover, in this problem there is only one potential surface and thus only one electrode. There is no need to specify an electrode FILE or LIST since Nullspace ES will default to setting the electrode list to the one-and-only-one potential surface in the mesh file.

Putting all the pieces together we arrive at the following input deck. This input deck will compute the surface charge densities ( $\rho$ ) and store the results in the HDF5 file. It will also compute (and store in the HDF5 file) the potential ( $\phi$ ) and E-fields ( $\mathbf{E}$ ) for the single electrode and all grids in the grid file. We will not perform any Paraview-related visual workflows so there is no VTU or XMF desired output.

```
<MESH_BEG>
  FILE = sphere_octant.inp
<MESH_END>

<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = EVEN
  SYMMETRY_PLANE_XY_OFFSET = 1.5
  SYMMETRY_PLANE_YZ = EVEN
  SYMMETRY_PLANE_YZ_OFFSET = 0.0
  SYMMETRY_PLANE_XZ = EVEN
  SYMMETRY_PLANE_XZ_OFFSET = 0.0
<SYMMETRY_END>
```



```

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  GRID = grid.h5
<OUTPUT_END>

```

## 7.2.4 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses sphere_octant.in
```

The Nullspace ES output should look similar to the following.

```

*****
***                               Nullspace Electrostatics                               ***
***                               Nullspace, Inc.                                       ***
***                               Copyright 2024                                       ***
***                               Version - 2024 R1                                    ***
*****

parsing precision from sphere_octant.in ... done ... (single)
parsing full deck sphere_octant.in ... done
parsing grid file grid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    symmetry rotation not applied
  processing symmetry rotation end
processing simulation base end
parsing mesh sphere_octant.inp ... done
processing mesh sphere_octant.inp ... done
  774 nodes
  1453 elements
  2226 edges
  1453 unknowns
  1 surfaces

  minimum edge length is 0.0387064 owned by element 55
  maximum edge length is 0.0781565 owned by element 54
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

1 of 1 electrodes selected for excitation
1

xy symmetry : EVEN : 1.500000
xz symmetry : EVEN : 0.000000
yz symmetry : EVEN : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 8.1 (MB) : 0.0 (GB) : 0.0 (TB)

```



```
memory required dense B (SINGLE) : 0.0 (MB) : 0.0 (GB) : 0.0 (TB)
```

```
hdf create simulation skeleton beg
  create pre process beg
  create pre process end
  create post process beg
  create post process end
hdf create simulation skeleton end
```

```
simulation beg
```

```
  B fill beg
  B fill end
  B fill time : 0 (sec)
```

```
  Z fill beg
    dense fill beg
    dense fill end
  Z fill end
  Z fill time : 0 (sec)
```

```
  J solve beg
    dense solve beg
    dense solve end
  J solve end
  J solve time : 0 (sec)
```

```
output beg
```

```
  hdf output density beg
    rho :      1
  hdf output density end
```

```
  vtu output density beg
    vtu output not specified
  vtu output density end
```

```
  hdf V and E potential beg
    V and E :      1 :      1 : POLYVERTEX : (0 sec) (0 sec)
    V and E :      1 :      2 : 3DRECTMESH : (0 sec) (0 sec)
  hdf V and E potential end
```

```
  xmf output beg
    xmf output not specified
  xmf output end
```

```
output end
```

```
simulation end
simulation time : 1 (sec)
```

```
Simulation completed
```



### 7.2.5 Step 5. Post-processing Analysis

In the Figures 23-26, we see the NSES octant-sphere symmetry model demonstrates very good accuracy compared to the analytic solution.

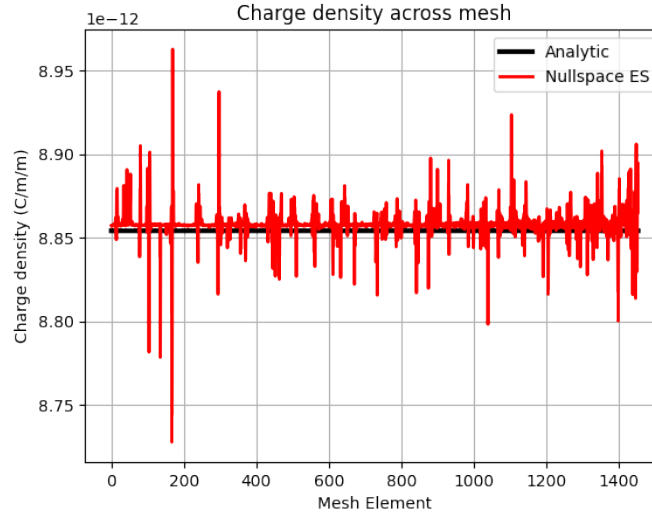


Figure 23: Octant-sphere comparison to analytic charge density.

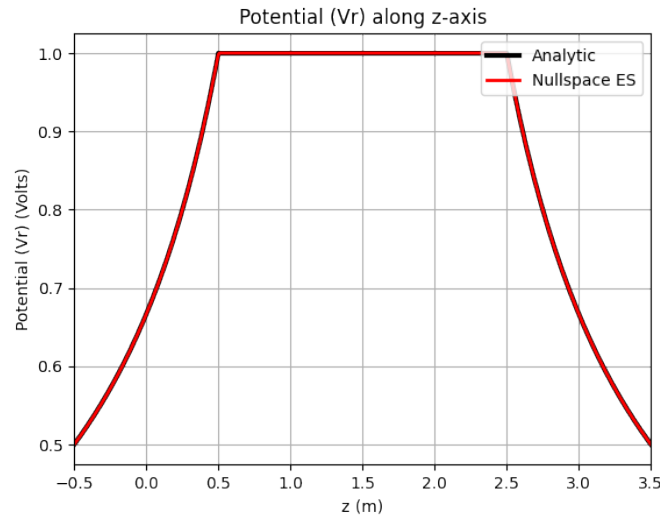


Figure 24: Octant-sphere comparison to analytic potential along  $z$ -axis.

### Ground plane → Dirichlet boundary condition.

Let's model the same sphere but instead with a ground plane at  $z = 0.0$ .

### 7.2.6 Step 1. Nullspace Prep mesh model

In this case we mesh a quarter of sphere ( $yz$  and  $xz$  symmetry planes) with center  $(0.0, 0.0, 1.5)$ . However, we do not create the physical ground plane. We instead invoke image theory with the NSES symmetry tags.



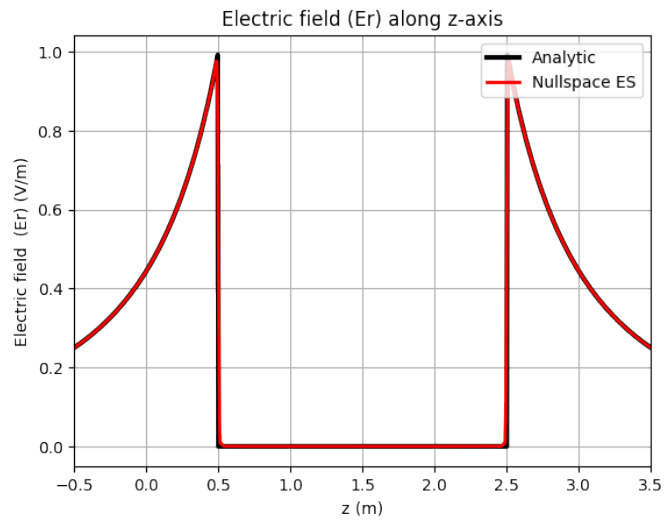


Figure 25: Octant-sphere comparison to electric field along  $z$ -axis.

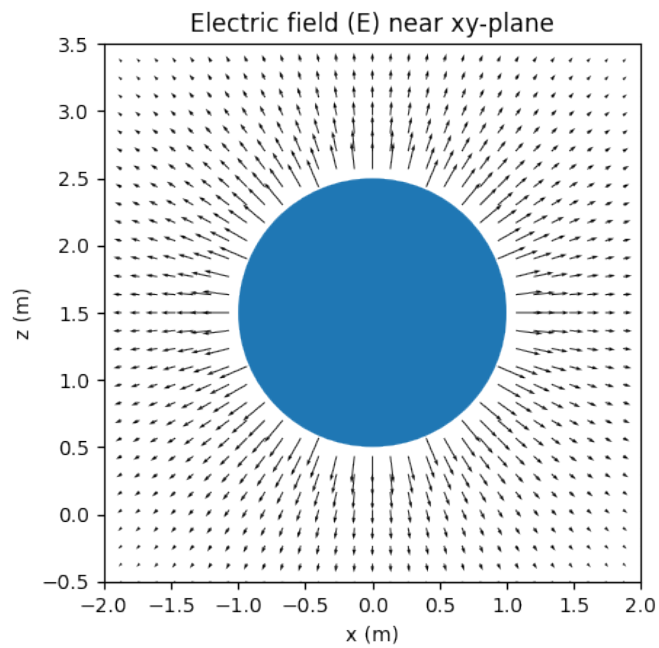


Figure 26: Octant-sphere electric field in the  $xz$  plane.



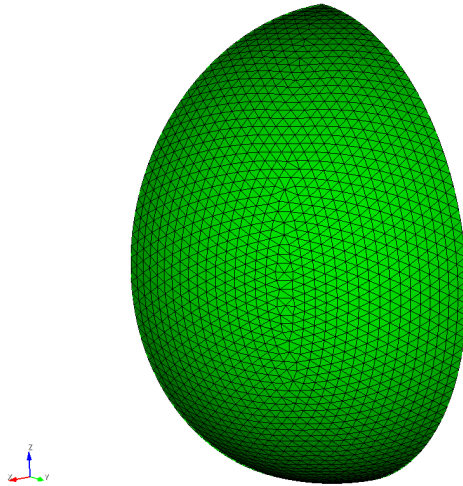


Figure 27: Quarter sphere mesh ( $R = 1.0$  m, offset to  $z = 1.5$  m).

The mesh model can be created in Nullspace Prep using the following journal file code snippet. Notice in this case the sphere has no edges, thus there is no need to use an adaptive mesh.

```
reset

create sphere radius 1 ypositive xpositive

move volume 1 z 1.5 include_merged

block 1 surface 3

surface 3 size 0.05
surface 3 scheme trimesh
mesh surface 3

export abaqus "sphere_ground.inp" overwrite everything
```

### 7.2.7 Step 2. Grids

We will use similar grids as before but focus only on the region of space between the sphere and the ground plane for the electric field quiver plot.

```
import numpy as np
import nses
import math

fileName = 'grid.h5'

nP = 1001
x = np.zeros(nP)
y = np.zeros(nP)
z = np.linspace(0.0, 3.5, nP)
nses.writeGrid(fileName, 1, 'POLYVERTEX', x, y, z, True)

x = np.linspace(-2, 2, 21)
```



```
y = np.array([0])
z = np.linspace(0.0, 0.5, 11)
nses.writeGrid(fileName, 2, '3DRECTMESH', x, y, z, False)
```

### 7.2.8 Step 3. Input Deck

To account for the ground plane, we invoke odd symmetry along the  $xy$  plane.  $yz$  and  $xz$  symmetries are used to reduce the sphere mesh to a quarter of full sphere. The input deck is essentially the same except for the symmetry context.

```
<MESH_BEG>
  FILE = sphere_ground.inp
<MESH_END>

<SYMMETRY_BEG>
  SYMMETRY_PLANE_XY = ODD
  SYMMETRY_PLANE_XY_OFFSET = 0.0
  SYMMETRY_PLANE_YZ = EVEN
  SYMMETRY_PLANE_YZ_OFFSET = 0.0
  SYMMETRY_PLANE_XZ = EVEN
  SYMMETRY_PLANE_XZ_OFFSET = 0.0
<SYMMETRY_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  GRID = grid.h5
<OUTPUT_END>
```

### 7.2.9 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses sphere_ground.in
```

The Nullspace ES output should look similar to the following.

```
*****
***                               Nullspace Electrostatics          ***
***                               Nullspace, Inc.                    ***
***                               Copyright 2024                     ***
***                               Version - 2024 R1                  ***
*****

parsing precision from sphere_ground.in ... done ... (single)
parsing full deck sphere_ground.in ... done
parsing grid file grid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    symmetry rotation not applied
  processing symmetry rotation end
```





```

processing simulation base end
parsing mesh sphere_ground.inp ... done
processing mesh sphere_ground.inp ... done
  1582 nodes
  3036 elements
  4617 edges
  3036 unknowns
  1 surfaces

  minimum edge length is 0.0353512 owned by element 2652
  maximum edge length is 0.0773604 owned by element 41
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

1 of 1 electrodes selected for excitation
1

xy symmetry : ODD : 0.000000
xz symmetry : EVEN : 0.000000
yz symmetry : EVEN : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 35.2 (MB) : 0.0 (GB) : 0.0 (TB)
memory required dense B (SINGLE) : 0.0 (MB) : 0.0 (GB) : 0.0 (TB)

hdf create simulation skeleton beg
  create pre process beg
  create pre process end
  create post process beg
  create post process end
hdf create simulation skeleton end

simulation beg

  B fill beg
  B fill end
  B fill time : 0 (sec)

  Z fill beg
  dense fill beg
  dense fill end
  Z fill end
  Z fill time : 3 (sec)

  J solve beg
  dense solve beg
  dense solve end
  J solve end
  J solve time : 0 (sec)

output beg

  hdf output density beg
  rho : 1
  hdf output density end

  vtu output density beg
  vtu output not specified

```



```

vtu output density end

hdf V and E potential beg
  V and E :      1 :      1 : POLYVERTEX : (0 sec) (0 sec)
  V and E :      1 :      2 : 3DRECTMESH : (0 sec) (0 sec)
hdf V and E potential end

xmf output beg
  xmf output not specified
xmf output end

output end

simulation end
simulation time : 3 (sec)

Simulation completed

```

### 7.2.10 Step 5. Post-processing Analysis

Figures 28 and 29 show the potential and electric field along the  $z$ -axis, respectively. Note that the potential goes to 0 at the location of the ground plane, and the electric field is at a local minimum.

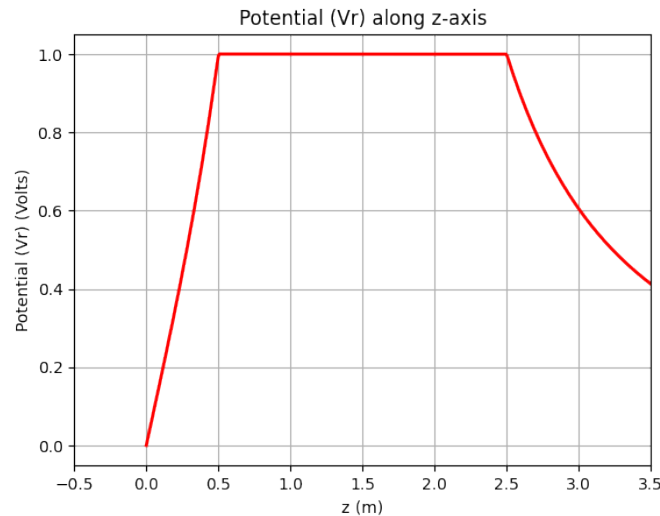


Figure 28: Potential along the  $z$ -axis for the sphere over a ground plane.

Compare the electric field lines in Figure 26, which uniformly radiate out from the sphere in all directions, to those in Figure 30. The effect of the ground plane is evident as  $E_t$  is approximately 0 near the ground plane.



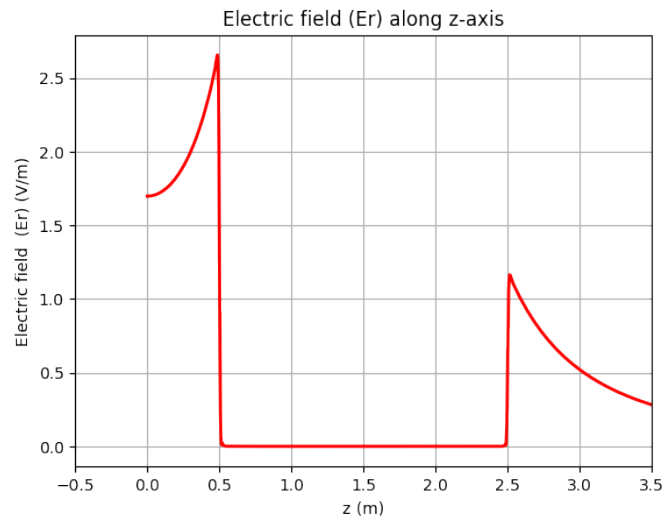


Figure 29: Electric field along the  $z$ -axis for the sphere over a ground plane.

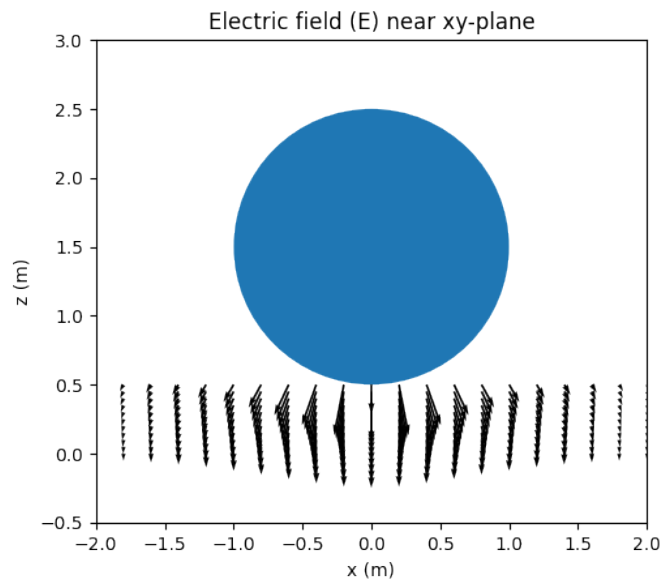


Figure 30: Electric field near  $xy$  plane (ground plane).

## 7.3 Planar Ion Trap

Planar ion traps are an interesting application for electrostatic simulation<sup>1</sup>. A detailed discussion of them is well beyond the scope of this example, and the user is encouraged to search the literature for a variety of geometries and application. Here, a simple planar, linear ion trap geometry is constructed and the initial electrostatic simulation is performed. Visualizations are presented of the electric field magnitude due to the RF rails, which are treated electrostatically for the spatiotemporal scale involved, as well as a voltage weighting of a set of DC electrodes. This example also illustrates the use of the multi-run workflow to support in-depth analysis of a given geometry.

### 7.3.1 Step 1. Nullspace Prep Mesh Model

The mesh model can be created in Nullspace Prep using the following journal file code snippet. Notice we use an adaptive mesh since charge will rapidly change near the electrode boundaries. The journal file is formulated to allow a variable number of DC control electrodes of a given size. For this example, the number of electrode pairs is 7. The units of the mesh are microns. The geometry resulting from this journal file is shown in Figure 31.

```
reset

#{numDCElectrodes = 7}
#{gap = 5}
#{dcWidth = 150}
#{dcLength = 100}
#{rfWidth = 50}
#{rfLength = (dcLength + gap)*(numDCElectrodes-1) + dcLength}
#{centerGndWidth = 100}
#{sideGndWidth = 500}
#{botGndWidth = centerGndWidth + 2*rfWidth + 2*dcWidth + 2*sideGndWidth + 6*gap}
#{botGndOffset = 2*gap}

create surface rectangle width {centerGndWidth/2} height {rfLength} zplane
#{sCenterGnd = Id("surface")}
move surface {sCenterGnd} x {-centerGndWidth/4}
block 1 surface {sCenterGnd}

create surface rectangle width {rfWidth} height {rfLength} zplane
#{sLeftRf = Id("surface")}
move surface {sLeftRf} x {-(centerGndWidth/2 + gap + rfWidth/2)}

create surface rectangle width {sideGndWidth} height {rfLength} zplane
#{sLeftSideGnd = Id("surface")}
move surface {sLeftSideGnd} x {-(centerGndWidth/2 + gap + rfWidth + gap + dcWidth +
    gap + sideGndWidth/2)}

create surface rectangle width {dcWidth} height {dcLength} zplane
#{sLeftDC1 = Id("surface")}
move surface {sLeftDC1} x {-(centerGndWidth/2 + gap + rfWidth + gap + dcWidth/2)} y {-
    rfLength/2 + dcLength/2}

Surface {sLeftDC1} copy move y {dcLength+gap} repeat {numDCElectrodes-1}
```

<sup>1</sup>J. Chiaverini, *et al.* "Surface-electrode architecture for ion-trap quantum information processing," Quantum Inf. Comput., 5:419, 2005.



```

curve all scheme equal
curve all except curve 4 size {gap/2}
curve 4 size {4*gap}
trimesher surface gradation {1.4}
surface all scheme trimesh
mesh surface all

Surface {sCenterGnd} copy reflect x
#{sCenterGnd2 = Id("surface")}
block 1 surface {sCenterGnd} {sCenterGnd2}

imprint all
merge all

Surface {sLeftRf} copy reflect x
#{sRightRf = Id("surface")}
block 2 surface {sLeftRf} {sRightRf}

Surface {sLeftSideGnd} copy reflect x
#{sRightSideGnd = Id("surface")}
block 3 surface {sLeftSideGnd} {sRightSideGnd}

Surface {sLeftDC1} to {sLeftDC1+numDCElectrodes-1} copy reflect x

#{iBlk = 4}
#{cnt = 0}
#{loop(numDCElectrodes)}
  block {iBlk} surface {sLeftDC1 + cnt}
  block {iBlk+1} surface {sLeftDC1 + 3 + cnt + numDCElectrodes}
  #{iBlk = iBlk + 2}
  #{cnt = cnt + 1}
#{endloop}

compress node
compress tri

export abaqus "ionTrap.inp" block all dimension 3 overwrite

```

### 7.3.2 Step 2. Grid File

For the initial analysis, the primary interest is in locating the pseudopotential (i.e., electric field magnitude) minimum due to the RF rails (mesh block 2). To simplify the analysis, a 3DRECTMESH grid is created at the center of the trap geometry.

```

import numpy as np
import nses
import math
from matplotlib import pyplot
from mpl_toolkits import mplot3d

fileName = 'ionTrapGrid.h5'
x = np.linspace(-30, 30, 61)
y = np.array([0])
z = np.linspace(20, 100, 81)
x = x*1e-6

```



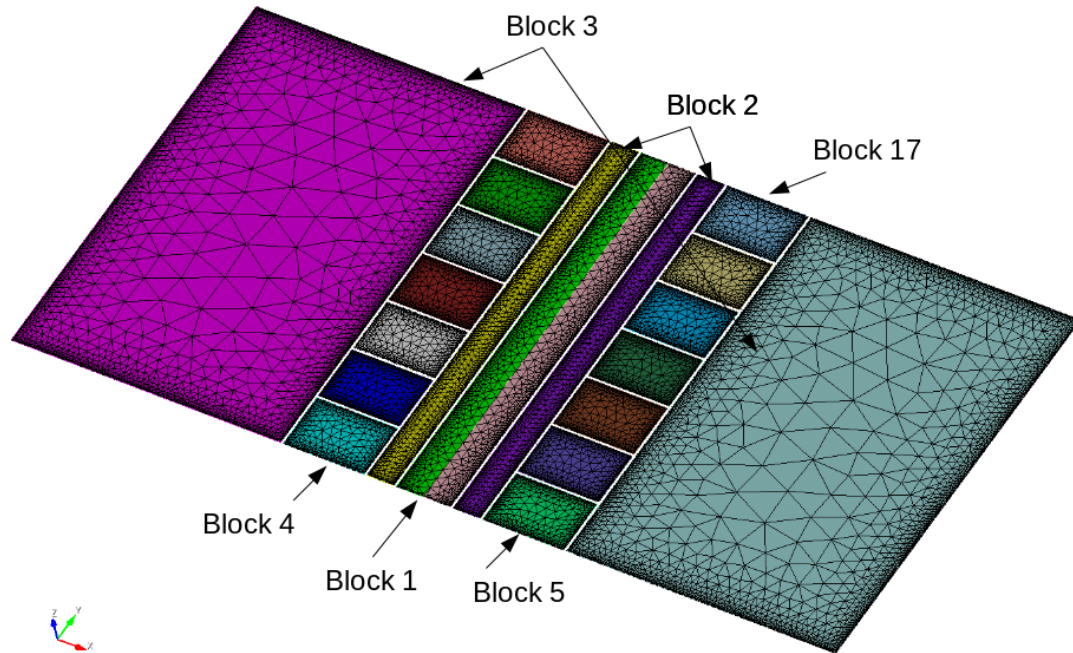


Figure 31: Linear ion trap geometry.

```
y = y*1e-6
z = z*1e-6
nses.writeGrid(fileName, 0, '3DRECTMESH', x, y, z, True)
```

### 7.3.3 Step 3. Input Deck

The trap mesh has 37,510 triangle elements. A reasonably powerful laptop or low-end work station will easily have RAM to solve this problem as a dense linear system. There is no need to solve the ground planes as electrode excitations, so the ELECTRODE\_EXCITATION tag with used with the LIST parameter to specify only the RF and DC electrodes.

Putting all the pieces together we arrive at the following input deck. This input deck will compute the surface charge densities ( $\rho$ ) and store the results in the HDF5 file. It will also compute (and store in the HDF5 file) the potential ( $\phi$ ) and E-fields ( $\mathbf{E}$ ) for the electrodes and the specified grid. VTU and XMF output is on for additional visualization options.

```
<MESH_BEG>
  FILE = ionTrap.inp
  SCALE = 1e-6
<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 2 4 5 6 7 8 9 10 11 12 13 14 15 16 17
<ELECTRODE_EXCITATION_END>

<SOLVER_BEG>
  SYSTEM = DENSE
```



```

<SOLVER_END>

<OUTPUT_BEG>
  GRID = ionTrapGrid.h5
  CHARGE_VTU = ON
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  XMF = ON
<OUTPUT_END>

```

### 7.3.4 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses ionTrap.in
```

The Nullspace ES output should look similar to the following.

```

*****
***                               Nullspace Electrostatics                               ***
***                               Nullspace, Inc.                                       ***
***                               Copyright 2024                                       ***
***                               Version - 2024 R1                                   ***
*****

parsing precision from ionTrap.in ... done ... (single)
parsing full deck ionTrap.in ... done
parsing grid file ionTrapGrid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    symmetry rotation not applied
  processing symmetry rotation end
processing simulation base end
parsing mesh ionTrap.inp ... done
processing mesh ionTrap.inp ... done
  22114 nodes
  37510 elements
  59605 edges
  37510 unknowns
  17 surfaces

  minimum edge length is 1.90868e-06 owned by element 847
  maximum edge length is 8.05979e-05 owned by element 16646
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

15 of 17 electrodes selected for excitation
2 4 5 6 7 8 9 10 11 12
13 14 15 16 17

xy symmetry : NONE : 0.000000
xz symmetry : NONE : 0.000000
yz symmetry : NONE : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 5367.3 (MB) : 5.2 (GB) : 0.0 (TB)

```



```
memory required dense B (SINGLE) : 2.1 (MB) : 0.0 (GB) : 0.0 (TB)
```

```
hdf create simulation skeleton beg
```

```
  create pre process beg
```

```
  create pre process end
```

```
  create post process beg
```

```
  create post process end
```

```
hdf create simulation skeleton end
```

```
simulation beg
```

```
  B fill beg
```

```
  B fill end
```

```
  B fill time : 0 (sec)
```

```
  Z fill beg
```

```
    dense fill beg
```

```
    dense fill end
```

```
  Z fill end
```

```
  Z fill time : 33 (sec)
```

```
  J solve beg
```

```
    dense solve beg
```

```
    dense solve end
```

```
  J solve end
```

```
  J solve time : 113 (sec)
```

```
output beg
```

```
  hdf output density beg
```

```
    rho :      2
```

```
    rho :      4
```

```
    rho :      5
```

```
    rho :      6
```

```
    rho :      7
```

```
    rho :      8
```

```
    rho :      9
```

```
    rho :     10
```

```
    rho :     11
```

```
    rho :     12
```

```
    rho :     13
```

```
    rho :     14
```

```
    rho :     15
```

```
    rho :     16
```

```
    rho :     17
```

```
  hdf output density end
```

```
  vtu output density beg
```

```
  vtu output density end
```

```
  hdf V and E potential beg
```

```
    V and E :      2 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      4 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      5 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      6 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      7 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      8 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :      9 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```

```
    V and E :     10 :      0 : 3DRECTMESH : (1 sec) (0 sec)
```





```

V and E :    11 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    12 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    13 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    14 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    15 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    16 :    0 : 3DRECTMESH : (1 sec) (0 sec)
V and E :    17 :    0 : 3DRECTMESH : (1 sec) (0 sec)
hdf V and E potential end

xmf output beg
xmf output end

output end

simulation end
simulation time : 166 (sec)

Simulation completed

```

### 7.3.5 Step 5. Post-processing Analysis

In order to gain a quick, intuitive understanding of the initial results, the VTU and XMF files are loaded into Paraview as shown in Figure 32. Block 2, corresponding to the RF rails, is selected for the VTU and grid visualization. The initial Paraview colorscale should be adjusted to better observe the field minimum. By using Paraview’s “probe location” function, one can quickly ascertain that the minimum is  $74\ \mu\text{m}$  above the surface.

Using the Python post-processing interface, similar analysis can be performed. The file `makePlots.py` is provided to demonstrate plotting the electric field magnitude on a plane to visualize the field minimum as shown in Figure 33. Further, the fields along the  $x$ -axis are shown at the pseudopotential null height in Figure 34.

```

import nses
import numpy as np
from matplotlib import pyplot

fileName = 'ionTrap.in.h5'

electrodes = nses.getSimulationInfo(fileName, 'electrodes')

gridId = 0
x, y, z = nses.getSimulationInfo(fileName, 'grid', gridId)
numX = x.shape[0]
numY = x.shape[1]
numZ = x.shape[2]
electrodeId = 2
phi = nses.getPotentials(fileName, gridId, electrodeId)
Ex, Ey, Ez = nses.getElectricFields(fileName, gridId, electrodeId)

E = np.sqrt(np.power(Ex, 2) + np.power(Ey, 2) + np.power(Ez, 2))

idxY = np.where(np.unique(y) == 0)
idxZ = np.where(np.unique(z) == 74e-6)
fig1 = pyplot.figure()
pyplot.plot(np.squeeze(x[:, idxY, idxZ])/1e-6, np.squeeze(Ex[:, idxY, idxZ]))

```



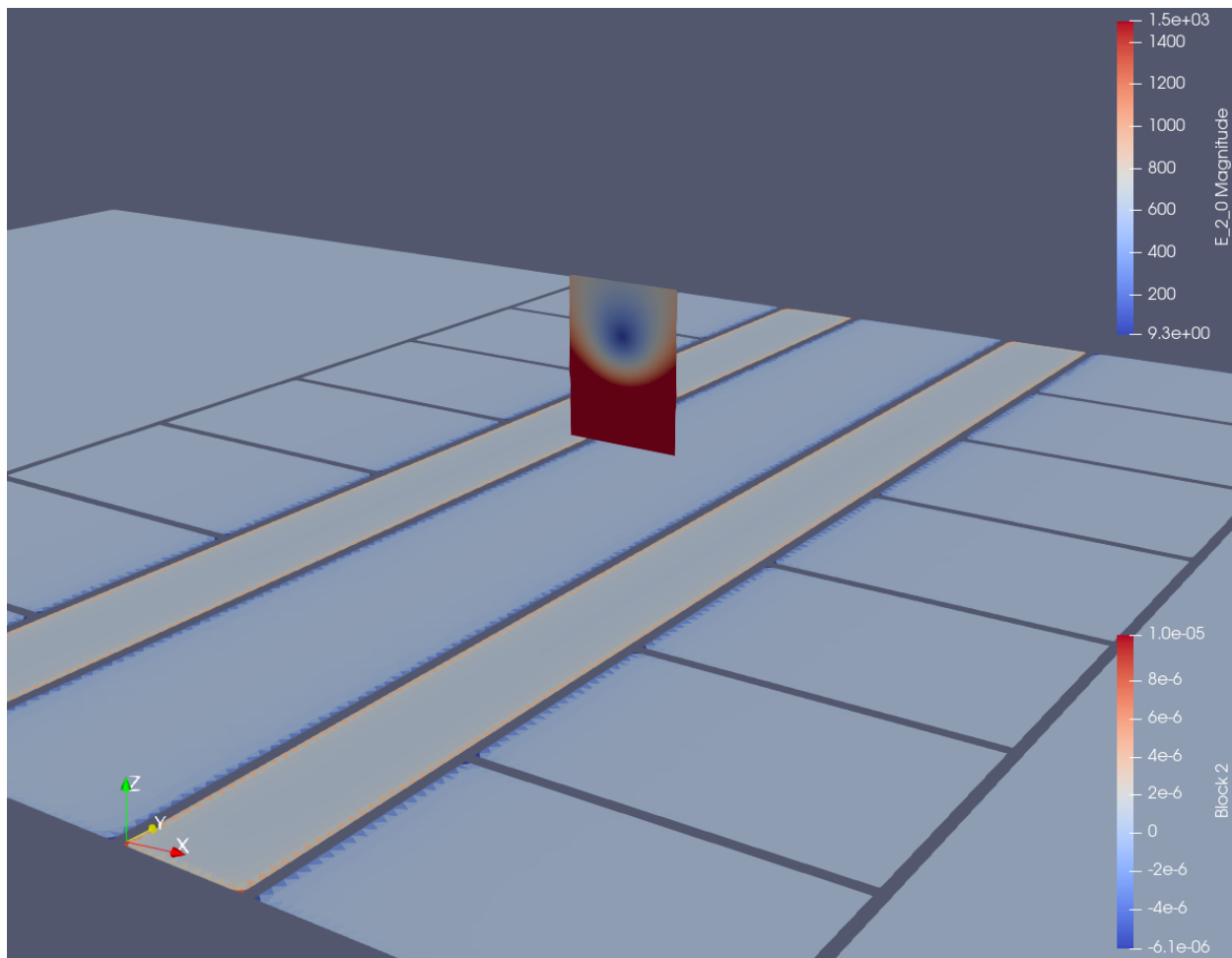


Figure 32: Paraview visualization of RF rail induced charge density and E-field magnitude.

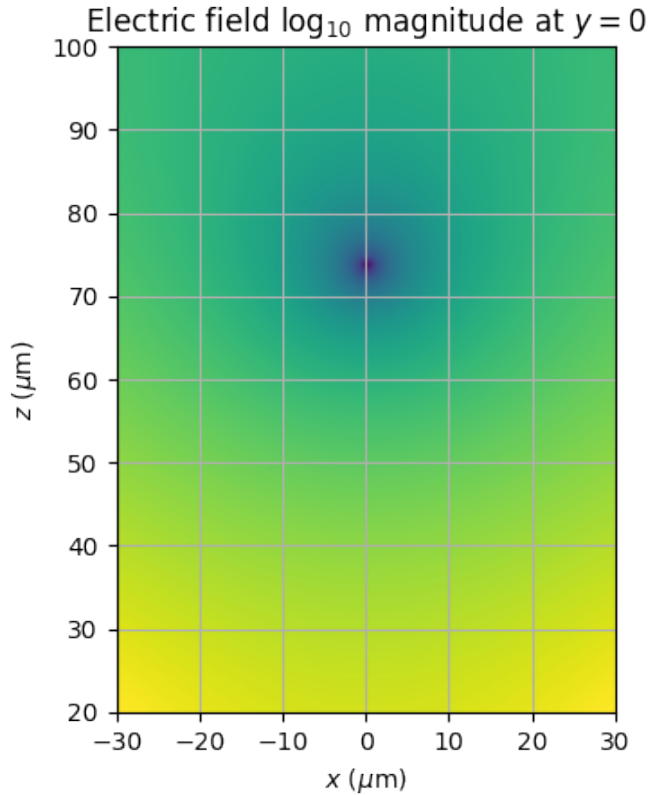


Figure 33: Python visualization of the electric field magnitude along the  $x = 0$  plane.

```

pyplot.plot(np.squeeze(x[:, idxY, idxZ])/1e-6, np.squeeze(Ey[:, idxY, idxZ]))
pyplot.plot(np.squeeze(x[:, idxY, idxZ])/1e-6, np.squeeze(Ez[:, idxY, idxZ]))
pyplot.plot(np.squeeze(x[:, idxY, idxZ])/1e-6, np.squeeze(E[:, idxY, idxZ]))
pyplot.xlabel('$x$ ($\mu$m)')
pyplot.ylabel('$E_{\alpha}$ (V/m)')
pyplot.title('Electric fields at $(y,z) = (0,74\mu$m)$')
pyplot.grid()
pyplot.legend(['$E_x$', '$E_y$', '$E_z$', '$E$'])
fig1.savefig('ionTrapFieldsX.png', transparent=True, bbox_inches='tight')

idxY = np.where(np.unique(y) == 0)
fig2 = pyplot.figure()
Exz = np.log10(np.squeeze(E[:, idxY, :]))
pyplot.imshow(np.transpose(Exz), interpolation='bilinear',
    extent = [x.min()/1e-6, x.max()/1e-6, z.min()/1e-6, z.max()/1e-6], origin='lower',
    vmin = Exz.min(), vmax = Exz.max())
pyplot.xlabel('$x$ ($\mu$m)')
pyplot.ylabel('$z$ ($\mu$m)')
pyplot.title('Electric field log$_{10}$ magnitude at $y = 0$')
pyplot.grid()
fig2.savefig('ionTrapExz.png', transparent=True, bbox_inches='tight')

pyplot.show(block=True)

```



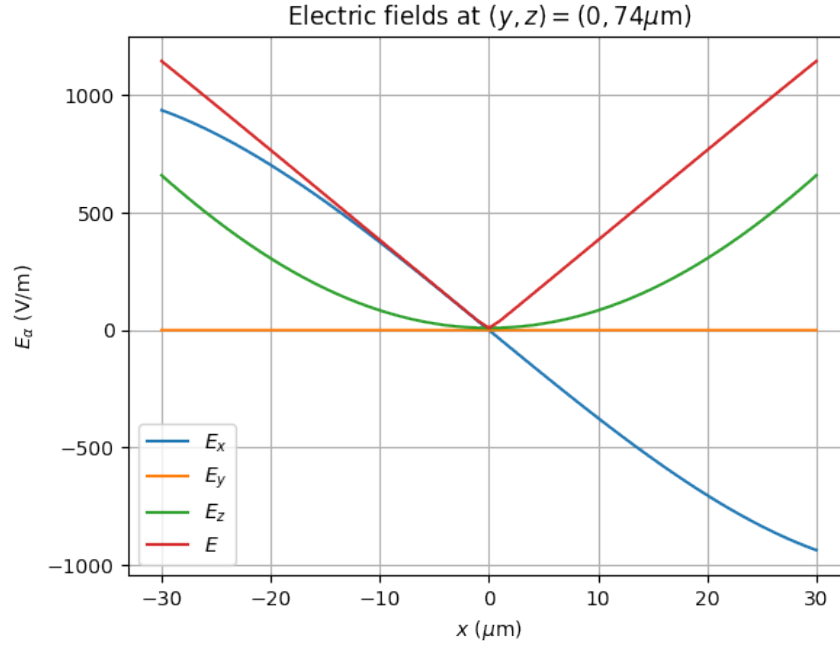


Figure 34: Electric field values along the  $x$  direction at the pseudopotential null height.

### 7.3.6 Step 6. Second Grid File

A user may wish to perform analysis on the behavior of an ion inside the pseudopotential well. Since the location of the minimum has been determined, a second grid file is generated with fine resolution at the height of the minimum along the length of the ion trap. The uniformity of the pseudopotential minimum is expected to degrade toward the ends of the trap, and computing this over a fine grid may aid in identifying useful operating positions for this trap. As such, the second grid file is generated as follows:

```
import numpy as np
import nses

fileName = 'ionTrapGridPP.h5'
x = np.linspace(-10, 10, 21)
y = np.linspace(-300, 300, 151)
z = np.linspace(64, 84, 21)
x = x*1e-6
y = y*1e-6
z = z*1e-6
nses.writeGrid(fileName, 0, '3DRECTMESH', x, y, z, True)
```

### 7.3.7 Step 7. Second Input Deck

For the second run, only the potentials, electric fields, and an XMF file are required. Therefore, the following input deck is created:

```
<OUTPUT_BEG>
  GRID = ionTrapGridPP.h5
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
```



```
XMF = ON
<OUTPUT_END>
```

### 7.3.8 Step 8. Nullspace ES Execution

At the command line, use the following syntax to generate the second data set:

```
>> nses ionTrapPP.in ionTrap.in.h5
```

The Nullspace ES output should look similar to the following.

```
*****
***                      Nullspace Electrostatics                      ***
***                      Nullspace, Inc.                                ***
***                      Copyright 2024                                ***
***                      Version - 2024 R1                             ***
*****

parsing precision from ionTrap.in.h5 ... done ... (single)
parsing prior simulation ionTrap.in.h5 ... done

parsing partial deck ionTrapPP.in ... done
parsing grid file ionTrapGridPP.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    symmetry rotation not applied
  processing symmetry rotation end
processing simulation base end
parsing mesh ionTrap.inp ... done
processing mesh ionTrap.inp ... done
  22114 nodes
  37510 elements
  59605 edges
  37510 unknowns
  17 surfaces

  minimum edge length is 1.90868e-06 owned by element 847
  maximum edge length is 8.05979e-05 owned by element 16646
  shape metric minimum (0.25) and maximum (1) passed for all triangles
  condition number metric minimum (1) and maximum (3) passed for all triangles

15 of 17 electrodes selected for excitation
2 4 5 6 7 8 9 10 11 12
13 14 15 16 17

xy symmetry : NONE : 0.000000
xz symmetry : NONE : 0.000000
yz symmetry : NONE : 0.000000
z-rotation symmetry (NONE)

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 5367.3 (MB) : 5.2 (GB) : 0.0 (TB)
memory required dense B (SINGLE) : 2.1 (MB) : 0.0 (GB) : 0.0 (TB)

hdf create simulation skeleton beg
  create pre process beg
  create pre process end
```



```

create post process beg
create post process end
hdf create simulation skeleton end

simulation beg

densities obtained from prior run

output beg

  hdf output density beg
    rho :      2
    rho :      4
    rho :      5
    rho :      6
    rho :      7
    rho :      8
    rho :      9
    rho :     10
    rho :     11
    rho :     12
    rho :     13
    rho :     14
    rho :     15
    rho :     16
    rho :     17
  hdf output density end

  vtu output density beg
    vtu output not specified
  vtu output density end

  hdf V and E potential beg
    V and E :      2 :      0 : 3DRECTMESH : (16 sec) (0 sec)
    V and E :      4 :      0 : 3DRECTMESH : (16 sec) (0 sec)
    V and E :      5 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :      6 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :      7 :      0 : 3DRECTMESH : (14 sec) (0 sec)
    V and E :      8 :      0 : 3DRECTMESH : (16 sec) (0 sec)
    V and E :      9 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     10 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     11 :      0 : 3DRECTMESH : (16 sec) (0 sec)
    V and E :     12 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     13 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     14 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     15 :      0 : 3DRECTMESH : (14 sec) (0 sec)
    V and E :     16 :      0 : 3DRECTMESH : (15 sec) (0 sec)
    V and E :     17 :      0 : 3DRECTMESH : (16 sec) (0 sec)
  hdf V and E potential end

  xmf output beg
  xmf output end

output end

simulation end
simulation time : 234 (sec)

Simulation completed

```



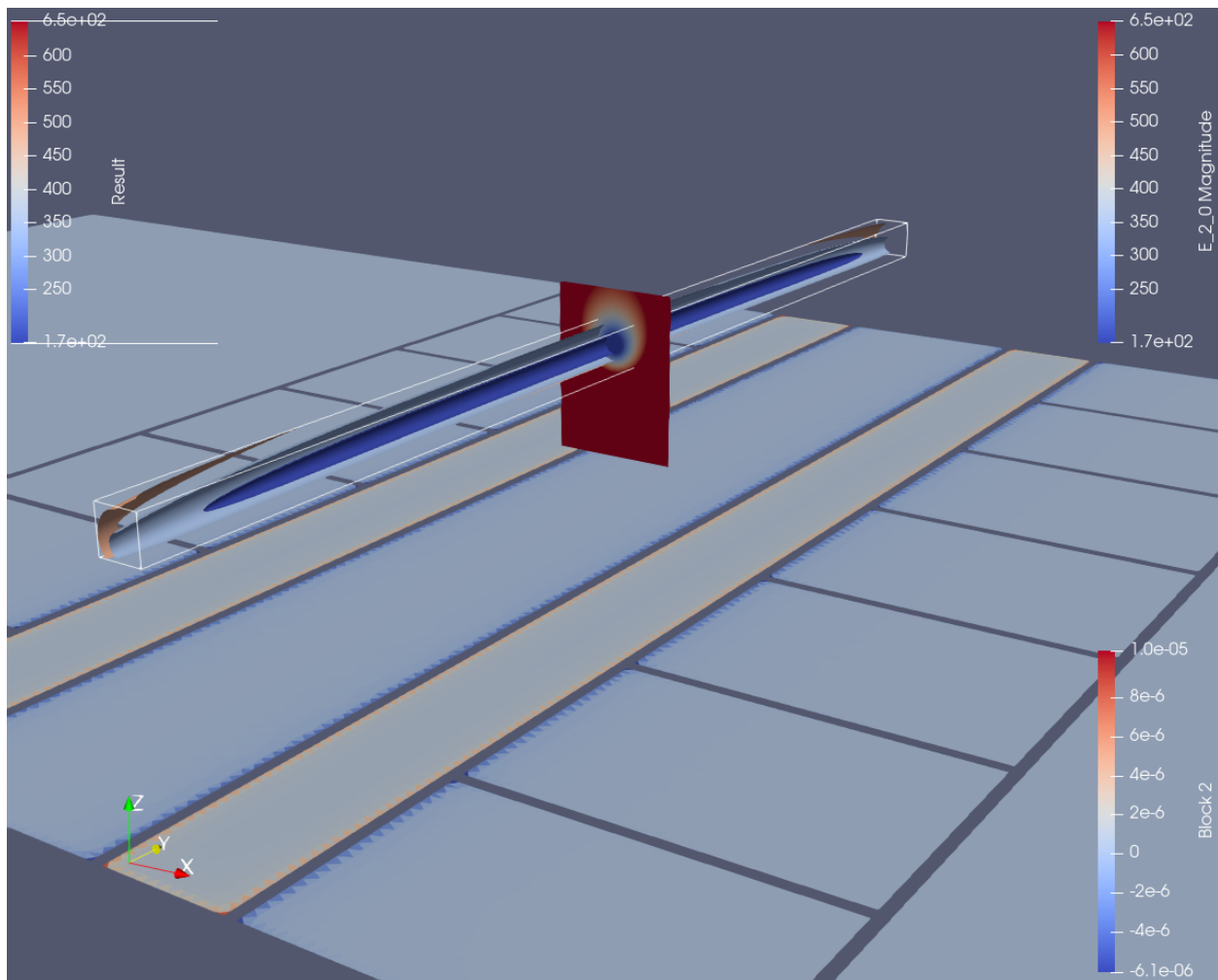


Figure 35: Paraview visualization of RF rail induced charge density and E-field magnitude.

### 7.3.9 Step 9. Secondary Post-processing Analysis

Once again, the Paraview is employed to visualize the new results. Loading the new ionTrapPP.in.xmf file into the same Paraview session as above, the field magnitude is visualized within the new grid using a sliced isosurface view as shown in Figure 35. Note that the color scale of the original grid's field slice has been adjusted to match the selected contour levels.

To visualize weighting the DC electrodes, Figure 36 shows isosurfaces from 0-40 mV in 10 mV intervals for  $V = 3(V_8 + V_9 + V_{12} + V_{13}) - 3.75(V_{10} + V_{11})$ .

A similar visualization (see Figure 37) may be obtained using the Python functions. In the following, the same weighted DC electrode potential is calculated and visualized along the  $yz$ -plane:

```
import nses
import numpy as np
from matplotlib import pyplot

fileName = 'ionTrapPP.in.h5'
```



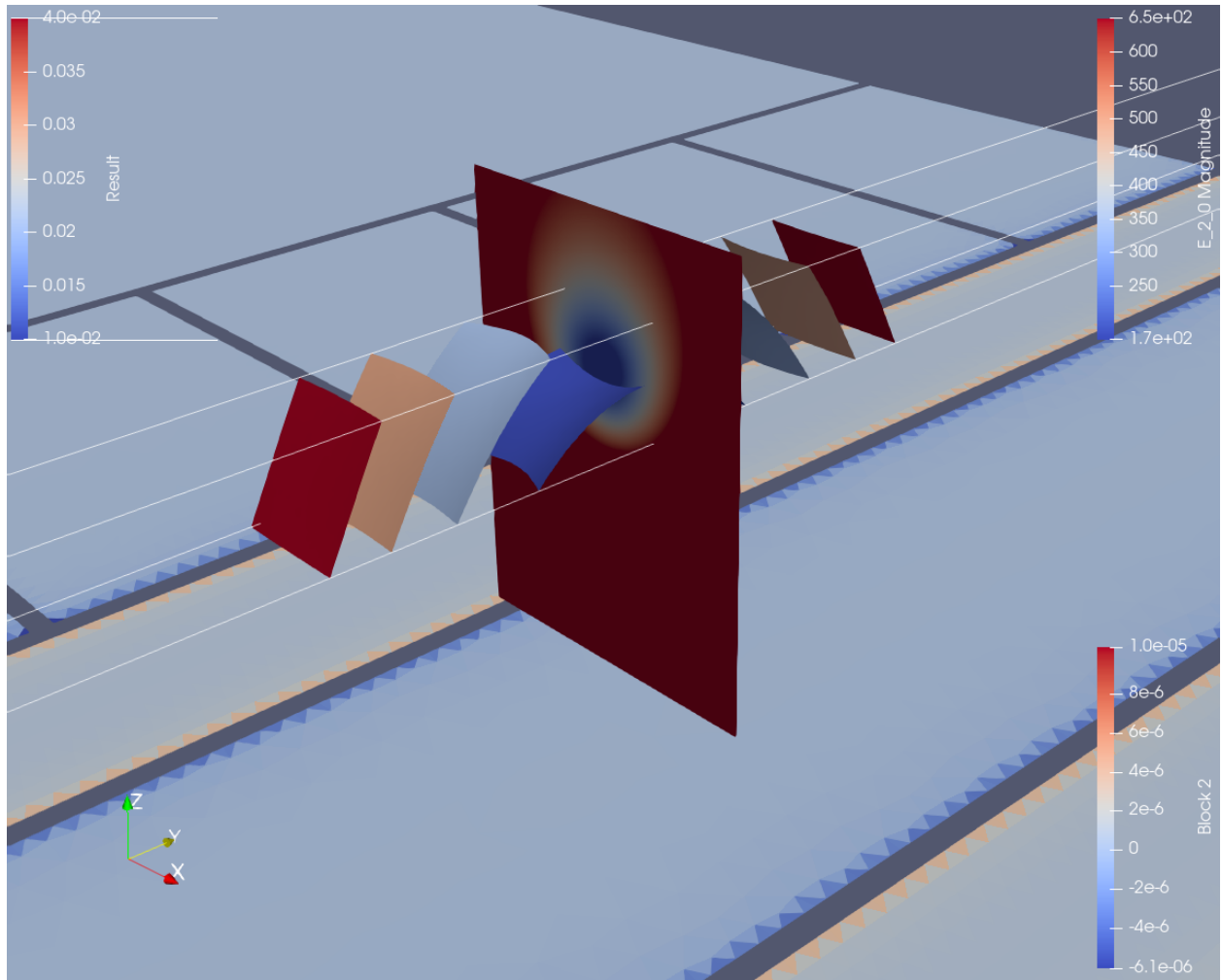


Figure 36: Paraview visualization of weighted DC electrode potential.



```

electrodes = nses.getSimulationInfo(fileName, 'electrodes')

gridId = 0
x, y, z = nses.getSimulationInfo(fileName, 'grid', gridId)
electrodeId = 2
phi = nses.getPotentials(fileName, gridId, electrodeId)

phi8 = nses.getPotentials(fileName, gridId, 8)
phi9 = nses.getPotentials(fileName, gridId, 9)
phi10 = nses.getPotentials(fileName, gridId, 10)
phi11 = nses.getPotentials(fileName, gridId, 11)
phi12 = nses.getPotentials(fileName, gridId, 12)
phi13 = nses.getPotentials(fileName, gridId, 13)

phiWell = 3*(phi8 + phi9) + -3.75*(phi10 + phi11) + 3*(phi12 + phi13)

idxX = np.where(np.unique(x) == 0)
yRng = np.unique(y)/1e-6
idxY = (yRng > -75) & (yRng < 75)
fig = pyplot.figure()
phiWellyz = np.squeeze(phiWell[idxX, idxY, :])
im = pyplot.imshow(np.transpose(phiWellyz), interpolation='bilinear',
    extent = [-75, 75, z.min()/1e-6, z.max()/1e-6], origin='lower',
    vmin = phiWellyz.min(), vmax = phiWellyz.max())
pyplot.xlabel('$y$ ($\\mu$m)')
pyplot.ylabel('$z$ ($\\mu$m)')
pyplot.title('Potential well at electrodes 10/11')
pyplot.grid()
fig.colorbar(im)
fig.savefig('ionTrapDC.png', transparent=True, bbox_inches='tight')

pyplot.show(block=True)

```



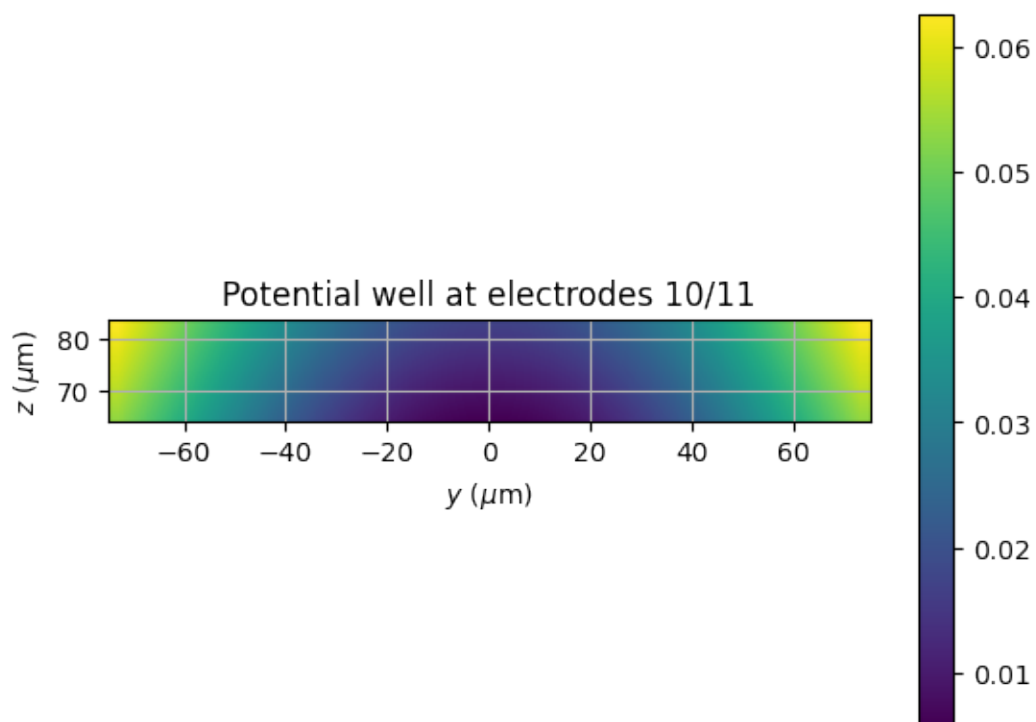


Figure 37: Python visualization of weighted DC electrode potential along  $yz$ -plane.

## 7.4 Ring Ion Trap

Ring-style ion traps exploit rotational symmetry to form ion chains. A number of unique properties of ring traps are presented in a recent dissertation by Urban<sup>2</sup>. This example follows the geometry presented in Urban's dissertation and highlights the utility of Nullspace ES's rotational symmetry feature.

### 7.4.1 Step 1. Nullspace Prep Mesh Model

The mesh model can be created in Nullspace Prep using the following Python file code snippet. The geometry exploits 45° rotational symmetry. Note that this geometry is suitable only for simulating the effect of the RF electrodes. Full control of the DC electrodes may require the assumption of no or limited symmetry depending on the application. The units of the mesh are microns. The geometry resulting from this python file is shown in Figure 31.

```
#!/python
#!/python
# Ring ion trap geometry derived from E. Urban, "Implementation of a Rotationally
  Symmetric Ring Ion
# Trap and Coherent Control of Rotational States", PhD Dissertation
# Accessed 2021-04-26 from https://escholarship.org/content/qt6nk6w90k/
  qt6nk6w90k_noSplash_0657e6a1b19efa7cb7ad7cb8040387fd.pdf

cubit.cmd('reset')

# Units are microns
innerRFRad = 125
innerGap = 15
gap = 25
centerGndRad = 600
outerRFRad = 1100
outerDCRad = 3000
outerGndRad = 4000
rotationAngle = 360/8

cubit.cmd(f'create surface circle radius {innerRFRad} zplane')
cubit.cmd(f'create surface circle radius {innerRFRad + innerGap} zplane')
innerGap = cubit.get_last_id('surface')

cubit.cmd(f'create surface circle radius {centerGndRad} zplane')
centerGnd = cubit.get_last_id('surface')
cubit.cmd(f'subtract surface {innerGap} from surface {centerGnd}')
centerGnd = cubit.get_last_id('surface')

cubit.cmd(f'create surface circle radius {centerGndRad + gap} zplane')
gapTool = cubit.get_last_id('surface')
cubit.cmd(f'create surface circle radius {outerRFRad} zplane')
outerRF = cubit.get_last_id('surface')
cubit.cmd(f'subtract surface {gapTool} from surface {outerRF}')
outerRF = cubit.get_last_id('surface')

cubit.cmd(f'create surface circle radius {outerRFRad + gap} zplane')
gapTool = cubit.get_last_id('surface')
cubit.cmd(f'create surface circle radius {outerDCRad} zplane')
```

<sup>2</sup>E. Urban, "Implementation of a Rotationally Symmetric Ring Ion Trap and Coherent Control of Rotational States", PhD Dissertation, 2019



```

outerDC = cubit.get_last_id('surface')
cubit.cmd(f'subtract surface {gapTool} from surface {outerDC}')
outerDC = cubit.get_last_id('surface')

cubit.cmd(f'create surface circle radius {outerDCRad + gap} zplane')
gapTool = cubit.get_last_id('surface')
cubit.cmd(f'create surface circle radius {outerGndRad} zplane')
outerGnd = cubit.get_last_id('surface')
cubit.cmd(f'subtract surface {gapTool} from surface {outerGnd}')
outerGnd = cubit.get_last_id('surface')

cubit.cmd(f'webcut body all with general plane y')
cutGeom = cubit.parse_cubit_list('surface', 'with y_max < 1e-6')
for s in cutGeom:
    cubit.cmd(f'delete surface {s}')

cubit.cmd(f'webcut body all with general plane y spin about axis direction 0 0 1 angle
45')
newOuterGnd = cubit.get_last_id('surface')

cubit.cmd(f'delete surface with x_min < -1e-6')

innerRF = newOuterGnd - 8
centerGnd = newOuterGnd - 6
outerRF = newOuterGnd - 4
dc = newOuterGnd - 2
outerGnd = newOuterGnd
surfaces = (innerRF, centerGnd, outerRF, dc, outerGnd)

curves = cubit.parse_cubit_list('curve', f'in surface {dc} with length < {outerDCRad-
outerRFRad} and length > {outerDCRad-outerRFRad-gap-1}')
for c in curves:
    cubit.cmd(f'tweak curve {c} offset {-gap/2}')

cubit.cmd(f'block 1 surface {innerRF} {outerRF}')
cubit.cmd(f'block 2 surface {centerGnd} {dc} {outerGnd}')

cubit.cmd('curve all scheme bias')
cubit.cmd(f'curve 55 60 61 66 67 72 73 78 71 36 size {gap/2}')
cubit.cmd('surface all scheme trimesh')
cubit.cmd('mesh surface all')

cubit.cmd('compress node ')
cubit.cmd('compress tri')

cubit.cmd('export abaqus "ringTrap.inp" block all dimension 3 overwrite')

```

## 7.4.2 Step 2. Grid File

For the initial analysis, the primary interest is in locating the pseudopotential (i.e., electric field magnitude) minimum due to the RF rails (mesh block 1). To simplify the analysis, a 3DRECTMESH grid is created at the center of the trap geometry.

```

import numpy as np
import nses as nses
import math

```



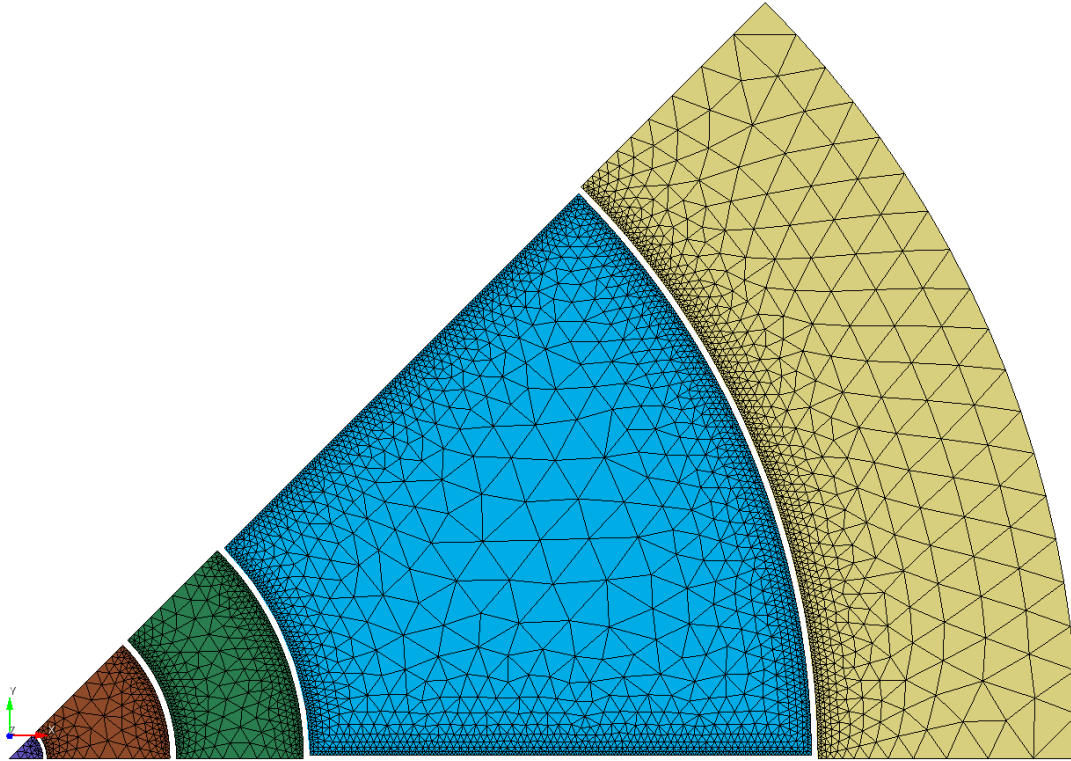


Figure 38: Ring ion trap geometry with  $45^\circ$  rotational symmetry.

```

fileName = 'grid.h5'

x = np.linspace(-100, 100, 51)
y = np.linspace(-100, 100, 51)
z = np.linspace(350, 420, 21)
x = x*1e-6
y = y*1e-6
z = z*1e-6
nses.writeGrid(fileName, 1, '3DRECTMESH', x, y, z, True)

```

### 7.4.3 Step 3. Input Deck

The trap mesh has 6,879 triangle elements. A modest laptop or low-end work station will easily have RAM to solve this problem as a dense linear system. The Python geometry script grouped both RF electrodes into the first block and all other electrodes into the second block. Since we are only interested in the RF electrode electric fields, the excitation is limited to block 1.

This input deck will compute the surface charge densities ( $\rho$ ) and store the results in the HDF5 file. It will also compute (and store in the HDF5 file) the potential ( $\phi$ ) and E-fields ( $\mathbf{E}$ ) for the electrodes and the specified grid. VTU and XMF output is on for additional visualization options.

```

<MESH_BEG>
  FILE = ringTrap.inp
  SCALE = 1e-6

```



```

<MESH_END>

<ELECTRODE_EXCITATION_BEG>
  LIST = 1
<ELECTRODE_EXCITATION_END>

<SYMMETRY_BEG>
  SYMMETRY_ROTATION_MODE = EVEN
  SYMMETRY_ROTATION_Z = 45.0
  SYMMETRY_PLANE_XY = ODD
  SYMMETRY_PLANE_XY_OFFSET = -200e-6
<SYMMETRY_END>

<SOLVER_BEG>
  SYSTEM = DENSE
<SOLVER_END>

<OUTPUT_BEG>
  POTENTIAL = ON
  ELECTRIC_FIELD = ON
  GRID = grid.h5
  CHARGE_VTU = ON
  XMF = ON
<OUTPUT_END>

```

#### 7.4.4 Step 4. Nullspace ES Execution

At the command line, navigate to the location of the input deck and invoke Nullspace ES as follows:

```
>> nses ringTrap.in
```

The Nullspace ES output should look similar to the following.

```

*****
***                               Nullspace Electrostatics          ***
***                               Nullspace, Inc.                    ***
***                               Copyright 2024                     ***
***                               Version - 2024 R1                  ***
*****

parsing precision from ringTrap.in ... done ... (single)
parsing full deck ringTrap.in ... done
parsing grid file grid.h5 ... done
processing simulation base beg
  processing symmetry rotation beg
    (even) : 8 : 45.000000
  processing symmetry rotation end
processing simulation base end
parsing mesh ringTrap.inp ... done
processing mesh ringTrap.inp ... done
  3952 nodes
  6879 elements
  10826 edges
  6879 unknowns
  2 surfaces

  minimum edge length is 1.12038e-05 owned by element 6856

```



```

maximum edge length is 0.000234222 owned by element 187
shape metric minimum (0.25) and maximum (1) passed for all triangles
condition number metric minimum (1) and maximum (3) passed for all triangles

1 of 2 electrodes selected for excitation
1

xy symmetry : ODD : -0.000200
xz symmetry : NONE : 0.000000
yz symmetry : NONE : 0.000000
z-rotation symmetry (even) : 45.000000

memory available : 64019.4 (MB) : 62.5 (GB) : 0.1 (TB)
memory required dense Z (SINGLE) : 180.5 (MB) : 0.2 (GB) : 0.0 (TB)
memory required dense B (SINGLE) : 0.0 (MB) : 0.0 (GB) : 0.0 (TB)

hdf create simulation skeleton beg
  create pre process beg
  create pre process end
  create post process beg
  create post process end
hdf create simulation skeleton end

simulation beg

  B fill beg
  B fill end
  B fill time : 0 (sec)

  Z fill beg
    dense fill beg
    dense fill end
  Z fill end
  Z fill time : 57 (sec)

  J solve beg
    dense solve beg
    dense solve end
  J solve end
  J solve time : 0 (sec)

output beg

  hdf output density beg
    rho :      1
  hdf output density end

  vtu output density beg
  vtu output density end

  hdf V and E potential beg
    V and E :      1 :      1 : 3DRECTMESH : (35 sec) (0 sec)
  hdf V and E potential end

  xmf output beg
  xmf output end

output end

```



```
simulation end
simulation time : 93 (sec)

Simulation completed
```

#### 7.4.5 Step 5. Post-processing Analysis

The VTU and XMF files are loaded into Paraview as shown in Figure 39. Block 1, corresponding to the RF electrodes, is selected for the VTU and grid visualization. The initial Paraview colorscale should be adjusted to better observe the field minimum. Additional post-processing is provided through the included `makePlots.py` Python script. The pseudopotential height map plot in Figure 40 is shown with code given in `makePlots.py`.





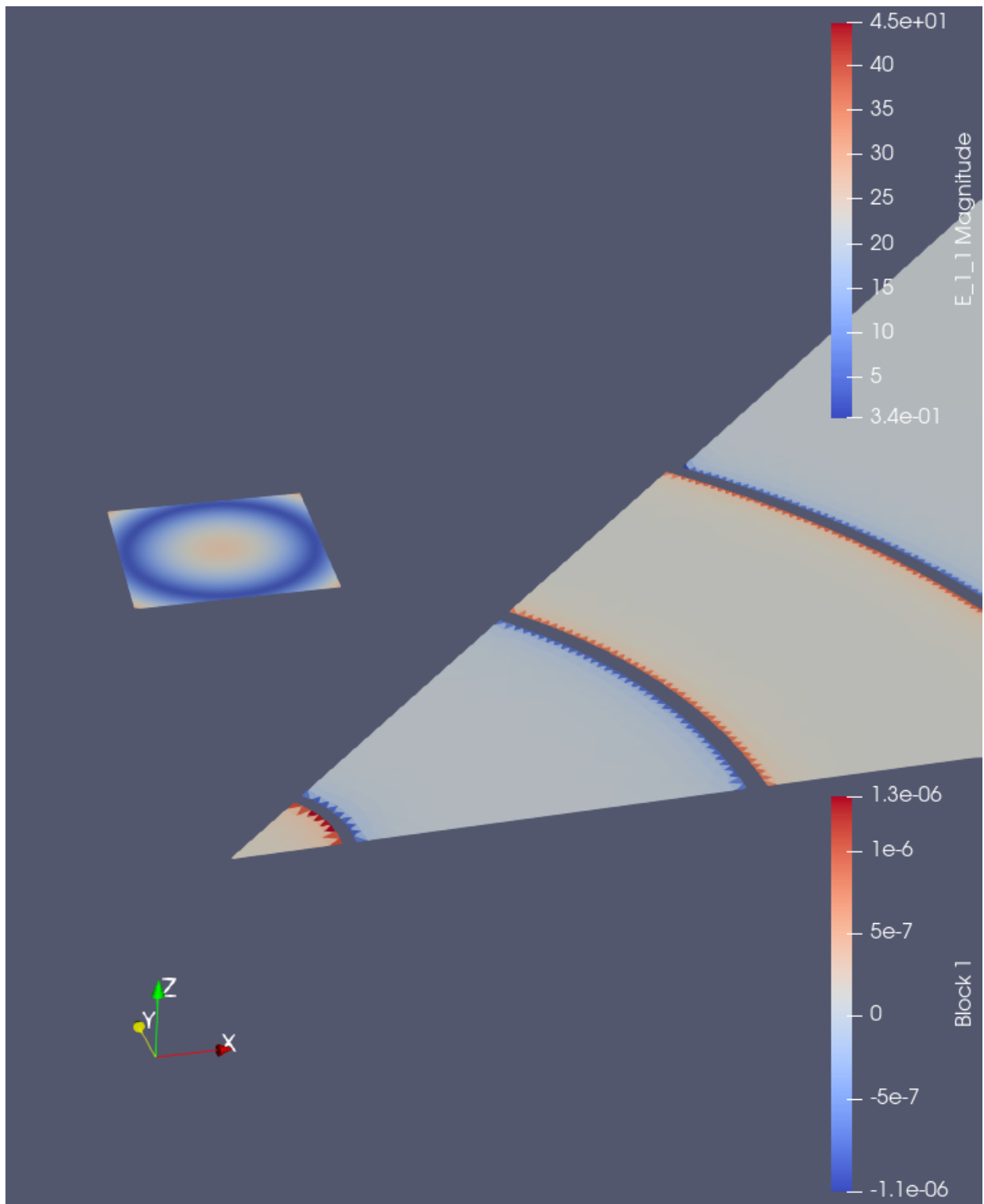


Figure 39: Paraview visualization of RF rail induced charge density and E-field magnitude.

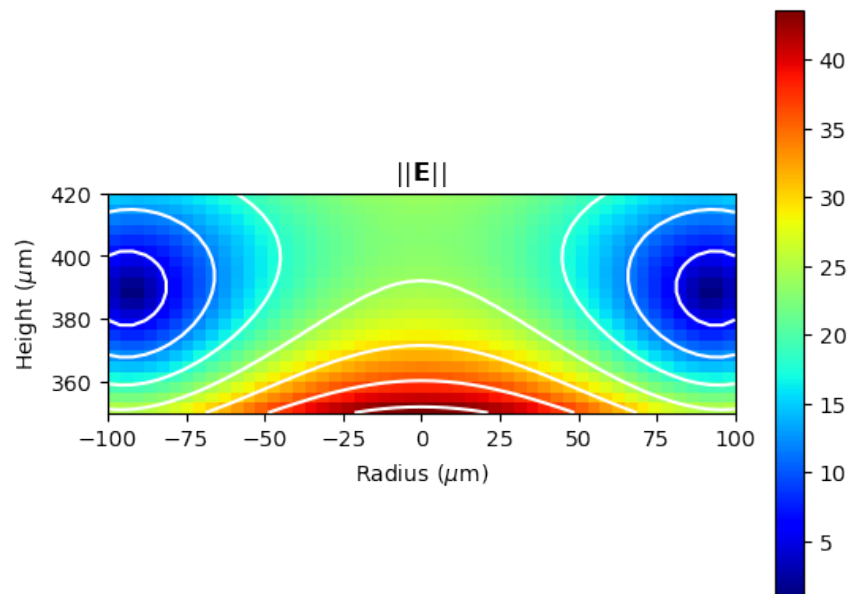


Figure 40: Python visualization of RF rail induced E-field (pseudopotential) height map.

## A Nullspace Prep Meshing

Nullspace ES simulations are dependent upon a mesh file that must be generated with external resources. A common mesh format supported is the Nullspace Prep Abaqus file. As discussed in prior sections, Nullspace ES echoes mesh statistics when it loads the mesh into memory. A sample output is shown below.

```
parsing mesh ionTrap.inp ... done
processing mesh ionTrap.inp ... done
  22602 nodes
  38486 elements
  61069 edges
  38486 unknowns
  17 surfaces

minimum edge length is 2.49996e-06 owned by element 9998
maximum edge length is 4.07683e-05 owned by element 15155
shape metric minimum (0.25) and maximum (1) passed for all triangles
element (329) violates condition number metric maximum (3) with condition number
(3.693)
element (2006) violates condition number metric maximum (3) with condition number
(3.693)
```

Here are several tips on generating CEM friendly mesh models using Nullspace Prep.

- The mesh should be composed of the standard 3-node triangular elements. TriMesh is usually the best choice of meshing scheme.
- Each triangular mesh element represents a constant value for the surface charge density. Thus, the mesh should be dense enough such that a step-wise approximation to the charge density along a surface is sufficiently accurate. Generally, the charge density is singular at sharp conductor edges. The mesh should be refined along such edges and can gradually grow larger on smooth regions.
- Avoid geometries that have nearly overlapping mesh elements (e.g., folded triangle pairs that nearly fold onto one another.)
- Avoid high aspect ratio triangles (long skinny triangles). Note that Nullspace Prep will warn during the mesh generation process that poor triangles have been produced.
- Use the Nullspace Prep Mesh Quality Power Tools to perform mesh diagnostics as needed.
- It is often better to simply re-formulate the geometry creation in the problematic area rather than attempting to overcome difficulties with mesh tuning.

Some geometry does not lend itself well to constant size meshing. Such models may require Nullspace Prep's Geometry Adaptive meshing workflow. Here are some tips:

- Use Geometry Adaptive meshing when standard meshing techniques yield high aspect ratio triangles.
- Condition number smoothing will generally improve the aspect ratio of most triangles.
- Ideally, a triangle will have a condition number of 1.0 (corresponding to an equilateral triangle.)
- Append smoothing commands to a journal file after the mesh command.

**!WARNING!** Nullspace ES does not support duplicate nodes or non-contiguous node numbering in any mesh file.



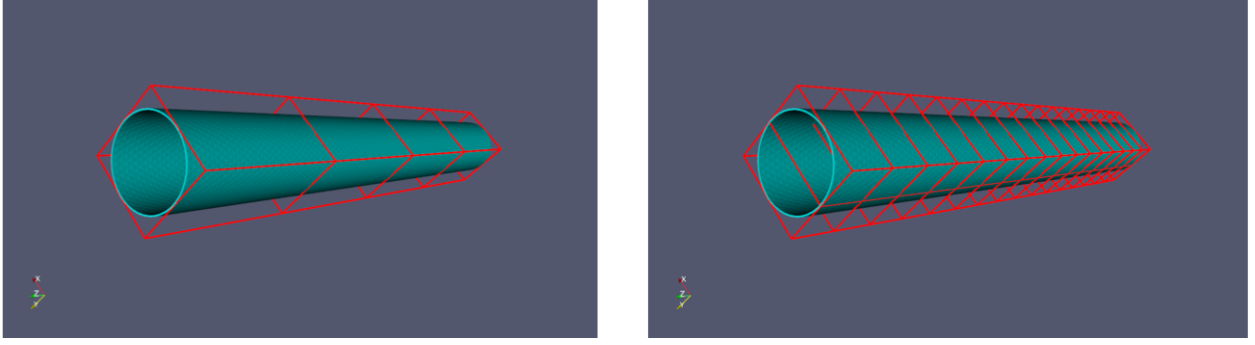


Figure 41: ACA grouping for an open pipe geometry.

## B Adaptive Cross Approximation (ACA)

Our goal here is not to give a discourse on ACA theory and its implementation in CEM related problems. There is a wealth of literature on the subject. Instead, we will only motivate aspects of the Nullspace ES ACA implementation and point out how ACA can be tuned within Nullspace ES to achieve a balance between accuracy and run-time performance.

The underlying structure of the MoM electrostatics linear system can lead to numerically rank-deficient sub-blocks. This corresponds to the interactions between two well-separated clusters (groups) of mesh elements. These low rank sub-blocks can then be approximated by the purely algebraic ACA algorithm.

Nullspace ES currently uses a simple binary tree block clustering / grouping scheme to isolate interactions between groups of mesh elements. The binary-tree cluster first determines a bounding cube of the meshed object and systematically halves the bounding cube until a minimal division criteria is met. Each new splitting produces  $2(l - 1)$  cubes where  $l \geq 1$  is the associated level.

In Figure 41 an open pipe (outer diameter = 4.0 in, inner diameter = 3.87 in, and length = 36.0 in) is triangle-meshed corresponding to 141,036 planar triangle elements (unknowns) assuming pulse basis functions. Figure 41 (left) illustrates the 3rd level of the binary tree splitting. Figure 41 (right) shows the 6th level of the binary tree splitting. The end result is to transform the element-to-element interaction problem into a cluster-to-cluster interaction problem.

Tolerances, as noted previously, are used to balance trade-offs to accuracy and performance of ACA. In essence, they control how well the cluster-to-cluster interaction MoM sub-blocks are approximated by low rank ACA matrices. Well separated clusters will typically yield low rank sub-blocks and thus very good ACA compression. Adjacent clusters can yield nearly full rank sub-blocks. In such cases very poor ACA compression would result. However, in practice, adjacent sub-blocks typically only slightly degrade the overall compression statistics.

### B.1 Troubleshooting

ACA, being an approximation technique, can diverge. The most common reason for divergence is usually due to poor mesh quality. ACA divergence will occur if the ACA blocks become poorly compressed (either in the ACA fill or ACA block LU solve algorithm.) ACA block compression can be found in the ACAProcessing.txt file. Divergence may be overcome by loosening ACA tolerances (at the cost of accuracy) or

isolating and correcting mesh issues. See the ACA related tolerance tags in the Input Deck section for more details.

## C Reduced Singular Value Decomposition (rSVD)

ACA is known to leave spurious linearly dependent rows/columns in the compressed block that can artificially inflate the rank and thus degrade the overall memory footprint and run-time. An alternative to ACA would be the Singular Value Decomposition (SVD). Though the SVD is an optimal rank reduction algorithm, it is prohibitively costly to use as a compression technique. One compromise is to use the Reduced Singular Value Decomposition (rSVD).

The rSVD technique operates on existing compressed  $UV$  pairs. For example, if ACA has already been used to find  $Z = U * V$  for a particular block, then it has been shown rSVD can further reduce the numerical rank of the  $UV$  pair at negligible costs and without loss in accuracy. Reducing the numerical rank of the  $UV$  pair leads to smaller memory footprints and thus faster run-times. The asymptotic costs of rSVD is  $O(k^2(m + n))$ , similar to ACA, though in practice rSVD is often considerably faster than ACA.

Nullspace ES uses rSVD as a post fill re-compression step with the `FILL_RECOMPRESS = RSVD` tag. Since integral equation evaluations dominate the run-time costs during fill, applying rSVD as a post fill re-compression step is inexpensive.

The core underlying block LU compressed update equation requires adding two compressed  $UV$  pairs. Research has shown rSVD can be an efficient means to perform this operation and it is often referred to as the rSVD "+" operator. Nullspace ES thus uses rSVD as an alternative block LU compression technique.

Tolerances, as noted previously, are used to balance trade-offs to accuracy and performance of rSVD. In essence, rSVD tolerances clip the internal SVD singular values used within the rSVD algorithm.

## D Nullspace ES Memory Requirements

CEM MoM based problems can be challenging to solve from a memory standpoint. The underlying reason being that MoM formulations produce dense inner linear systems.

Nullspace ES uses floating point arithmetic so that each entry in the MoM system is a real-valued number. This yields 4/8 bytes of memory for each entry for single/double precision usages. Thus, an estimate to the amount of memory required to fill a full MoM NUnknowns-by-NUnknowns single/double precision system using traditional techniques would be

$$(Z \text{ Memory}) = NUnknowns^2 * (4 \text{ or } 8) / 1024^3 \text{ (GB)}$$

Excitations / solutions consume memory as well, which can be estimated for single/double precision as

$$(RHS \text{ Memory}) = NUnknowns * NExcitations * (4 \text{ or } 8) / 1024^3 \text{ (GB)}$$

These two memory estimates constitute the bulk of Nullspace ES memory requirements and should be used as rule-of-thumbs to determine if a particular problem can be run on a given machine. The user is not required to perform these calculations by hand since NSES displays this information at the beginning of a simulation. However, Nullspace ES supports a variety of fill / solver techniques that can greatly alleviate these memory requirements.



The following workflow should be performed during building of the Nullspace ES input deck and identifying hardware in which to run the problem. These rules become simple mental checks after running several problems and obtaining a feel for mesh size.

- Nullspace ES currently uses scalar pulse basis functions. An exact number of required unknowns is  $N_{\text{Unknowns}} = N_{\text{Facets}}$ . Here  $N_{\text{Facets}}$  represents the number of facets in a given mesh model.
- Use the (Z Memory) and (RHS Memory) formulas above after determining  $N_{\text{Unknowns}}$  and  $N_{\text{Facets}}$ . Compare to the machine RAM.
- If the memory estimate is well within the machine RAM, always choose to use the Nullspace ES direct solve technique, provided the time required for dense solving is acceptable. Recall  $\text{SYSTEM} = \text{DENSE}$  by default.
- If the memory estimate exceeds the machine RAM, one should consider the ACA in-core workflow. Recall ACA will typically yield good matrix compression, especially for electrostatics problems. Assuming a nominal 95% compression, then  $(1.0 - 0.95) * (\text{Z Memory}) + (\text{RHS Memory})$  is a rough estimate on ACA required memory. If the compression memory estimate is well within the machine RAM, choose the  $\text{SYSTEM} = \text{DENSE}$ .

